Programación multihilo

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Introducción

Como hemos visto, los procesos son la unidad de tratamiento del sistema operativo. Un proceso, a su vez, se puede descomponer en otros subprocesos (ligth weight process) llamados hilos o threads. Cada uno de estos threads puede ejecutarse de forma concurrente o paralela en diferentes núcleos, incrementando el rendimiento del sistema.

> ### Ejemplo
>
> Podemos imaginar que un camarero nos prepara un café con leche y unas tostadas para desayunar. Si vemos esta labor como un proceso, estaremos de acuerdo en que el camarero trata de optimizar su tiempo, y no está a la espera de moler el café para ponerlo en la cafetera, aguarda a que acabe para empezar a calentar la leche, etc. Podríamos imaginar todo el proceso particionado en hilos de ejecución y que, mientras mete las rebanadas de pan en la tostadora, muele el café y sitúa una taza debajo de la cafetera... A la par que el agua hirviendo se deposita en la taza, calienta la leche y pone un plato con la cuchara y el azucarillo; en otro, la mantequilla, mermelada, etc. Cuando la leche ya se ha calentado, la vierte con maestría sobre el café.
> Al tiempo que lleva la taza, recoge las tostadas, las dispone en los platos y, en cuanto está todo listo, en la bandeja para ser servido, en su punto exacto.
> En el anterior proceso vemos cómo todo se descompone en hilos, aprovechando los tiempos de espera para iniciar, continuar o completar otras tareas, con el objetivo de minimizar el tiempo y los recursos.

Seguro que en más de una ocasión mientras te descargabas una imagen desde tu navegador web, seguías navegando por Internet e incluso iniciabas la descarga de un nuevo archivo, y todo esto ejecutándose el navegador como un único proceso, es decir, teniendo un único ejemplar del programa en ejecución.

Pues bien, ¿cómo es capaz de hacer el navegador web varias tareas a la vez? Seguro que estarás pensando en la programación concurrente, y así es; pero un nuevo enfoque de la concurrencia, denominado "programación multihilo". Justo lo que vamos a estudiar en esta unidad.

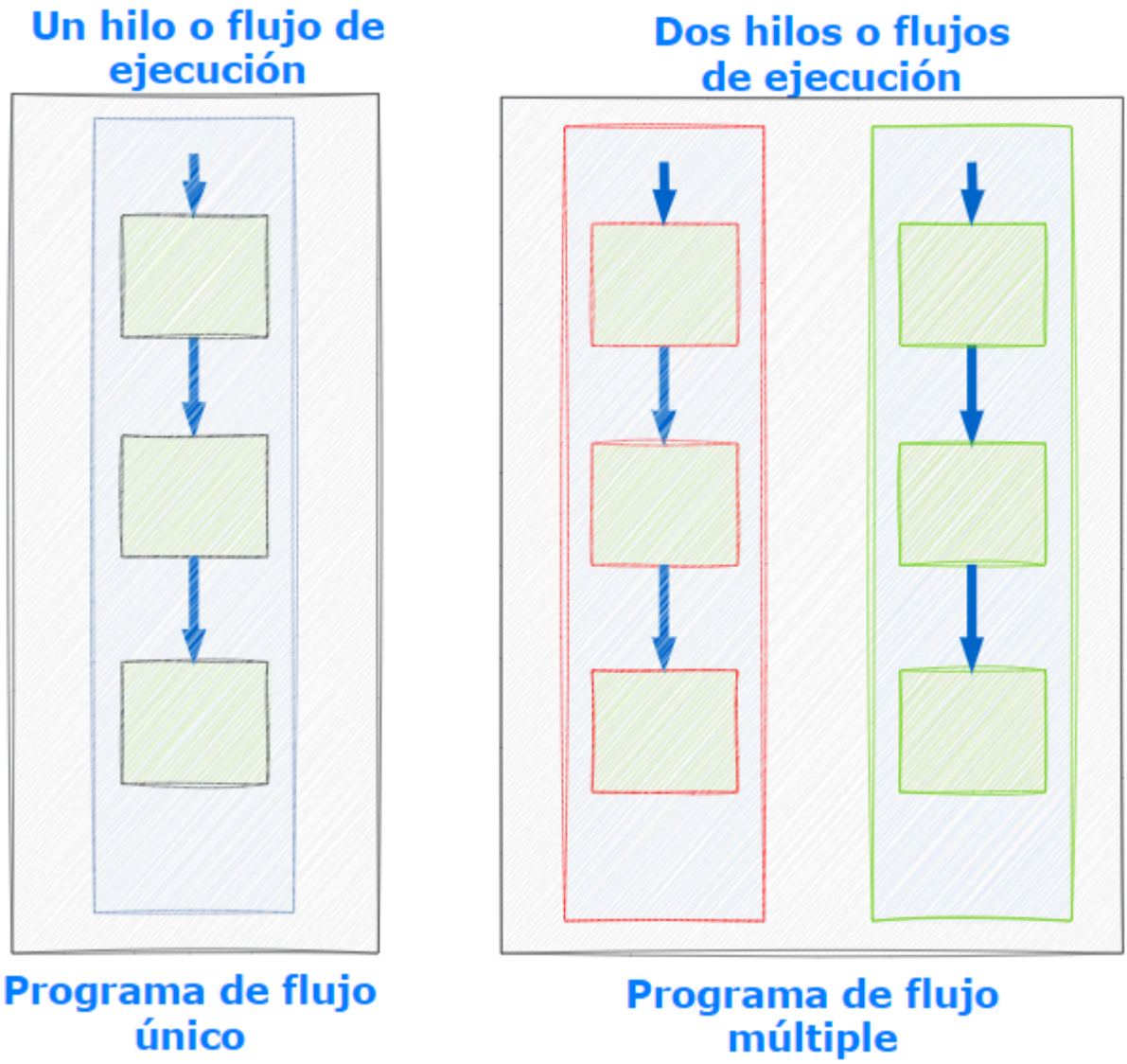
Los programas realizan actividades o tareas, y para ello pueden seguir uno o más flujos de ejecución. Dependiendo del número de flujos de ejecución, podemos hablar de dos tipos de

programas:

- **Programa de flujo único**. Es aquel que realiza las actividades o tareas que lleva a cabo una a continuación de la otra, de manera secuencial, lo que significa que cada una de ellas debe concluir por completo, antes de que pueda iniciarse la siguiente.
- **Programa de flujo múltiple**. Es aquel que coloca las actividades a realizar en diferentes flujos de ejecución, de manera que cada uno de ellos se inicia y termina por separado, pudiéndose ejecutar éstos de manera simultánea o concurrente.
  La programación multihilo o multithrearing consiste en desarrollar programas o aplicaciones de flujo múltiple. Cada uno de esos flujos de ejecución es un thread o hilo.

En el ejemplo anterior sobre el navegador web, un hilo se encargaría de la descarga de la imagen, otro de continuar navegando y otro de iniciar una nueva descarga. La utilidad de la programación multihilo resulta evidente en este tipo de aplicaciones. El navegador puede realizar "a la vez" estas tareas, por lo que no habrá que esperar a que finalice una descarga para comenzar otra o seguir navegando.

Cuando decimos "a la vez" recuerda que nos referimos a que las tareas se realizan concurrentemente, pues el que las tareas se ejecuten realmente en paralelo dependerá del Sistema Operativo y del número de procesadores del sistema donde se ejecute la aplicación. En realidad, esto es transparente para el programador y usuario, lo importante es la sensación real de que el programa realiza de forma simultánea diferentes tareas.



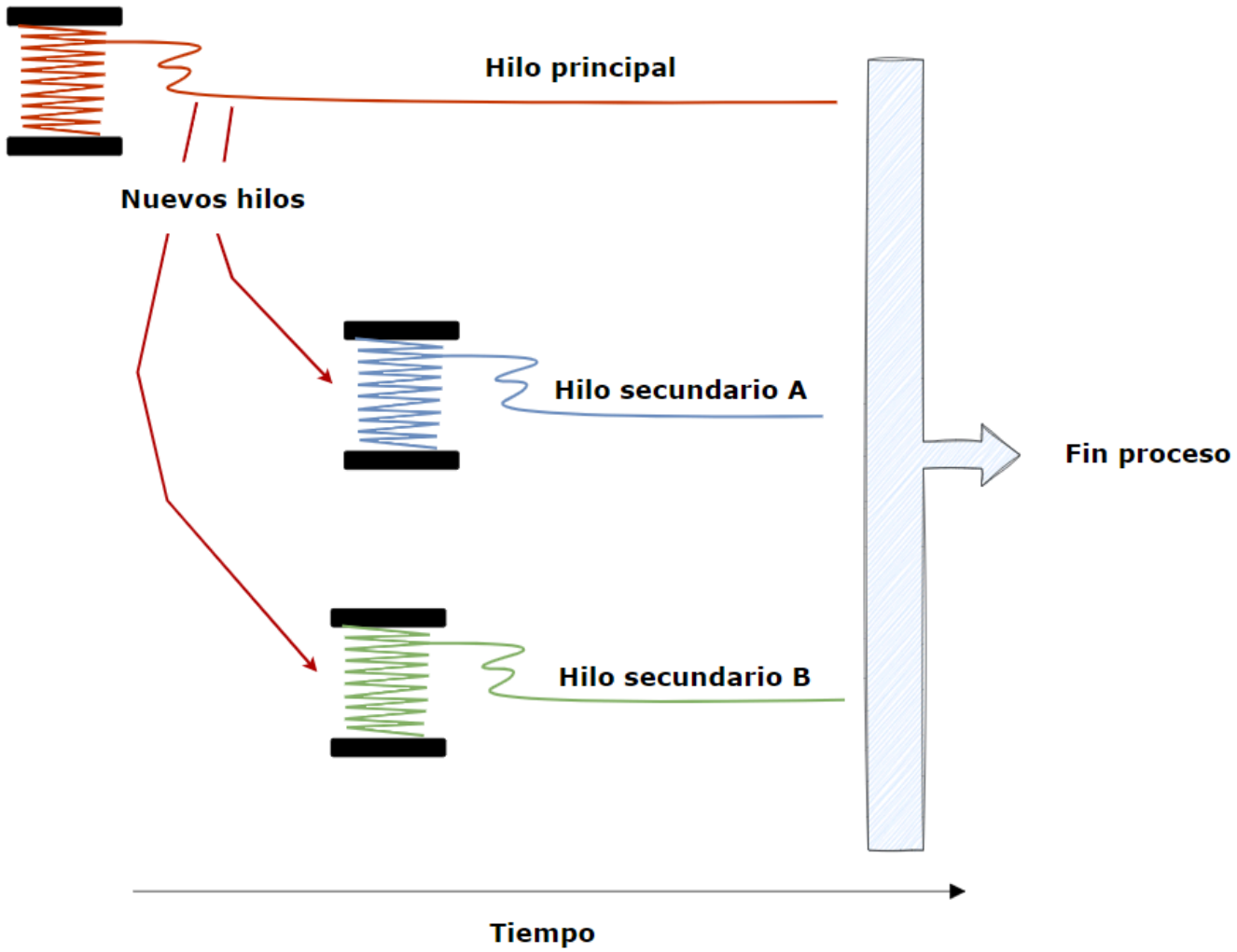
En el siguiente enlace puedes ver un ejemplo muy intuitivo de lo que se puede conseguir mediante programas de flujo múltiple o multihilo, en concreto mediante threads en Java.

Si un proceso está bien organizado en threads, este puede incrementar su ren. dimiento dado que, si uno de esos hilos quedase bloqueado a la espera de un recurso u operación de entrada-salida, podrían seguir en movimiento el resto de
El maneio de hilos se sostiene en tres patas:

- El hardware, que desempeña un papel importante, donde la optimización en la carga de los núcleos es fundamental.

- El sistema operativo desempeña otra función fundamental, dado que tiene que gestionar información relativa a cada hilo, pues, aunque comparten elementos comunes dentro del proceso, tienen otras propiedades particulares, como el estado, el punto de ejecución, etc.
- El programador, o persona que codifica los programas. Este individuo debe ejecutar unas labores de creatividad absoluta, ya que es el encargado de:
  - Particionar un programa o proceso en unidades más pequeñas que se puedan ejecutar en paralelo.
  - Determinar los recursos compartidos o críticos.
  - Disponer los mecanismos de sincronismo.
  - Establecer los dispositivos de bloqueo para el acceso a regiones de ejecución en exclusión.
  - Colocar las excepciones propias del paralelismo.
  - Probar, documentar, establecer la seguridad, etc.

En esta unidad nos vamos a basar en las cuestiones propias del programador, en las librerías y técnicas de descomposición de un programa en hilos, prestando atención a los errores más típicos para aprender a solventarlos en su implantación.
Un hilo solo puede existir dentro de un proceso, y no puede ejecutarse por sí solo. Por lo tanto, todo proceso tiene un hilo principal (main thread) del cual se pueden crear otros secundarios.



Un programa de un único hilo es secuencial; por lo tanto, si nos pide un dato, permanece paralizado hasta que lo introduzcamos. En cambio, un programa de varios hilos puede hacer cálculos mientras el usuario interactúa con la interfaz en un sistema basado en eventos y se está descargando un fichero, por ejemplo.

Ejemplo

Cuando ejecutamos un programa en Java o Python, hay muchos módulos que trabajan en paralelo, y lo hacen a través de threads. Se encargan, entre otras cosas, de transformar el lenguaje en alto nivel a un lenguaje intermedio o instrucciones básicas llamadas bytecodes. Esos bytecodes pueden almacenarse temporalmente en memoria o en ficheros con extensión .pyc (en el caso de Python) en una ruta especial. A su vez, hay una serie de hilos en lo que es la propia máquina virtual llamada JVM (Java) o PVM (Python) que interpretan y ejecutan los bytecodes línea a línea. Puede haber ligeras variaciones como JIT, que compila un subconjunto antes de ser ejecutado. Además, en la JVM o PVM existen hilos que lanzan el Grabage Collector para liberar la memoria que ya no se usa.

Explotando múltiples procesadores

Los sistemas multiprocesador solían ser caros y raros, encontrados solo en grandes centros de datos y en instalaciones de computación científica. Hoy en día son económicos y abundantes; incluso los sistemas de servidor de gama baja y los de escritorio de rango medio a menudo tienen múltiples procesadores. Esta tendencia solo se acelerará; a medida que se hace más difícil aumentar las velocidades de reloj, los fabricantes de procesadores en su lugar colocan más núcleos en un solo chip. Todos los principales fabricantes de chips han comenzado esta transición, y ya estamos viendo máquinas con un número significativamente mayor de procesadores. Dado que la unidad básica de planificación es el hilo, un programa con un solo hilo puede ejecutarse en un máximo de un procesador a la vez. En un sistema de dos procesadores, un programa de un solo hilo renuncia al acceso a la mitad de los recursos de CPU disponibles; en un sistema de 100 procesadores, renuncia al acceso al 99%. Por otro lado, los programas con múltiples hilos activos pueden ejecutarse simultáneamente en varios procesadores. Cuando están bien diseñados, los programas multihilo pueden mejorar el rendimiento al utilizar de manera más eficaz los recursos del procesador disponibles. El uso de múltiples hilos también puede ayudar a mejorar el rendimiento en sistemas de un solo procesador. Si un programa es de un solo hilo, el procesador permanece inactivo mientras espera a que se complete una operación de E/S sincrónica. En un programa multihilo, otro hilo puede seguir ejecutándose mientras el primer hilo espera a que se complete la E/S, lo que permite que la aplicación siga avanzando durante la E/S bloqueante. (Esto es como leer el

periódico mientras se espera a que el agua hierva, en lugar de esperar a que el agua hierva antes de empezar a leer).

Simplicidad de modelado

A menudo es más fácil administrar tu tiempo cuando solo tienes un tipo de tarea que realizar (arreglar estos doce errores) que cuando tienes varias (arreglar errores, entrevistar candidatos para reemplazar al administrador del sistema, completar las evaluaciones de desempeño de tu equipo y crear las diapositivas para tu presentación de la próxima semana). Cuando solo tienes un tipo de tarea por hacer, puedes comenzar en la parte superior de la lista y seguir trabajando hasta agotar las tareas (o hasta agotarte tú); no tienes que gastar energía mental en decidir qué hacer a continuación. Lo mismo ocurre con el software: un programa que procesa un tipo de tarea secuencialmente es más fácil de escribir, menos propenso a errores y más fácil de probar que uno que gestiona múltiples tipos de tareas a la vez. Asignar un hilo a cada tipo de tarea o a cada elemento en una simulación crea la ilusión de secuencialidad y aísla la lógica del dominio de los detalles de la planificación, las operaciones intercaladas, la E/S asíncrona y las esperas de recursos. Un flujo de trabajo complejo y asíncrono puede descomponerse en varios flujos de trabajo más simples y síncronos, cada uno ejecutándose en un hilo separado e interactuando solo en puntos de sincronización específicos. Este beneficio es aprovechado a menudo por marcos como servlets o RMI (invocación de métodos remotos). El framework maneja los detalles de la gestión de solicitudes, la creación de hilos y el equilibrio de carga, despachando partes del manejo de la solicitud al componente de la aplicación adecuado en el momento adecuado del flujo de trabajo. Los desarrolladores de servlets no necesitan preocuparse por cuántas otras solicitudes se están procesando al mismo tiempo o si los flujos de entrada y salida de los sockets se bloquean; cuando se llama al método service de un servlet en respuesta a una solicitud web, puede procesar la solicitud de forma síncrona como si fuera un programa de un solo hilo. Esto puede simplificar el desarrollo de componentes y reducir la curva de aprendizaje para el uso de dichos frameworks.

Los hilos están en todas partes

Incluso si tu programa nunca crea explícitamente un hilo, los marcos pueden crear hilos en tu nombre, y el código llamado desde estos hilos debe ser seguro para hilos. Esto puede imponer una carga significativa de diseño e implementación sobre los desarrolladores, ya que desarrollar clases seguras para hilos requiere más cuidado y análisis que desarrollar clases que no lo son. Cada aplicación Java usa hilos. Cuando se inicia la JVM, crea hilos para tareas de mantenimiento de la JVM (recolección de basura, finalización) y un hilo principal para ejecutar el método main. AWT (Abstract Window Toolkit) y los frameworks de interfaz de usuario Swing crean hilos para gestionar eventos de la interfaz de usuario. Los marcos de componentes, como servlets y RMI, crean pools de hilos e invocan métodos de componentes en estos hilos. Si utilizas estas funcionalidades, como lo hacen muchos desarrolladores, debes estar familiarizado con la concurrencia y la seguridad de los hilos, ya que estos marcos crean

hilos y llaman a tus componentes desde ellos. Sería ideal creer que la concurrencia es una característica "opcional" o "avanzada" del lenguaje, pero la realidad es que casi todas las aplicaciones Java son multihilo, y estos marcos no te aíslan de la necesidad de coordinar adecuadamente el acceso al estado de la aplicación. Cuando un marco introduce la concurrencia en una aplicación, generalmente es imposible restringir la consciencia de concurrencia al código del marco, porque los marcos, por naturaleza, hacen callbacks a los componentes de la aplicación que, a su vez, acceden al estado de la aplicación. De manera similar, la necesidad de seguridad en hilos no termina con los componentes llamados por el marco, sino que se extiende a todas las rutas de código que acceden al estado del programa accedido por esos componentes. Por lo tanto, la necesidad de seguridad en hilos es contagiosa.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

¿Qué es la programación multihilo?

La programación multihilo consiste en ejecutar múltiples tareas dentro de un mismo proceso de forma concurrente. Antes de profundizar en esto, es importante entender la diferencia entre multiproceso y multihilo. Los sistemas operativos multitarea pueden ejecutar varias tareas al mismo tiempo, aunque esta simultaneidad solo es real en sistemas con más de un procesador. En un solo procesador, el sistema alterna entre tareas rápidamente, dando la impresión de que se ejecutan a la vez.

Cuando se ejecuta un proceso, utiliza el procesador y la memoria asignada. En sistemas multitarea, el sistema operativo alterna entre procesos, liberando la memoria y el procesador cada vez que cambia de proceso, lo que conlleva cierta carga de trabajo adicional para el sistema.

La programación multihilo permite dividir un proceso en varias tareas que se ejecutan en paralelo dentro del mismo proceso. A diferencia del multiproceso, en el multihilo no es necesario liberar la memoria o el procesador al cambiar entre tareas, lo que mejora el rendimiento al reducir la sobrecarga. El sistema operativo se encarga de gestionar cuándo se ejecuta cada tarea, pero todas comparten los mismos recursos del proceso.

La programación multihilo es útil cuando una aplicación tiene varias tareas independientes que requieren gran potencia de cálculo y no dependen entre sí. También es ventajosa en situaciones donde ciertas tareas podrían quedar bloqueadas, como en operaciones de lectura de disco, permitiendo que otras tareas sigan ejecutándose mientras una está en espera.

Un ejemplo de uso es cuando una aplicación accede a varias bases de datos o servidores al mismo tiempo. En lugar de esperar a que se complete una tarea para empezar otra, el multihilo permite hacer todas las solicitudes simultáneamente, mejorando el rendimiento. Otro caso común es en videojuegos, donde se puede dar prioridad a la renderización de gráficos mientras otros procesos, como la lógica del juego, se ejecutan en paralelo.

Sin embargo, no siempre es recomendable usar multihilos. En aplicaciones simples, el esfuerzo de implementar múltiples hilos puede no justificar los beneficios. En aplicaciones extremadamente complejas, el multihilo puede introducir una sobrecarga que degrade el rendimiento.

En resumen, la programación multihilo es una técnica eficaz para mejorar el rendimiento en aplicaciones con tareas simultáneas e independientes, y su uso se ve potenciado en sistemas con múltiples procesadores.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Conceptos sobre hilos

Pero ¿qué es realmente un hilo? Un hilo, denominado también subproceso, es un flujo de control secuencial independiente dentro de un proceso y está asociado con una secuencia de instrucciones, un conjunto de registros y una pila.

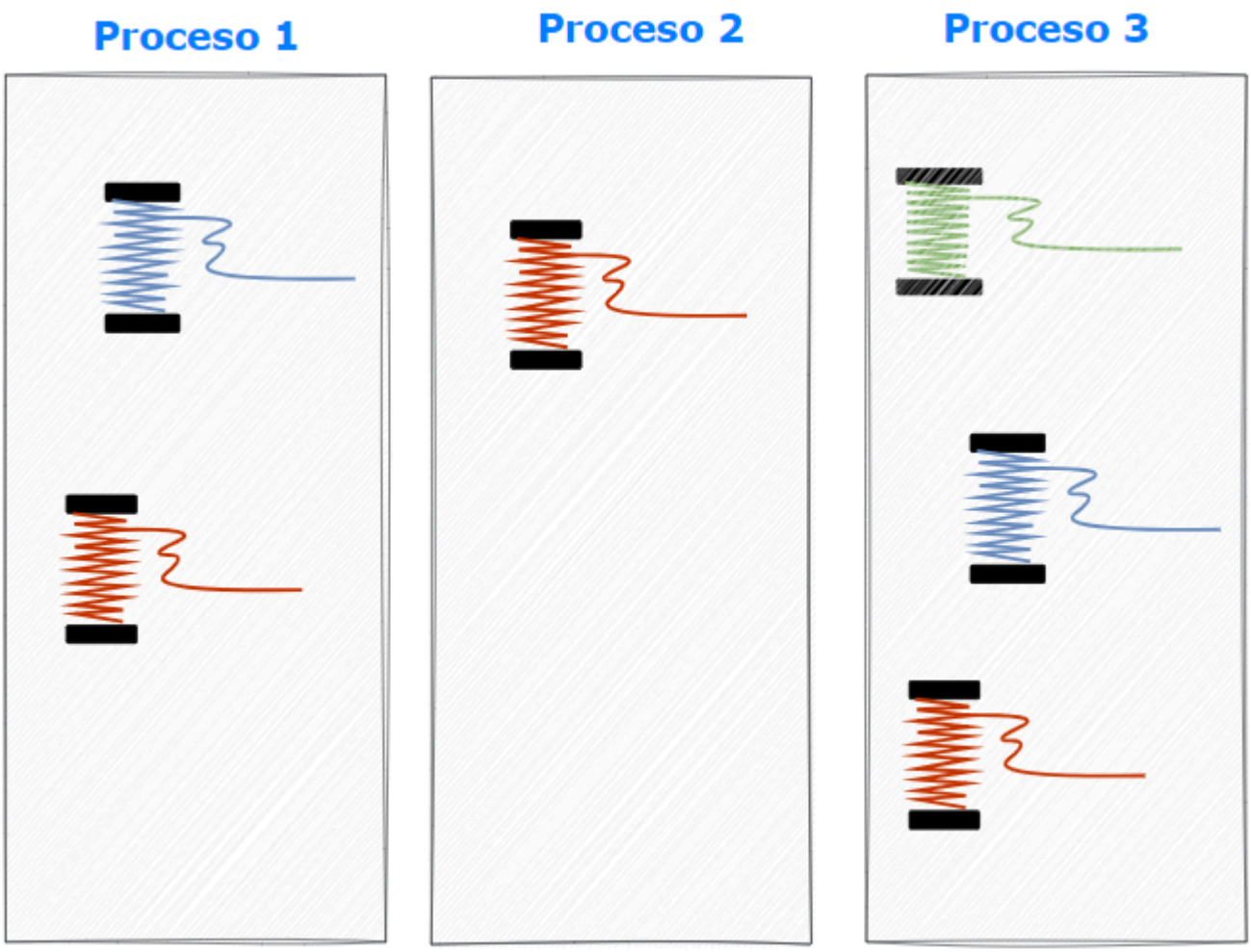
Cuando se ejecuta un programa, el Sistema Operativo crea un proceso y también crea su primer hilo, hilo primario, el cual puede a su vez crear hilos adicionales.

Desde este punto de vista, un proceso no se ejecuta, sino que solo es el espacio de direcciones donde reside el código que es ejecutado mediante uno o más hilos.

Por lo tanto podemos hacer las siguientes observaciones:

- Un hilo no puede existir independientemente de un proceso.
- Un hilo no puede ejecutarse por si solo.

Dentro de cada proceso puede haber varios hilos ejecutándose.
Un único hilo es similar a un programa secuencial; por si mismo no nos ofrece nada nuevo. Es la habilidad de ejecutar varios hilos dentro de un proceso lo que ofrece algo nuevo y útil; ya que cada uno de estos hilos puede ejecutar actividades diferentes al mismo tiempo. Así en un programa un hilo puede encargarse de la comunicación con el usuario, mientras que otro hilo transmite un fichero, otro puede acceder a recursos del sistema (cargar sonidos, leer ficheros, …), etc.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

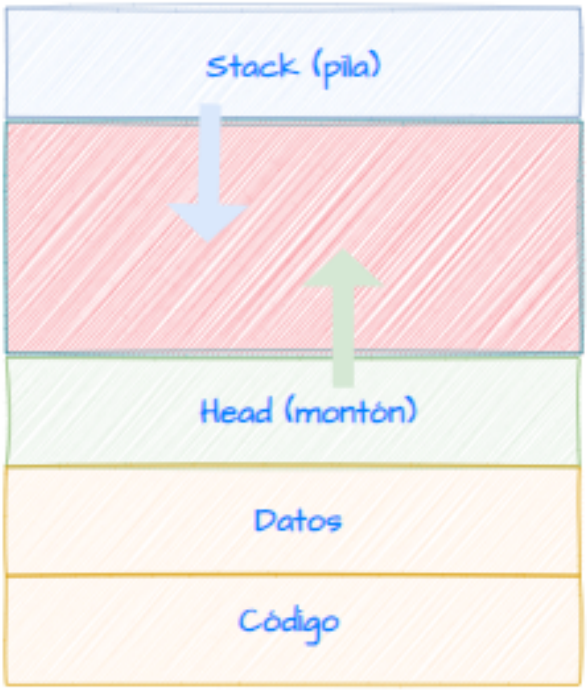
Recursos compartidos por los hilos.

Un hilo lleva asociados los siguientes elementos:

- Un identificador único.
- Atributos propios como nombre, prioridad, etc.
- Un contador de programa propio, es el contador específico de línea de ejecución.
- Un conjunto de registros.
- Una pila (variables locales).

Por otra parte, un hilo puede compartir con otros hilos del mismo proceso los siguientes recursos:

- Código.
- Datos (como variables globales).
- Stack o pila de proceso que se crece/decrece a medida que invocamos métodos o retornamos.
- Heap o memoria que se asigna al proceso dinámicamente en tiempo de ejecución.
- Otros recursos del sistema operativo, como los ficheros abiertos y las señales.



Seguro que te estarás preguntando "si los hilos de un proceso comparten el mismo espacio de memoria, ¿qué pasa si uno de ellos la corrompe?" La respuesta es, que los otros hilos también sufrirán las consecuencias. Recuerda que en el caso de procesos, el sistema operativo

normalmente protege a un proceso de otro y si un proceso corrompe su espacio de memoria los demás no se verán afectados

El hecho de que los hilos compartan recursos (por ejemplo, pudiendo acceder a las mismas variables) implica que sea necesario utilizar esquemas de bloqueo y sincronización, lo que puede hacer más difícil el desarrollo de los programas y así como su depuración.

Realmente, es en la sincronización de hilos donde reside el arte de programar con hilos; ya que de no hacerlo bien, podemos crear una aplicación totalmente ineficiente o inútil, como por ejemplo, programas que tardan horas en procesar servicios, o que se bloquean con facilidad y que intercambian datos de manera equivocada.

Profundizaremos más adelante en la sincronización, comunicación y compartición de recursos entre hilos dentro del contexto de Java.

Pero ¿qué ventajas aportan los hilos y cuando deben utilizarse? Eso es lo que vamos a ver precisamente en el siguiente apartado.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Ventajas y uso de hilos

Cuando se utilizan correctamente, los hilos pueden reducir los costos de desarrollo y mantenimiento, y mejorar el rendimiento de aplicaciones complejas. Los hilos facilitan la modelación de cómo las personas trabajan e interactúan, al convertir flujos de trabajo asíncronos en flujos mayormente secuenciales. También pueden transformar un código complicado en un código lineal que es más fácil de escribir, leer y mantener. Los hilos son útiles en aplicaciones GUI para mejorar la capacidad de respuesta de la interfaz de usuario y en aplicaciones de servidor para mejorar la utilización de recursos y el rendimiento. Además, simplifican la implementación de la JVM: el recolector de basura generalmente se ejecuta en uno o más hilos dedicados. La mayoría de las aplicaciones Java no triviales dependen en cierta medida de los hilos para su organización.

Como consecuencia de compartir el espacio de memoria, los hilos aportan las siguientes ventajas sobre los procesos:

- Se consumen menos recursos en el lanzamiento y la ejecución de un hilo que en el lanzamiento y ejecución de un proceso.
- Se tarda menos tiempo en crear y terminar un hilo que un proceso.
- La conmutación entre hilos del mismo proceso o cambio de contexto es bastante más rápida que entre procesos.

Es por esas razones, por lo que a los hilos se les denomina también procesos ligeros.

Y ¿cuándo se aconseja utilizar hilos? Se aconseja utilizar hilos en una aplicación cuando:

- La aplicación maneja entradas de varios dispositivos de comunicación.
- La aplicación debe poder realizar diferentes tareas a la vez.
- Interesa diferenciar tareas con una prioridad variada. Por ejemplo, una prioridad alta para manejar tareas de tiempo crítico y una prioridad baja para otras tareas.
- La aplicación se va a ejecutar en un entorno multiprocesador.

Por ejemplo, imagina la siguiente situación:

- Debes crear una aplicación que se ejecutará en un servidor para atender peticiones de clientes. Esta aplicación podría ser un servidor de bases de datos, o un servidor web.
- Cuando se ejecuta el programa éste abre su puerto y queda a la escucha, esperando recibir peticiones.

- Si cuando recibe una petición de un cliente se pone a procesarla para obtener una respuesta y devolverla, cualquier petición que reciba mientras tanto no podrá atenderla, puesto que está ocupado.
- La solución será construir la aplicación con múltiples hilos de ejecución.
- En este caso, al ejecutar la aplicación se pone en marcha el hilo principal, que queda a la escucha.
- Cuando el hilo principal recibe una petición, creará un nuevo hilo que se encarga de procesarla y generar la consulta, mientras tanto el hilo principal sigue a la escucha recibiendo peticiones y creando hilos.
- De esta manera un gestor de bases de datos puede atender consultas de varios clientes, o un servidor web puede atender a miles de clientes.
- Si el número de peticiones simultáneas es elevado, la creación de un hilo para cada una de ellas puede comprometer los recursos del sistema. En este caso, como veremos al final de la unidad lo resolveremos mejor con un pool de hilos.

Resumiendo, los hilos son idóneos para programar aplicaciones de entornos interactivos y en red, así como simuladores y animaciones.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Multihilo en Java

Tal y como ha comentado Juan, Java da soporte al concepto de hilo desde el propio lenguaje, con algunas clases e interfaces definidas en el paquete java.lang y con métodos específicos para la manipulación de hilos en la clase Object.

A partir de la versión JavaSE 5.0, se incluye el paquete java.util.concurrent con nuevas utilidades para desarrollar aplicaciones multihilo e incluso aplicaciones con un alto nivel de concurrencia.

En este enlace puedes consultar el tutorial oficial (en inglés) de Oracle sobre Hilos en Java.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Utilidades de concurrencia del paquete java.lang

Dentro del paquete java.lang disponemos de una interfaz y las siguientes clases para trabajar con hilos:

Tabla de Clases e Interfaces en Java

Nombre	Tipo	Descripción
Runnable	Interface	Esta interface debe implementarse por aquellas clases que se quieren ejecutar como un thread. Define un método sin argumentos run. Proporciona la capacidad de añadir la funcionalidad de hilo a una clase simplemente implementando la interfaz, en lugar de derivándola de la clase Thread.
Thread	Clase	Esta clase implementa la interface Runnable, siendo un hilo en sí misma. Una clase hereda de Thread y que sobrescribe el método run se ejecuta de manera concurrente si se invoca el método start. Con la clase Thread se puede envolver un objeto que implemente Runnable provocando que se ejecute en un hilo independiente. Es responsable de producir hilos funcionales para otras clases y proporciona gran parte de los métodos utilizados para su gestión.
ThreadDeath	Clase	Es una clase de error, deriva de la clase Error, y proporciona medios para manejar y notificar errores.
ThreadGroup	Clase	Esta clase se utiliza para manejar un grupo de hilos de modo conjunto, de manera que se pueda controlar su ejecución de forma eficiente.
Object	Clase	Esta clase no es estrictamente de apoyo a los hilos, pero proporciona unos cuantos métodos cruciales dentro de la arquitectura multihilo de Java. Estos métodos son wait(), notify() y notifyAll().
Timer	Clase	Permite la programación de la ejecución de tareas en diferido y de forma repetitiva mediante el método schedule. Este método espera tareas programables, representadas por objetos de la clase TimerTask, así como información del momento de inicio de la tarea y del tiempo que debe transcurrir entre cada una de las ejecuciones de la misma.

Nombre	Tipo	Descripción
TimerTask	Clase Abstracta	Heredando de esta clase y sobrescribiendo el método run se puede crear una tarea programable con la clase Timer.

En el siguiente enlace tienes una tabla resumida de la clase thread, en español, que incluye los métodos más comunes para gestionar y controlar hilos.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Utilidades de concurrencia del paquete java.util.concurrent.

El paquete java.util.concurrent incluye una serie de clases que facilitan enormemente el desarrollo de aplicaciones multihilo y aplicaciones complejas, ya que están concebidas para utilizarse como bloques de diseño.

Concretamente estas utilidades están dentro de los siguientes paquetes:

- java.util.concurrent. En este paquete están definidos los siguientes elementos:Clases de sincronización. Semaphore, CountDownLatch, CyclicBarrier y Exchanger.
- Interfaces para separar la lógica de la ejecución, como por ejemplo Executor, ExecutorService, Callable y Future.
- Interfaces para gestionar colas de hilos. BlockingQueque, LinkedBlokingQueque, ArrayBlockingQueque, SynchronousQueque, PriorityBl
- java.util.concurrent.atomic. Incluye un conjunto de clases para ser usadas como variables atómicas en aplicaciones multihilo y con diferentes tipos de dato, por ejemplo AtomicInteger y AtomicLong.
- java.util.concurrent.locks. Define una serie de clases como uso alternativo a la cláusula sinchronized. En este paquete se encuentran algunas interfaces como por ejemplo Lock, ReadWriteLock.

A lo largo de esta unidad estudiaremos las clases e interfaces más importantes de este paquete.

En este enlace puedes consultar detalladamente las utilidades proporcionadas por el paquete java.util.concurrent.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

La clase Thread

La clase Thread representa un hilo de ejecución. Cuando una clase hereda de Thread puede implementar el método run y ejecutarse de forma asíncrona.

Para lanzar un objeto Thread de manera asíncrona, basta ejecutar su método start.

El siguiente código muestra la creación y ejecución de un hilo básico utilizando la clase Thread.

Código Java



```
public class HiloBasico extends Thread {

    private int id;

    public HiloBasico(int id) {
        this.id = id;
    }

    public void run() {
        while (true) {
            System.out.println("Procesando hilo " + id);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    public static void main(String[] args) {
        for (int i = 0; i < 10; i++) {
            new Thread(new HiloBasico(i)).start();
        }
    }
```

```
}
```

Los métodos principales de la clase Thread se muesetran en la siguiente tabla:

Método	Descripción
start	Método de inicio del bloque asíncrono de la clase.
run	Método en el que se programa el bloque asíncrono. Se ejecuta cuando se invoca el método start.
join	Bloquea el hilo hasta que termina el hilo referenciado.
sleep	Método estático que detiene temporalmente la ejecución del hilo.
getId	Devuelve el identificador del hilo. Es un long positivo generado cuando se crea el hilo.
getName	Devuelve el nombre del hilo, asignado en algunas de las formas del constructor.
getState	Devuelve el estado del hilo como un valor de la enumeración Thread.State.
interrupt	Interrumpe la ejecución del hilo.
interrupted	Método estático que comprueba si el hilo actual ha sido interrumpido.
isInterrupted	Comprueba si el hilo en el que se encuentra ha sido interrumpido.
isAlive	Comprueba si el hilo está vivo.
setPriority	Cambia la prioridad del hilo. El valor debe estar entre las constantes MIN_PRIORITY y MAX_PRIORITY de la propia clase Thread.
setDaemon	Permite marcar el hilo como un hilo demonio.
isDaemon	Determina si un hilo es un hilo demonio.
yield	Método estático que indica al planificador que está dispuesto a ceder su uso actual de procesador. El planificador decide si atiende o no esta sugerencia.

Suspensión de ejecución: el método sleep

El método estático sleep de la clase Thread permiste suspender la ejecución del hilo desde el que se invoca. Acepta como parámetro la cantidad de tiempo en milisegundos que se desea realizar esta suspensión, cuya precisión queda sometida a la precisión de los temporizadores del sistema y al planificador.

Actividad propuesta: Hilos durmientes

- Crea una aplicación en Java que construya 5 hilos a partir de una única clase que herede de Thread.
- Cada hilo tiene nombre.
- Cada hilo, en su método run, tiene un bucle infinito. Dentro del bucle:
  - Escribe el texto: "Soy el bucle nombre y estoy trabajando".
  - Detiene la ejecución durante un período de tiempo aleatorio entre 1 y 10 segundos.

Interrupciones

En computación, una interrupción es una suspensión temporal de la ejecución de un proceso o de un hilo de ejecución. Las interrupciones no suelen pertenece a los programas, sino al sistema operativo, viniendo generadas por peticiones realizadas por los dispositivos periféricos.

En Java, una interrupción es una indicación a un hilo de que debe detener su ejecución para hacer otra cosa. Es responsabilidad del programador decidir qué quiere hacer ante una interrupción, siendo lo más habitual detener la ejecución del hilo.

Las interrupciones se capturan mediante la excepción InterruptedException, provocada por algunas operaciones como la invocación al método sleep. En caso de que la excepción no se tenga que controlar, se deberá invocar al método estático interrupted para gestionar la interrupción.

En el siguiente bloque de código se realiza la gestión de una excepción que se lanza de manera forzada ante cierta condición. En la captura de la excepción se invoca a la sentencia return para finalizar la ejecución del hilo.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Creación de hilos

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

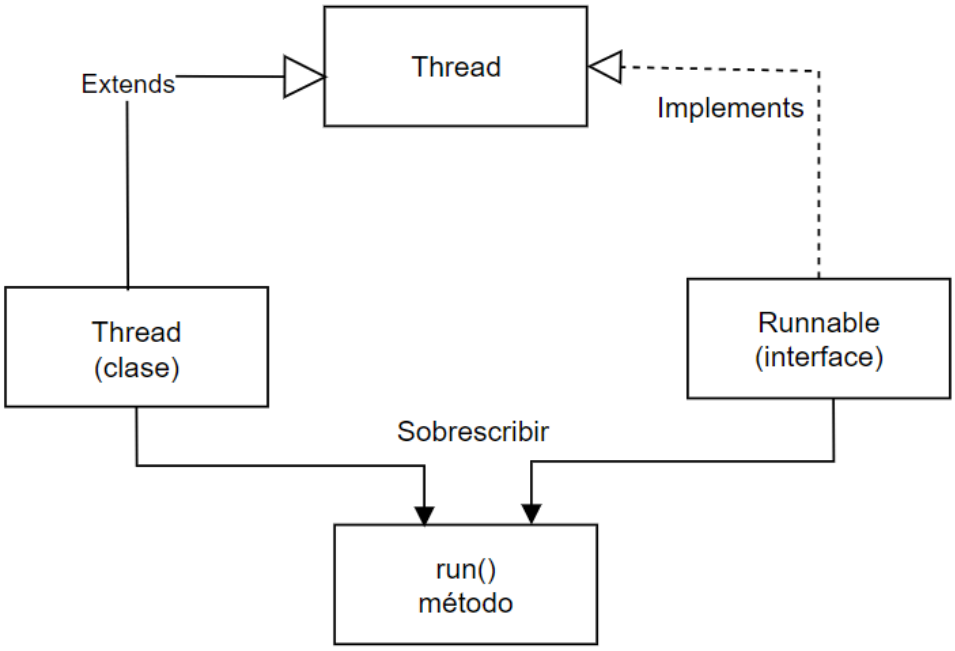
Creación de hilos en Java

En Java, un hilo se representa mediante una instancia de la clase java.lang.thread.
Este objeto thread se emplea para iniciar, detener o cancelar la ejecución del hilo de ejecución.

Los hilos o threads se pueden implementar o definir de dos formas:

- Extendiendo la clase thread.
- Mediante la interfaz Runnable .

En ambos casos, se debe proporcionar una definición del método run(), ya que este método es el que contiene el código que ejecutará el hilo, es decir, su comportamiento.



> El procedimiento de construcción de un hilo es independiente de su uso, pues una vez creado se emplea de la misma forma. Entonces, **¿cuando utilizar uno u otro procedimiento?**
>
> - **Extender** la clase **Thread** es el procedimiento más sencillo, pero no siempre es posible. Si la clase ya hereda de alguna otra clase padre, no será posible heredar también de la clase thread (recuerda que Java no permite la herencia múltiple), por lo que habrá que recurrir al otro procedimiento.
> - **Implementar Runnable** siempre es posible, es el procedimiento más general y también el más flexible.

Por ejemplo, piensa en la programación de applets, cualquiera de ellos tiene que heredar de la clase java.applet.Applet; y en consecuencia ya no puede heredar de thread si se quiere utilizar hilos. En este caso, no queda más remedio que crear los hilos implementando Runnable.

Cuando la Máquina Virtual Java (JVM) arranca la ejecución de un programa, ya hay un hilo ejecutándose, denominado hilo principal del programa, controlado por el método main(), que se ejecuta cuando comienza el programa y es el último hilo que termina su ejecución, ya que cuando este hilo finaliza, el programa termina.

El siguiente ejemplo muestra lo que te acabamos de comentar, siempre hay un hilo que ejecuta el método main(), y por defecto, este hilo se llama "main". Observa que para saber qué hilo se está ejecutando en un momento dado, el hilo en curso, utilizamos el método currentThread() y que obtenemos su nombre invocando al método getName(), ambos de la clase thread.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Creación de hilos extendiendo la clase Thread.

La forma más simple de crear un hilo es heredar de la clase Thread, que tiene todo lo necesario para crear y ejecutar hilos. El método más importante de Thread es **run( )**, que debe ser **sobrescrito** para hacer que el hilo haga lo que se le mande. Por consiguiente, run( ) es el código que se ejecutará "simultáneamente" con los otros hilos del programa.

Para definir y crear un hilo extendiendo la clase thread, haremos lo siguiente:

- Crear una nueva clase que herede de la clase thread.
- Redefinir en la nueva clase el método run() con el código asociado al hilo. Las sentencias que ejecutará el hilo.
- Crear un objeto de la nueva clase thread. Éste será realmente el hilo.

Una vez creado el hilo, para ponerlo en marcha o iniciarlo:

- Invocar al método start() del objeto thread (el hilo que hemos creado).

Un método run( ) suele tener siempre algún tipo de bucle que continúa hasta que el hilo deja de ser necesario, por lo que hay que establecer la condición en la que romper el bucle y salir (o, en el caso de arriba, simplemente return de run( )). A menudo, se convierte run( ) en forma de bucle infinito, lo que significa que a falta de algún factor externo que haga que run( ) termine, continuará para siempre.

En el siguiente ejemplo puedes ver los pasos indicados anteriormente para la creación de un hilo extendiendo la clase thread. El hilo que se crea (objeto thread hilo1) imprime un mensaje de saludo. Para simplificar el ejemplo se ha incluido el método main() que inicia el programa en la propia clase Saludo.



En el método main( ) se puede ver el número de hilos que se están creando y ejecutando. El método start( ) de la clase Thread lleva a cabo alguna inicialización especial para el hilo y después llama a run( ). Por tanto, los pasos son: se llama al constructor para que construya el

objeto, después start( ) configura el hilo y llama a run( ). Si no se llama a start( ) (lo que puede hacerse en el constructor si es apropiado) nunca se dará comienzo al hilo.

La salida de una ejecución de este programa (que será distinta cada vez) es:

```
Creado 1
Creado 2
Creado 3
Creado 4
Creado 5
Todos los hilos iniciados
Hilo 4(5)
Hilo 4(4)
Hilo 4(3)
Hilo 1(5)
Hilo 4(2)
Hilo 5(5)
Hilo 2(5)
Hilo 3(5)
Hilo 2(4)
Hilo 5(4)
Hilo 4(1)
Hilo 1(4)
Hilo 1(3)
Hilo 5(3)
Hilo 5(2)
Hilo 2(3)
Hilo 3(4)
Hilo 2(2)
Hilo 5(1)
Hilo 1(2)
Hilo 1(1)
Hilo 2(1)
Hilo 3(3)
Hilo 3(2)
Hilo 3(1)
```

Se verá que en este ejemplo no se llama nunca a sleep( ), y la salida sigue indicando que cada hilo obtiene una porción del tiempo de UCP en el que ejecutarse. Esto muestra que sleep( ), aunque descansa en la existencia de un hilo para poder ejecutarse, no está involucrado en la habilitación o deshabilitación de hilos. Es simplemente otro método.

También puede verse que los hilos no se ejecutan en el orden en el que se crean.
De hecho, el orden en que la UCP atiende a un conjunto de hilos existente es indeterminado, a menos que se cambien las prioridades haciendo uso del método setPriority( ) de Thread.
Cuando main( ) crea los objetos Thread no captura las referencias a ninguno de ellos.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Creación de hilos mediante la interfaz Runnable

Para definir y crear hilos implementando la interfaz Runnable seguiremos los siguientes pasos:

- Declarar una nueva clase que implemente a Runnable.
- Redefinir (o sombrear) en la nueva clase el método run() con el código asociado al hilo. Lo que queremos que haga el hilo.
- Crear un objeto de la nueva clase.
- Crear un objeto de la clase thread pasando como argumento al constructor, el objeto cuya clase tiene el método run(). Este será realmente el hilo.

Una vez creado el hilo, para ponerlo en marcha o iniciarlo:

- Invocar al método start() del objeto thread (el hilo que hemos creado).

El siguiente ejemplo muestra cómo crear un hilo implementado Runnable. El hilo que se crea (objeto thread hilo1) imprime un mensaje de saludo, como en el caso anterior.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

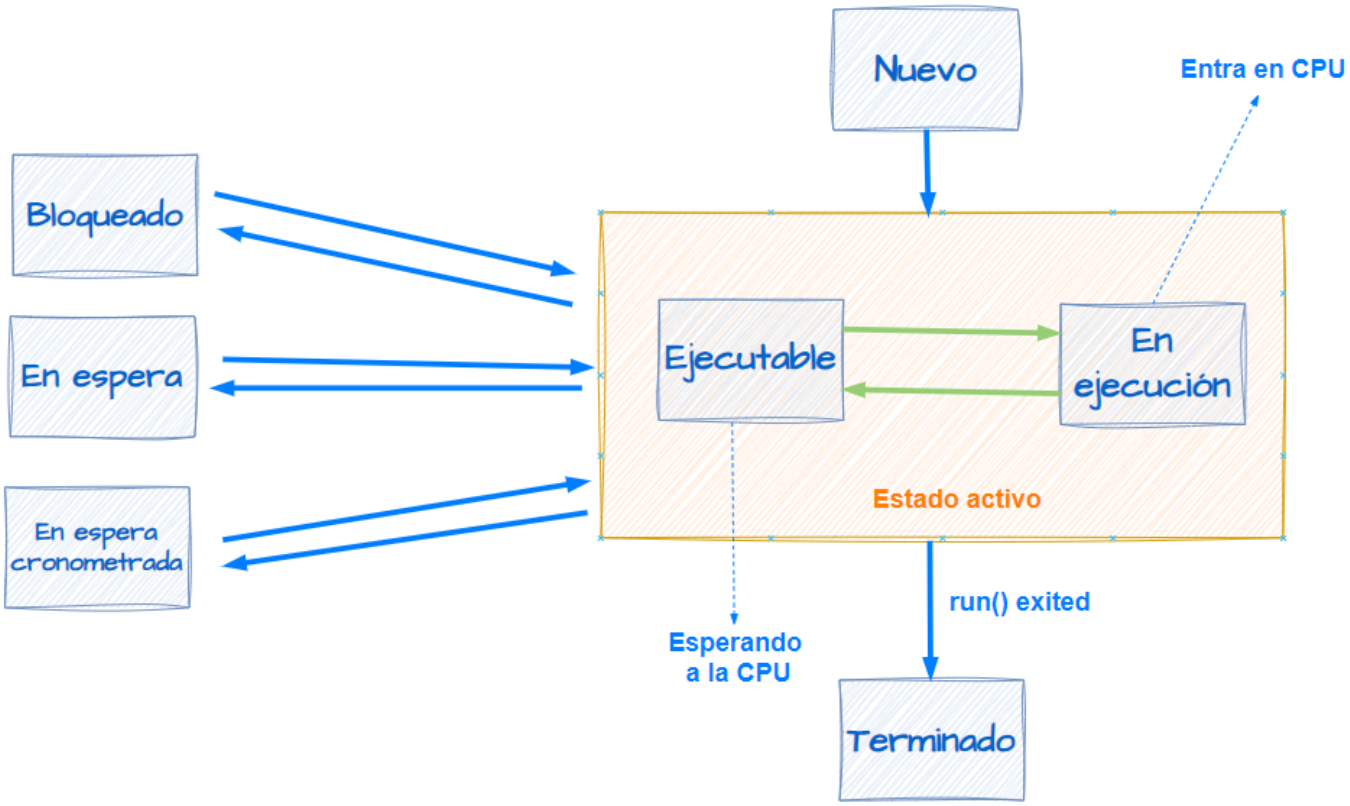
Estados de un hilo

Durante el ciclo de vida de los hilos, estos pasan por diversos estados. En Java, están recogidos dentro de la enumeración State contenida dentro de la clase java.lang.Thread.

El estado de un hilo se obtiene mediante el método geState() de la clase Thread, que devolverá algunos de los valores posibles recogidos en la enumeración indicada anteriormente.

Los estados de un thread son los que se muestran en la tabla:

Estado	Valor en Thread.State	Descripción
Nuevo	NEW	El hilo está creado, pero aún no se ha arrancado.
Ejecutable	RUNNABLE	El hilo está arrancado y podría estar en ejecución o pendiente de ejecución.
Bloqueado	BLOCKED	Bloqueado por un monitor.
Esperando	WAITING	El hilo está esperando a que otro hilo realice una acción determinada.
Esperando un tiempo	TIME_WAITING	El hilo está esperando a que otro hilo realice una acción determinada en un periodo de tiempo concreto.
Finalizado	TERMINATED	El hilo ha terminado su ejecución.



En el siguiente código de ejemplo, se almacenan en un ArrayList los estados por los que pasa un hilo que contien en su interior una llamada al método sleep. Se utiliza la clase Raton que implementa la interfaz Runnable de los ejemplos anteriores.

Código Java

```
import java.util.ArrayList;

public class Raton implements Runnable {

    private String nombre;
    private int tiempoAlimentacion;

    public Raton(String nombre, int tiempoAlimentacion) {
        super();
        this.nombre = nombre;
        this.tiempoAlimentacion = tiempoAlimentacion;
    }

    public void comer() {
        try {
            System.out.printf("El ratón %s ha comenzado a alimentarse%n", nombre);
            Thread.sleep(tiempoAlimentacion * 1000);
            System.out.printf("El ratón %s ha terminado de alimentarse%n",
```

```
nombre);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    @Override
    public void run() {
        this.comer();
    }

    public static void main(String[] args) {
        Raton pop = new Raton("Pop", 6);
        ArrayList estadosHilo = new ArrayList();
        Thread h = new Thread(pop);
        estadosHilo.add(h.getState());
        h.start();
        while (h.getState() != Thread.State.TERMINATED) {
            if (!estadosHilo.contains(h.getState())) {
                estadosHilo.add(h.getState());
            }
        } // fin de while

        if (!estadosHilo.contains(h.getState())) {
            estadosHilo.add(h.getState());
        }

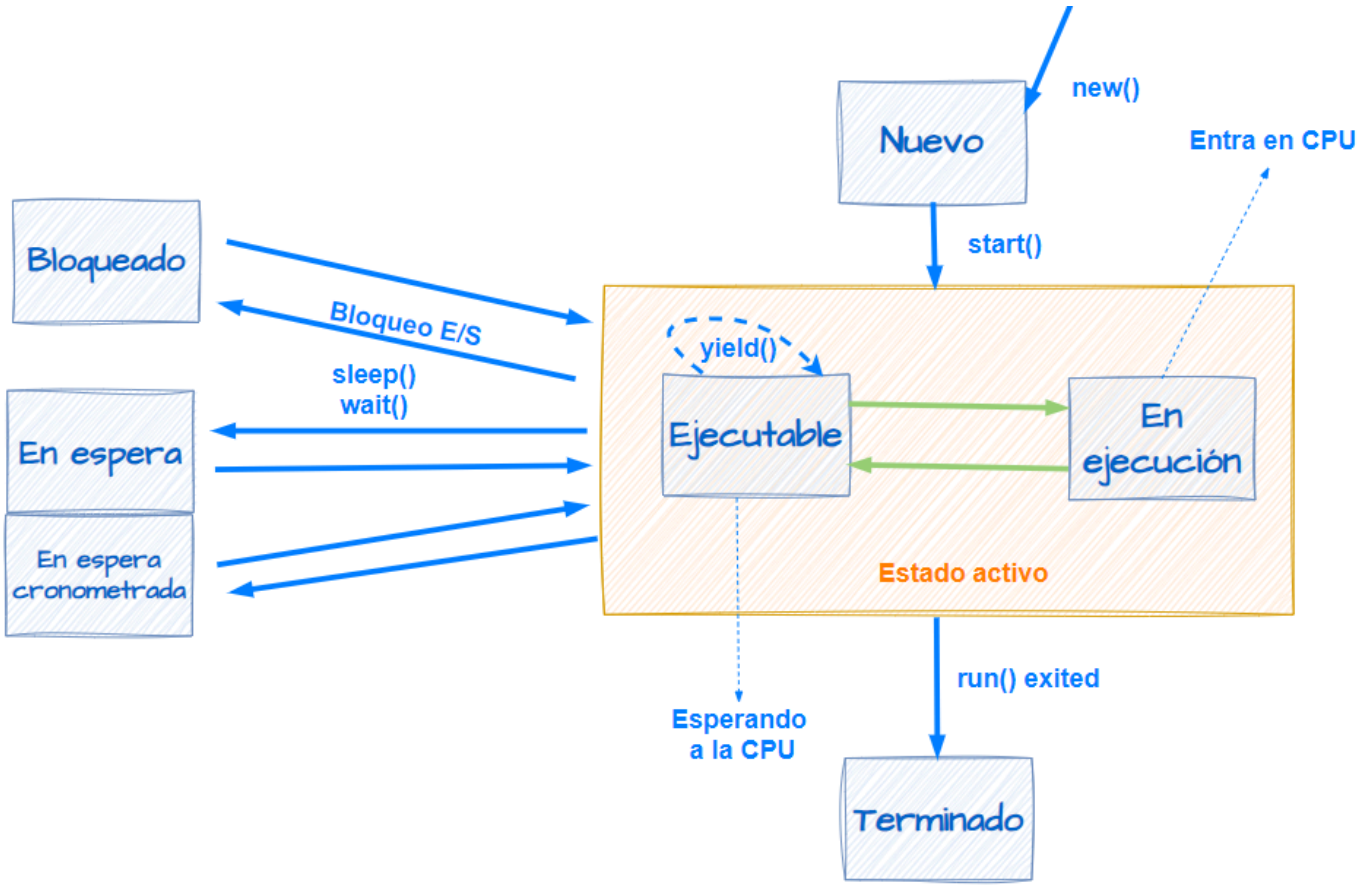
        for (Thread.State estado : estadosHilo) {
            System.out.println(estado);
        }
    }
}
```

Al finalizar la ejecución se muestran los estados recogidos.

El método getState() de la clase thread, permite obtener en cualquier momento el estado en el que se encuentra un hilo. Devuelve por tanto: NEW, RUNNABLE, NO RUNNABLE o TERMINATED.

Todos los hilos pasan por los estados NEW, RUNNABLE y TERMINATED. El resto de los estados están condicionados a circunstancias propias de la ejecución.

En la siguiente imagen, puedes ver algunos de los métodos que permiten obtener cada uno de esos estados. Los veremos con más detalle en los siguientes apartados.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Iniciar un hilo

Cuando se crea un nuevo hilo o thread mediante el método new(), no implica que el hilo ya se pueda ejecutar.

Para que el hilo se pueda ejecutar, debe estar en el estado "Ejecutable", y para conseguir ese estado es necesario iniciar o arrancar el hilo mediante el método start() de la clase thread().

En los ejemplos anteriores, recuerda que teníamos el código hilo1.start(); que precisamente se encargaba de iniciar el hilo representado por el objeto thread hilo1.

En realidad el método start() realiza las siguientes tareas:

- Crea los recursos del sistema necesarios para ejecutar el hilo.
- Se encarga de llamar a su método run() y lo ejecuta como un subproceso nuevo e independiente.

Es por esto último que cuando se invoca a start() se suele decir que el hilo está "corriendo" ("running"), pero recuerda que esto no significa que el hilo esté ejecutándose en todo momento, ya que un hilo "Ejecutable" puede estar "Preparado" o "Ejecutándose" según tenga o no asignado tiempo de procesamiento.

Algunas consideraciones importantes que debes tener en cuenta son las siguientes:

- Puedes invocar directamente al método run(), por ejemplo poner hilo1.run(); y se ejecutará el código asociado a run() dentro del hilo actual (como cualquier otro método), pero no comenzará un nuevo hilo como subproceso independiente.
- Una vez que se ha llamado al método start() de un hilo, no puedes volver a realizar otra llamada al mismo método. Si lo haces, obtendrás una excepción IllegalThreadStateException.
- El orden en el que inicies los hilos mediante start() no influye en el orden de ejecución de los mismos, lo que pone de manifiesto que el orden de ejecución de los hilos es no-determinístico (no se conoce la secuencia en la que serán ejecutadas la instrucciones del programa).

En el siguiente recurso didáctico puedes ver un programa que define dos hilos, construidos cada uno de ellos por los procedimientos vistos anteriormente. Cada hilo imprime una palabra 5 veces. Observa que si ejecutas varias veces el programa, el orden de ejecución de los hilos no es siempre el mismo y que no influye en absoluto el orden en el que se inician con start() (el orden de ejecución de los hilos es no-determinístico).

Nota. puede que tengas que aumentar el número de iteraciones (número de palabras que imprime cada hilo) para apreciar las observaciones indicadas anteriormente.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

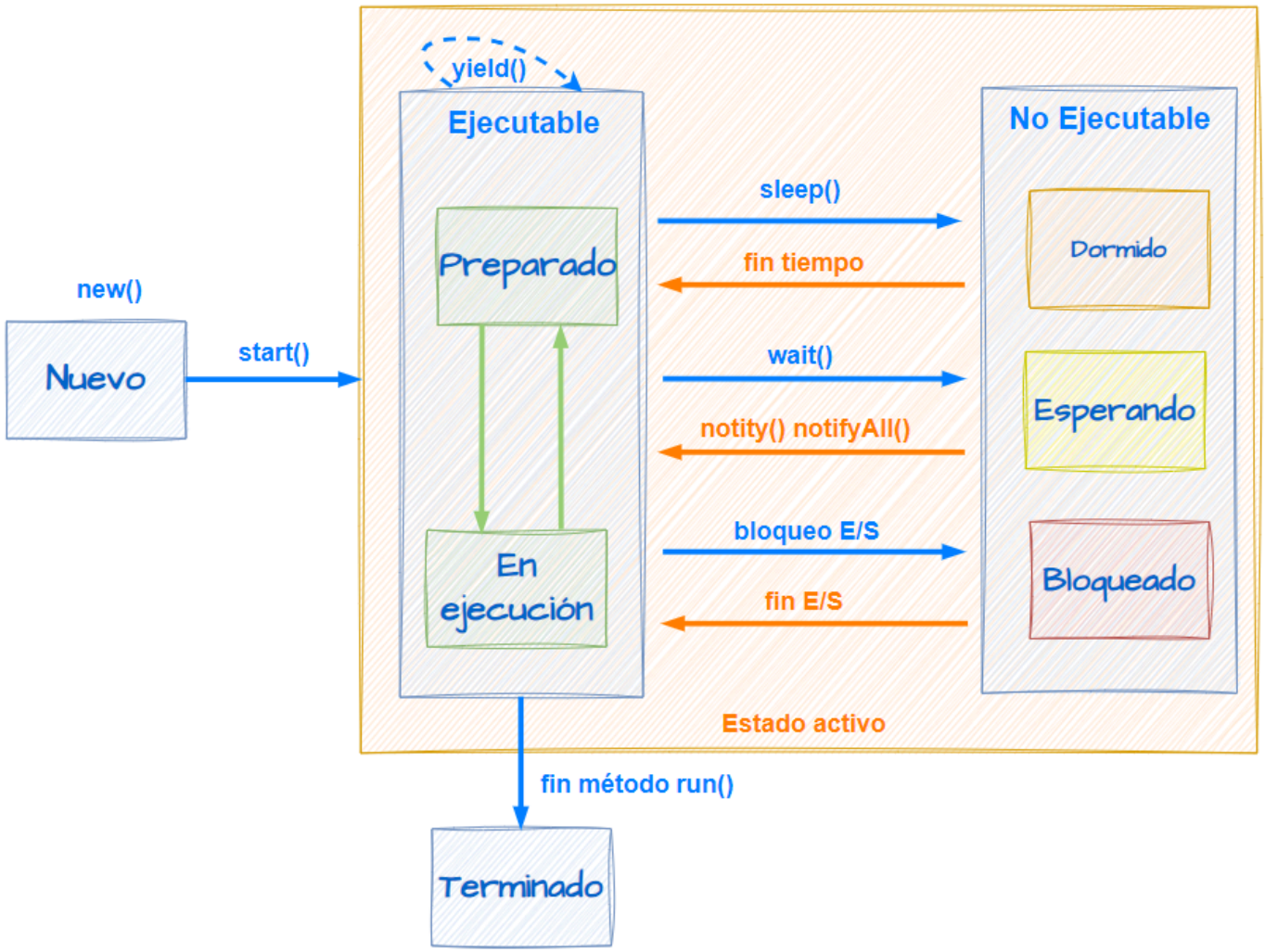
Detener temporalmente un hilo.

¿Qué significa que un hilo se ha detenido temporalmente? Significa que el hilo ha pasado al estado "No Ejecutable".

Y ¿cómo puede pasar un hilo al estado "No Ejecutable"? Un hilo pasará al estado "No Ejecutable" o "Detenido" por alguna de estas circunstancias:

- El hilo se ha dormido. Se ha invocado al método sleep() de la clase thread, indicando el tiempo que el hilo permanecerá detenido. Transcurrido ese tiempo, el hilo se vuelve "Ejecutable", en concreto pasa a "Preparado".
- El hilo está esperando. El hilo ha detenido su ejecución mediante la llamada al método wait(), y no se reanudará, pasará a "Ejecutable" (en concreto "Preparado") hasta que se produzca una llamada al método notify() o notifyAll() por otro hilo. Estudiaremos detalladamente estos métodos de la clase Object cuando veamos la sincronización y comunicación de hilos.
- El hilo se ha bloqueado. El hilo está pendiente de que finalice una operación de E/S en algún dispositivo, o a la espera de algún otro tipo de recurso; ha sido bloqueado por el sistema operativo. Cuando finaliza el bloqueo, vuelve al estado "Ejecutable", en concreto "Preparado".

En la siguiente imagen puedes ver un esquema con los diferentes métodos que hacen que un hilo pase al estado "No Ejecutable", así cómo los que permiten salir de ese estado y volver al estado "Ejecutable".



El método suspend() (actualmente en desuso o deprecated) también permite detener temporalmente un hilo, y en ese caso se reanudaría mediante el método resume() (también en desuso). No debes utilizar estos métodos, de la clase thread ya que no son seguros y provocan muchos problemas. Te lo indicamos simplemente porque puede que encuentres programas que aún utilizan estos métodos.

Importante

Consulta este enlace para conocer algunos métodos de la clase thread declarados obsoletos o no recomendados, como suspend() y resume() y el porqué.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Finalizar un hilo

La forma natural de que muera o finalice un hilo es cuando termina de ejecutarse su método run(), pasando al estado 'Muerto'.

Una vez que el hilo ha muerto, no lo puedes iniciar otra vez con start(). Si en tu programa deseas realizar otra vez el trabajo desempeñado por el hilo, tendrás que:

- Crear un nuevo hilo con new().
- Iniciar el hilo con start().

Y ¿hay alguna forma de comprobar si un hilo no ha muerto?

No exactamente, pero puedes utilizar el método isAlive() de la clase thread para comprobar si un hilo está vivo o no. Un hilo se considera que está vivo (alive) desde la llamada a su método start() hasta su muerte. isAlive() devuelve verdadero (true) o falso (false), según que el hilo esté vivo o no.

Cuando el método isAlive() devuelve:

- False: sabemos que estamos ante un nuevo hilo recién "creado" o ante un hilo "muerto".
- True: sabemos que el hilo se encuentra en estado "ejecutable" o "no ejecutable".

El método stop() de la clase thread (actualmente en desuso) también finaliza un hilo, pero es poco seguro. No debes utilizarlo. Te lo indicamos aquí simplemente porque puede que encuentres programas utilizando este método.

En el siguiente ejemplo, te proporcionamos un programa cuyo hilo principal lanza un hilo secundario que realiza una cuenta atrás desde 10 hasta 1. Desde el hilo principal se verificará la muerte del hilo secundario mediante la función isAlive(). Además mediante el método getState() de la clase thread vamos obteniendo el estado del hilo secundario. Se usa también el método thread.join() que espera hasta que el hilo muere.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Ejemplo. Dormir un hilo con sleep

¿Por qué puede interesar dormir un hilo? Pueden ser diferentes las razones que nos lleven a dormir un hilo durante unos instantes. En este apartado veremos un ejemplo en el que si no durmiéramos unos instantes al hilo que realiza un cálculo, no le daría tiempo al hilo que dibuja el resultado a presentarlo en pantalla.

¿Cómo funciona el método sleep()?

El método sleep() de la clase thread recibe como argumento el tiempo que deseamos dormir el hilo que lo invoca. Cuando transcurre el tiempo especificado, el hilo vuelve a estar "Ejecutable" ("Preparado") para continuar ejecutándose.

Hay dos formas de llamar a este método.

- La primera le pasa como argumento un entero (positivo) que representa milisegundos:
  - sleep(long milisegundos)
- La segunda le agrega un segundo argumento entero (esta vez, entre 1 y 999999), que representa un tiempo extra en nanosegundos que se sumará al primer argumento:
  - sleep(long milisegundos, int nanosegundos)

Cualquier llamada a sleep() puede provocar una excepción, que el compilador de Java nos obliga a controlar ineludiblemente mediante un bloque try-catch.

El siguiente recurso didáctico ilustra la necesidad de dormir un hilo en una aplicación que muestra como avanza un marcador gráfico desde 0 hasta 20. Podrás comprobar que si no utilizamos un hilo auxiliar y lo dormimos, no podremos apreciar como se va incrementando el marcador.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Gestión y planificación de hilos

La ejecución de hilos se puede realizar mediante:

- Paralelismo. En un sistema con múltiples CPU, cada CPU puede ejecutar un hilo diferente.
- Pseudoparalelismo. Si no es posible el paralelismo, una CPU es responsable de ejecutar múltiples hilos.

La ejecución de múltiples hilos en una sola CPU requiere la planificación de una secuencia de ejecución (sheduling).

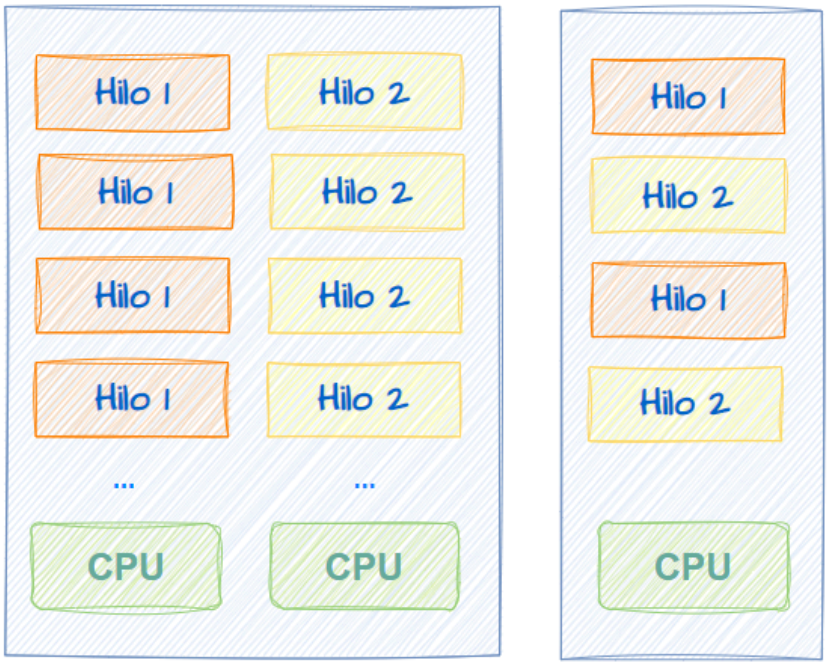
El planificador de hilos de Java (Sheduler) utiliza un algoritmo de secuenciación de hilos denominado fixed priority scheduling que está basado en un sistema de prioridades relativas, de manera que el algoritmo secuencia la ejecución de hilos en base a la prioridad de cada uno de ellos.

El funcionamiento del algoritmo es el siguiente:

- El hilo elegido para ejecutarse, siempre es el hilo "Ejecutable" de prioridad más alta.
- Si hay más de un hilo con la misma prioridad, el orden de ejecución se maneja mediante un algoritmo por turnos (round-rubin) basado en una cola circular FIFO (Primero en entrar, primero en salir).
- Cuando el hilo que está "ejecutándose" pasa al estado de "No Ejecutable" o "Muerto", se selecciona otro hilo para su ejecución.
- La ejecución de un hilo se interrumpe, si otro hilo con prioridad más alta se vuelve "Ejecutable". El hecho de que un hilo con una prioridad más alta interrumpa a otro se denomina "planificación apropiativa" ('preemptive sheudling').

Pero la responsabilidad de ejecución de los hilos es del Sistema Operativo sobre el que corre la JVM, y Sistemas Operativos distintos manejan los hilos de manera diferente:

- En un Sistema Operativo que implementa time-slicing (subdivisión de tiempo), el hilo que entra en ejecución, se mantiene en ella sólo un micro-intervalo de tiempo fijo o cuanto (quantum) de procesamiento, de manera que el hilo que está "ejecutándose" no solo es interrumpido si otro hilo con prioridad más alta se vuelve "Ejecutable", sino también cuando su "cuanto" de ejecución se acaba. Es el patrón seguido por Linux, y por todos los Windows a partir de Windows 95 y NT.
- En un Sistema Operativo que no implementa time-slicing el hilo que entra en ejecución, es ejecutado hasta su muerte; salvo que regrese a "No ejecutable", u otro hilo de prioridad más alta alcance el estado de "Ejecutable" (en cuyo caso, el primero regresa a "preparado" para que se ejecute el segundo). Es el patrón seguido en el Sistema Operativo Solaris.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Prioridades en hilos

Java asigna a cada hilo una prioridad que determina cómo se debe tratar ese hilo en comparación con los demás. Las prioridades de los hilos son valores enteros que especifican la prioridad relativa de un hilo sobre otro. Como valor absoluto, una prioridad no tiene sentido alguno; un hilo de prioridad más alta no se ejecuta más rápidamente que otro de prioridad más baja si es el único hilo que se está ejecutando.

En Java, cada hilo tiene una prioridad representada por un valor de tipo entero entre 1 y 10. Cuanto mayor es el valor, mayor es la prioridad del hilo.

Por defecto, el hilo principal de cualquier programa, o sea, el que ejecuta su método main() siempre es creado con prioridad 5.

El resto de hilos secundarios (creados desde el hilo principal, o desde cualquier otro hilo en funcionamiento), heredan la prioridad que tenga en ese momento su hilo padre.

La prioridad de un hilo se utiliza para decidir cuándo se debe pasar de la ejecución de un hilo a la del siguiente. A esto se denomina cambio de contexto. Las reglas que determinan cuándo debe tener lugar un cambio de contexto son muy sencillas:

- Un hilo puede ceder voluntariamente el control. Esto se hace por abandono explícito, al quedarse dormido, o bloqueado por una E/S pendiente. Cuando esto ocurre, se examinan todos los demás hilos, y se le asigna el CPU al que esté preparado para ejecutarse y tenga la más alta prioridad.
- Un hilo puede ser desalojado por otro con prioridad más alta. En este caso, un hilo de prioridad más baja que no libera el procesador es simplemente desalojado, sin tener en cuenta lo que esté haciendo, por otro de prioridad más alta. Básicamente, cuando un hilo de prioridad más alta desee ejecutarse, lo hará. A esto se denomina multitarea por desalojo.
- Una situación un poco más complicada es la que se produce cuando dos hilos de la misma prioridad compiten por el CPU. En sistemas operativos como Windows, los hilos con la misma prioridad se reparten automáticamente el tiempo mediante un algoritmo circular (round-robin). Para otros sistemas operativos, los hilos deben ceder voluntariamente el control a otros de la misma prioridad. Si no lo hacen así, estos últimos no se ejecutarán.

En la clase thread se definen 3 constantes para manejar estas prioridades:

- MAX_PRIORITY (= 10). Es el valor que simboliza la máxima prioridad.
- MIN_PRIORITY (=1). Es el valor que simboliza la mínima prioridad.
- NORM_PRIORITY (= 5). Es el valor que simboliza la prioridad normal, la que tiene por defecto el hilo donde corre el método main().

Además en cualquier momento se puede obtener y modificar la prioridad de un hilo, mediante los siguientes métodos de la clase thread:

- getPriority(). Obtiene la prioridad de un hilo. Este método devuelve la prioridad del hilo.
- setPriority(). Modifica la prioridad de un hilo. Este método toma como argumento un entero entre 1 y 10, que indica la nueva prioridad del hilo.

Java tiene 10 niveles de prioridad que no tienen por qué coincidir con los del sistema operativo sobre el que está corriendo. Por ello, lo mejor es que utilices en tu código sólo las constantes MAX_PRIORITY, NORM_PRIORITY y MIN_PRIORITY.

Podemos conseguir aumentar el rendimiento de una aplicación multihilo gestionando adecuadamente las prioridades de los diferentes hilos, por ejemplo utilizando una prioridad alta para tareas de tiempo crítico y una prioridad baja para otras tareas menos importantes.

En el siguiente enlace tienes un ejemplo en el que se declara un hilo cuya tarea es llenar un vector con 20000 caracteres. Se inician 15 hilos con prioridades diferentes, 5 con prioridad máxima, 5 con prioridad normal y 5 con prioridad mínima. Al ejecutar el programa comprobarás que los hilos con prioridad más alta tienden a finalizar antes. Observa que se usa también el método yield() del que hablaremos en el siguiente apartado.



Saber más

Los hilos demonio (daemon) son hilos que se ejecutan en segundo plano. No dejes de visitar este enlace para conocer más sobre hilos demonio.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Prioridades de Hilos y Planificación de Hilos

Cada hilo en Java tiene una prioridad que ayuda a determinar el orden en el que los hilos son programados para ejecutarse. Cada nuevo hilo hereda la prioridad del hilo que lo creó. De manera informal, los hilos de mayor prioridad son más importantes para un programa y deberían recibir tiempo de procesador antes que los hilos de menor prioridad. No obstante, las prioridades de los hilos no pueden garantizar el orden en que los hilos se ejecutarán.

Se recomienda que no crees ni uses objetos Thread explícitamente para implementar concurrencia, sino que uses la interfaz Executor (descrita en la Sección 26.3). La clase Thread contiene algunos métodos estáticos útiles, que utilizarás más adelante en el capítulo.

La mayoría de los sistemas operativos admiten el reparto de tiempo (timeslicing), lo que permite a hilos de igual prioridad compartir un procesador. Sin reparto de tiempo, cada hilo en un conjunto de hilos de igual prioridad se ejecuta hasta completarse (a menos que salga del estado ejecutable y entre en el estado de espera o de espera temporizada, o sea interrumpido por un hilo de mayor prioridad) antes de que otros hilos de igual prioridad tengan la oportunidad de ejecutarse. Con reparto de tiempo, incluso si un hilo no ha terminado de ejecutarse cuando expira su tiempo, el procesador se le quita al hilo y se le da al siguiente hilo de igual prioridad, si hay uno disponible.

El planificador de hilos de un sistema operativo determina qué hilo se ejecuta a continuación. Una implementación simple del planificador de hilos mantiene el hilo de mayor prioridad ejecutándose en todo momento y, si hay más de un hilo de mayor prioridad, asegura que todos esos hilos se ejecuten en turnos, en modo circular (round-robin), cada uno durante un tiempo determinado. Este proceso continúa hasta que todos los hilos se completan.

Cuando un hilo de mayor prioridad entra en el estado de listo, el sistema operativo generalmente reemplaza el hilo que se está ejecutando en ese momento (una operación conocida como planificación preventiva). Dependiendo del sistema operativo, los hilos de mayor prioridad podrían posponer—posiblemente indefinidamente—la ejecución de hilos de menor prioridad. Este aplazamiento indefinido a veces se conoce coloquialmente como inanición. Los sistemas operativos emplean una técnica llamada "envejecimiento" para prevenir la inanición: a medida que un hilo espera en el estado de listo, el sistema operativo aumenta gradualmente la prioridad del hilo, asegurando así que el hilo eventualmente se ejecutará.

Java proporciona utilidades de concurrencia de alto nivel para ocultar gran parte de esta complejidad y hacer que la programación multihilo sea menos propensa a errores. Las prioridades de los hilos se usan en segundo plano para interactuar con el sistema operativo,

pero la mayoría de los programadores que usan multihilos en Java no estarán preocupados por establecer y ajustar las prioridades de los hilos.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Hilos egoístas y programación expulsora

¿De verdad existen los hilos egoístas? En un Sistema Operativo que no implemente time-slicing puede ocurrir que un hilo que entra en "ejecución" no salga de ella hasta su muerte, de manera que no dará ninguna posibilidad a que otros hilos "preparados" entren en "ejecución" hasta que él muera. Este hilo se habrá convertido en un hilo egoísta.

Por ejemplo, supongamos la siguiente situación en un Sistema Operativo sin time-slicing:

La tarea asociada al método run() de un hilo consiste en imprimir 100 veces la palabra que se le pasa al constructor más el número de orden.
Se inician dos hilos en main(), uno imprimiendo "Azul" y otro "Rojo".
El hilo que sea seleccionado en primer lugar por el planificador se ejecutará íntegramente, por ejemplo el que imprime "Rojo" 100 veces. Después se ejecutará el otro hilo, tal y como muestra la imagen parcial de la derecha.
Este hilo tiene un comportamiento egoísta.

En un Sistema Operativo que si implemente time-slicing la ejecución de esos hilos se entremezcla, tal y como muestra la imagen parcial de la izquierda, lo cual indica que no hay comportamiento egoísta de ningún hilo, esto es, el Sistema operativo combate los hilos egoístas.

El proyecto completo lo puedes descargar desde el siguiente enlace.



Según lo anterior, un mismo programa Java se puede ejecutar de diferente manera según el Sistema Operativo donde corra. Entonces, ¿que pasa con la portabilidad de Java? Java da solución a este problema mediante lo que se conoce como programación expulsora a través del método yield() de la clase java.lang.thread:

- yield() hace que un hilo que está "ejecutándose" pase a "preparado" para permitir que otros hilos de la misma prioridad puedan ejecutarse.

Sobre el método yield() y el egoísmo de los threads debes tener en cuenta que:

- El funcionamiento de yield() no está garantizado, puede que después de que un hilo invoque a yield() y pase a "preparado", éste vuelva a ser elegido para ejecutarse.
- No debes asumir que la ejecución de una aplicación se realizará en un Sistema Operativo que implementa time-slicing.
- En la aplicación debes incluir adecuadamente llamadas al método yield(), incluso a sleep() o wait(), si el hilo no se bloquea por una Entrada/Salida.

El siguiente código muestra la forma de invocar a yield() dentro del método run() de un hilo. Ten en cuenta que si la invocación se hace desde un hilo Runnable tendrás que poner thread.yield();

> Este es el método `run()` que imprime el valor de `color + i` 100 veces, usando `yield()` para ceder la ejecución temporalmente.

```
public void run() {
    // se imprime 100 veces el valor de: color + i
    for (int i = 1; i <= 100; i++) {
        System.out.println(color + i);
        yield(); // llamada a yield()
    }
}
```

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Sincronización y comunicación de hilos

Cuando dos o más hilos tienen que acceder a un recurso compartido, es necesario asegurar de alguna manera que sólo uno de ellos accede a ese recurso en cada instante. El proceso mediante el que se consigue esto se denomina **sincronización.**

Los ejemplos realizados hasta ahora utilizan hilos independientes; una vez iniciados los hilos, éstos no se relacionan con los demás y no acceden a los mismos datos u objetos, por lo que no hay conflictos entre ellos. Sin embargo, hay ocasiones en las que distintos hilos de un programa necesitan establecer alguna relación entre sí y compartir recursos o información. Se pueden presentar las siguientes situaciones:

- Dos o más hilos compiten por obtener un mismo recurso, por ejemplo dos hilos que quieren escribir en un mismo fichero o acceder a la misma variable para modificarla.
- Dos o más hilos colaboran para obtener un fin común y para ello, necesitan comunicarse a través de algún recurso. Por ejemplo un hilo produce información que utilizará otro hilo.

En cualquiera de estas situaciones, es necesario que los hilos se ejecuten de manera controlada y coordinada, para evitar posibles interferencias que pueden desembocar en programas que se bloquean con facilidad y que intercambian datos de manera equivocada.

¿Cómo conseguimos que los hilos se ejecuten de manera coordinada? Utilizando sincronización y comunicación de hilos:

- Sincronización. Es la capacidad de informar de la situación de un hilo a otro. El objetivo es establecer la secuencialidad correcta del programa.
- Comunicación. Es la capacidad de transmitir información desde un hilo a otro. El objetivo es el intercambio de información entre hilos para operar de forma coordinada.

En Java la sincronización y comunicación de hilos se consigue mediante:

- **Monitores**. Se crean al marcar bloques de código con la palabra **synchronized**.
- **Semáforos**. Podemos implementar nuestros propios semáforos, o bien utilizar la clase **Semaphore** incluida en el paquete java.util.concurrent.
- **Notificaciones**. Permiten comunicar hilos mediante los métodos **wait**(), **notify**() y **notifyAll**() de la clase java.lang.Object.

Por otra parte, Java proporciona en el paquete java.util.concurrent varias clases de sincronización que permiten la sincronización y comunicación entre diferentes hilos de una aplicación multithreadring, como son: CountDownLatch, CyclicBarrier y Exchanger.

En los siguientes apartados veremos ejemplos de todo esto.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Información compartida entre hilos.

Las secciones críticas son aquellas secciones de código que no pueden ejecutarse concurrentemente, pues en ellas se encuentran los recursos o información que comparten diferentes hilos, y que por tanto pueden ser problemáticas.

Un ejemplo sencillo que ilustra lo que puede ocurrir cuando varios hilos actualizan una misma variable es el clásico "ejemplo de los jardines". En él, se pone de manifiesto el problema conocido como la "condición de carrera", que se produce cuando varios hilos acceden a la vez a un mismo recurso, por ejemplo a una variable, cambiando su valor y obteniendo de esta forma un valor no esperado de la misma. En el siguiente enlace te facilitamos este ejemplo detallado.



En el ejemplo del "problema de los jardines", el recurso que comparten diferentes hilos es la variable contador cuenta. Las secciones de código donde se opera sobre esa variable son dos secciones críticas, los métodos incrementaCuenta() y decrementaCuenta().

La forma de proteger las secciones críticas es mediante sincronización. La sincronización se consigue mediante:

- Exclusión mutua. Asegurar que un hilo tiene acceso a la sección crítica de forma exclusiva y por un tiempo finito.
  Por condición. Asegurar que un hilo no progrese hasta que se cumpla una determinada condición.

En Java, la sincronización para el acceso a recursos compartidos se basa en el concepto de monitor.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

El Problema

Imagine una clase que representa un stack. La clase podría parecerse a la siguiente:



Esta clase `MiPila` implementa una pila básica de caracteres con métodos para apilar (`push`) y desapilar (`pop`) elementos. El tamaño máximo de la pila está limitado a 6 elementos.

```
public class MiPila {
    int idx = 0;
    char[] data = new char[6];

    public void push(char c) {
        data[idx] = c;
        idx++;
    }

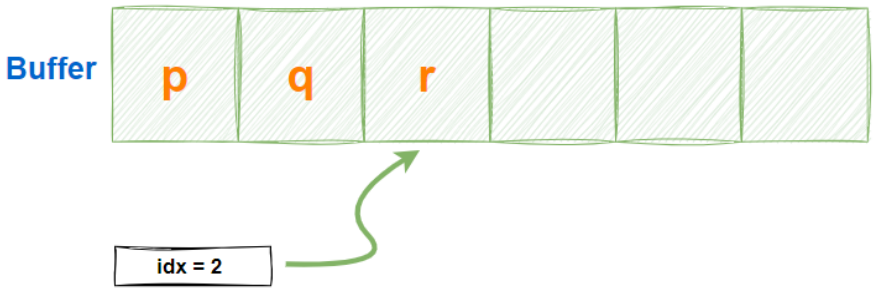
    public char pop() {
        idx--;
        return data[idx];
    }
}
```

La clase no realiza ninguna actividad para controlar los estados de overflow o underflow del stack. No obstante, su capacidad es limitada, aunque este aspecto no es relevante para la discusión actual.

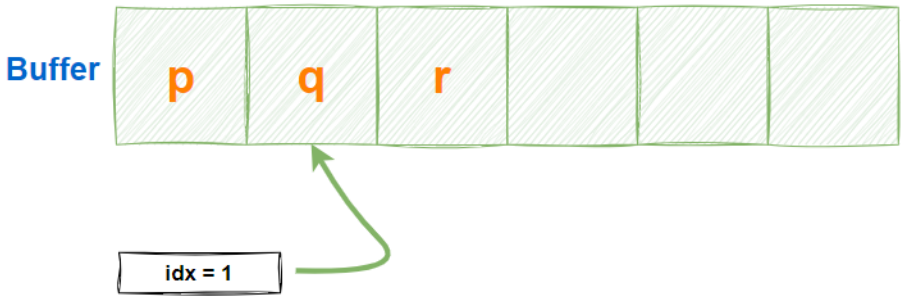
El comportamiento de este modelo requiere que el valor de idx corresponda al índice del arreglo de la siguiente celda vacía en el stack. La combinación de operaciones de pre-decremento y post-incremento ayuda a mantener esta información.

Imagine ahora que dos hilos tienen una referencia a una única instancia de esta clase. Un hilo realiza inserciones (push) en el stack, mientras que el otro realiza extracciones (pop) de manera independiente. En principio, los datos se agregan y eliminan satisfactoriamente. Sin embargo, existe un problema potencial.

Supongamos que el hilo A está agregando caracteres y el hilo B está quitándolos. El hilo A agrega un carácter, pero aún no realiza el incremento del índice. Por alguna razón, el hilo entra en estado de espera (preemptivo). En ese momento, el modelo de datos representado en el objeto se vuelve inconsistente.

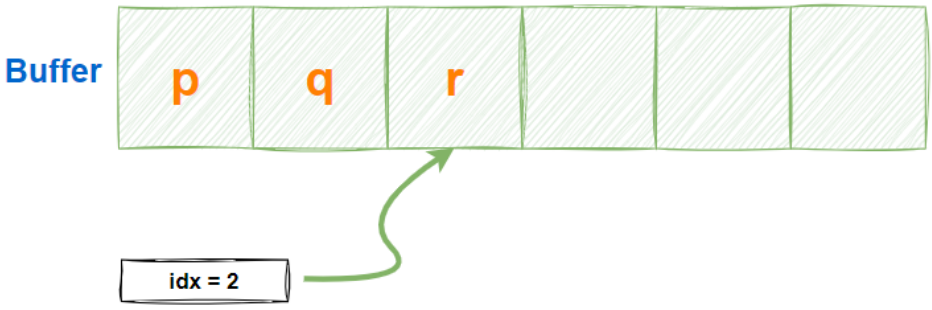


Para mantener la consistencia, idx debería ser igual a 3 o el carácter no debería haberse agregado aún. Si el hilo A retoma la ejecución, no habrá problemas. Pero si el hilo B intenta eliminar un carácter mientras el hilo A está esperando otra oportunidad para ejecutar, se genera una situación de inconsistencia en los datos, ya que el método pop reducirá el valor de idx.



Esto efectivamente provoca que el carácter r sea ignorado y el método devuelva el carácter q. Así, es como si la letra r nunca se hubiese agregado al stack, generando un problema.

Supongamos ahora que el hilo A retoma su ejecución en el método push y procede a incrementar el valor del índice. Esto producirá la siguiente situación:



En esta configuración, q se considera válido y la celda que contiene r se interpreta como la siguiente celda vacía. En otras palabras, es como si q se hubiera agregado dos veces y el elemento r nunca se hubiese insertado.

Este es un ejemplo simple de un problema general que puede ocurrir cuando varios hilos acceden a los mismos datos. Por lo tanto, se necesita un mecanismo para asegurar que los datos compartidos estén en un estado consistente antes de que otro hilo comience a utilizarlos para una tarea en particular.

Una opción podría consistir en impedir que el hilo abandone su ejecución hasta que complete la sección crítica del código. Esta opción es común en la programación de bajo nivel, pero suele ser inapropiada para sistemas multiusuario.

Otra opción, que es la que utiliza Java, consiste en proporcionar un mecanismo que permita manejar los datos de forma cuidadosa. Este mecanismo permite que un hilo acceda a los datos de manera indivisible, sin importar si se ve interrumpido en medio de ese acceso.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Sincronización básica: el método join

El método join permite indicar a un hilo que debe suspender su ejecución hasta que termine otro hilo de referencia. Este método debe ejecutarse dentro del bloque asíncrono del código, ya que lo contrario no tendrá ningún efecto. El método join es también parte de la clase Thread y es útil cuando se necesita que el hilo actual no avance hasta que un hilo específico haya finalizado. Puede ser útil, por ejemplo, cuando se deben realizar algunas tareas en orden secuencial, pero en diferentes hilos.

Propiedades y comportamiento de join:

- El hilo actual se bloquea y espera a que el hilo en el que se llama join complete su ejecución.
- Se puede especificar un tiempo máximo de espera (join(long millis)). Si el hilo aún no ha terminado cuando el tiempo expira, el hilo actual reanuda su ejecución.
- Es particularmente útil en aplicaciones donde los resultados de ciertos hilos son necesarios antes de que otros procesos puedan continuar.

En el siguiente programa, después de arrancar los dos hilos, se le indica al hilo Hilo 2 que debe quedar suspendido hasta que termine la ejecución de Hilo 1. Para ello es necesario que Hilo2 tenga una referencia a Hilo1 (hiloReferencia en el ejemplo) para invocar al método join.



Este ejemplo muestra cómo usar el método `join()` en Java. La clase `JoinBasico` suspende la ejecución de un hilo hasta que otro hilo de referencia finalice su ejecución.

```
public class JoinBasico extends Thread {

    private int id;
    private boolean suspender = false;
    private Thread hiloReferencia;

    public JoinBasico(int id) {
        this.id = id;
    }

    public void suspenderHilo(Thread hiloReferencia) {
        this.suspender = true;
        this.hiloReferencia = hiloReferencia;
    }
```

```
    @Override
    public void run() {
        try {
            for (int i = 0; i < 3; i++) {
                if (suspender) {
                    hiloReferencia.join();
                }
                System.out.printf("Hilo %d. Iteración %d\n", id, i);

                Thread.sleep(1000);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static void main(String[] args) {
        JoinBasico hilo1 = new JoinBasico(1);
        JoinBasico hilo2 = new JoinBasico(2);

        hilo1.start();
        hilo2.start();
        hilo2.suspenderHilo(hilo1);
    }
}
```

El resultado obtenido en este caso es el siguiente:

```
Hilo 1. Iteración 0
Hilo 1. Iteración 1
Hilo 1. Iteración 2
Hilo 2. Iteración 0
Hilo 2. Iteración 1
Hilo 2. Iteración 2
```

Vamos a ver otro ejemplo. Partamos el siguiente código:



Este ejemplo muestra cómo calcular el siguiente número primo en un hilo separado. La clase `HiloPrimo` inicia un hilo que busca el siguiente primo después de un número dado.

```
public class HiloPrimo {
    private static long primo;

    public static void main(String[] args) {
        Thread hilo;
        (hilo = new Thread(() -> calcularSiguientePrimo(10000000))).start();
        System.out.println(primo);
    }

    private static void calcularSiguientePrimo(long numero) {
        if (numero < 2) primo = 2;
        else for (primo = numero % 2 == 0 ? numero + 1 : numero; !esPrimo(primo);
primo += 2);
    }

    private static boolean esPrimo(long numero) {
        if (numero == 2 || numero == 3 || numero == 5 || numero == 7) return true;
        if (numero < 11 || numero % 2 == 0) return false;
        for (long i = 3, limite = (long) Math.sqrt(numero) + 1; i <= limite; i +=
2) {
            if (numero % i == 0) return false;
        }
        return true;
    }
}
```

El método `calcularSiguientePrimo()` determina el primer número primo que es mayor o igual al parámetro `numero`. Es un método que he mostrado anteriormente, y aquí es interesante que, si el parámetro `numero` es grande, puede tomar mucho tiempo ejecutar el método. En `main()` se crea un hilo que ejecuta el método y luego imprime el resultado como el valor de la variable `primo `. El programa imprimirá 0, lo cual, por supuesto, es incorrecto. La razón es que el hilo principal continúa después de que el hilo `hilo` se inicia e imprime el valor de `primo` antes de que el hilo haya actualizado la variable. El hilo principal no espera a que el hilo `hilo` termine.

El problema se puede solucionar cambiando el método main de la siguiente manera:



Este ejemplo muestra cómo calcular el siguiente número primo en un hilo separado usando el método `join()` para esperar a que el hilo termine su ejecución antes de mostrar el resultado.

```
public class HiloPrimo {
    private static long primo;

    public static void main(String[] args) {
        Thread hilo= new Thread(() -> nextPrime(10000000));
        hilo.start();
        try {
            hilo.join(); // Espera a que el hilo termine su ejecución
        } catch (InterruptedException e) {
            System.out.println("El hilo principal fue interrumpido.");
        }
        System.out.println(primo);
    }

    private static void calcularSiguientePrimo(long numero) {
        if (numero < 2) primo= 2;
        else for (primo= numero % 2 == 0 ? numero + 1 : numero; !esPrimo(primo);
primo+= 2);
    }

    private static boolean esPrimo(long numero) {
        if (numero == 2 || numero == 3 || numero == 5 || numero == 7) return true;
        if (numero < 11 || numero % 2 == 0) return false;
        for (long i = 3, limite = (long) Math.sqrt(numero) + 1; i <= limite ; i +=
2)
            if (numero % i == 0) return false;
        return true;
    }
}
```

Un hilo tiene un método llamado `join()`. Si otro hilo —en este caso, el hilo principal— está ejecutando el método `join()` sobre un objeto `Thread`, significa que el segundo hilo debe esperar hasta que el primer hilo termine. En este caso, esto significa que la última instrucción de impresión no se ejecutará hasta que el hilo `hilo` haya calculado el resultado y haya terminado.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Monitores. Métodos synchronized

La clave para la sincronización es el concepto de monitor, también llamado semáforo. Un monitor es un objeto que se utiliza como un **candado** mutuamente **exclusivo**, o **mutex**. Sólo uno
de los hilos puede poseer un monitor en un determinado instante. Cuando un hilo adquiere un candado, se dice que ha entrado en el monitor. Todos los demás hilos que intenten acceder al monitor bloqueado serán suspendidos hasta que el primero salga del monitor. Se dice que estos otros hilos están esperando al monitor. Un hilo que posea un monitor puede volver a entrar en el mismo monitor si así lo desea.

En Java, un monitor es una porción de código protegida por un mutex o lock. Para crear un monitor en Java, hay que marcar un bloque de código con la palabra synchronized, pudiendo ser ese bloque:

- Un método completo.
- Cualquier segmento de código.

La sincronización resulta sencilla en Java, porque todos los objetos tienen su propio monitor implícito asociado. Para entrar en el monitor de un objeto, basta con llamar a un método modificado con la palabra clave **synchronized**. Mientras un hilo esté dentro de un método sincronizado, todos los demás hilos que traten de llamar a ese método, o a otro método sincronizado sobre la misma instancia, tendrán que esperar. Para salir del monitor y abandonar el control del objeto, el propietario del monitor sólo tiene que salir del método sincronizado.

Añadir synchronized a un método significará que:

- Hemos creado un monitor asociado al objeto.
- Sólo un hilo puede ejecutar el método synchronized de ese objeto a la vez.
- Los hilos que necesitan acceder a ese método synchronized permanecerán bloqueados y en espera.
- Cuando el hilo finaliza la ejecución del método synchronized, los hilos en espera de poder ejecutarlo se desbloquearán. El planificador Java seleccionará a uno de ellos.

Comencemos con un ejemplo sencillo que no la usa, pero debería. El siguiente programa tiene tres clases. La primera, Callme, tiene un sólo método llamado call( ). El método call( ) tiene un parámetro del tipo String denominado msg. Este método intenta imprimir la cadena msg entre corchetes. La cuestión de interés es que, después de que el método call( ) imprime el corchete de apertura y la cadena msg, se llama a Thread.sleep(l000), lo que detiene el hilo en curso durante un segundo.

El constructor de la siguiente clase, Caller, toma una referencia a una instancia de la clase Callme y un String, los cuales se almacenan en las variables target y msg respectivamente.
El constructor también crea un nuevo hilo que llamará al método run de este objeto. El hilo es iniciado inmediatamente. El método run( ) de Caller llama al método call( ) de la instancia target de Callme, pasando la cadena msg. Finalmente, la clase Synch comienza creando una instancia de Callme, y tres instancias de Caller, cada una con una cadena diferente. La misma instancia de Callme se pasa a cada instancia de Caller.



Este ejemplo demuestra el uso de hilos en Java y la sincronización mediante la clase `Callme`, en la cual múltiples hilos intentan llamar al método `call()` simultáneamente.

```
class Callme {
    void call(String msg) {
        System.out.print("[" + msg);
        try {
            Thread.sleep(1000);  // Pausa de 1 segundo para simular una llamada
        } catch (InterruptedException e) {
            System.out.println("Interrumpido");
        }
        System.out.println("]");
    }
}

class Caller implements Runnable {
    String msg;
    Callme target;
    Thread t;

    public Caller(Callme targ, String s) {
        target = targ;
        msg = s;
        t = new Thread(this);
        t.start();
    }

    public void run() {
        target.call(msg);
    }
}

public class Synch {
```

```
    public static void main(String args[]) {
        Callme target = new Callme();
        Caller ob1 = new Caller(target, "Hola");
        Caller ob2 = new Caller(target, "Sincronizado");
        Caller ob3 = new Caller(target, "Mundo");

        // Espera a que terminen los hilos
        try {
            ob1.t.join();
            ob2.t.join();
            ob3.t.join();
        } catch (InterruptedException e) {
            System.out.println("Interrumpido");
        }
    }
}
```

La salida producida por este programa es la siguiente:
[Hola [Sincronizado [Mundo]
]
]

Explicación del Código

1. Clase Callme:
    1. La clase Callme tiene un método call(String msg), que simula una llamada imprimiendo un mensaje dentro de corchetes. Pausa la ejecución durante un segundo (Thread.sleep(1000)) para simular el tiempo de procesamiento.
2. Clase Caller:
    1. Esta clase implementa Runnable y representa un hilo que llama al método call en un objeto Callme.
    2. Caller toma el mensaje msg y el objeto Callme al que llamará. Al crear un objeto Caller, se inicia un nuevo hilo que ejecuta el método call de Callme.
3. Clase Principal Synch:
    1. En el método main, se crea una única instancia de Callme.
    2. Luego, se crean tres objetos Caller, cada uno con un mensaje diferente ("Hola", "Sincronizado", "Mundo"), que se ejecutan en hilos independientes.
    3. Finalmente, join() se usa para esperar que todos los hilos finalicen antes de que el programa continúe.

Dado que los tres hilos llaman al mismo método call en el mismo objeto Callme, puede ocurrir que las impresiones se mezclen en la consola. Para asegurarte de que cada mensaje se imprime de forma completa y sin interrupciones, puedes hacer que el método call sea synchronized:



Este ejemplo demuestra el uso del modificador `synchronized` en el método `call()` de la clase `Callme`, permitiendo que solo un hilo acceda a este método a la vez.

```
class Callme {
    synchronized void call(String msg) {
        System.out.print("[" + msg);
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            System.out.println("Interrumpido");
        }
        System.out.println("]");
    }
}
```

Si se usa synchronized, la salida será ordenada, y cada mensaje se imprimirá sin interferencias de otros hilos, algo como:

[Hola]

[Sincronizado]

[Mundo]

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Sincronización

La programación multihilo introduce un comportamiento asíncrono en los programas, aunque en ocasiones puede ser necesario el sincronismo. Esto sucede, por ejemplo, si se quiere que dos hilos se comuniquen y compartan una estructura complicada de datos, como una lista enlazada. En este caso será necesario asegurar que los dos hilos no entren en conflicto. Esto es, impedir que uno de los hilos escriba mientras el otro está realizando la lectura. Java implementa para este propósito un elegante modelo clásico de sincronización entre procesos, el monitor. El monitor es un mecanismo de control que fue definido por primera vez por C. A. R. Hoare. Se puede considerar un monitor como una pequeña caja que contiene solamente un hilo. Una vez que un hilo entra en el monitor, todos los demás hilos deben esperar hasta que el hilo salga del monitor. De esta forma, un monitor se puede utilizar para proteger el hecho de que varios hilos manipulen al mismo tiempo un recurso.

La mayor parte de los sistemas multihilo presentan los monitores como objetos que los programas deben obtener y manipular explícitamente. Java proporciona una solución más clara. No existe la clase “Monitor”; en su lugar, cada objeto tiene su propio monitor implícito que se introduce automáticamente cuando se llama a uno de los métodos sincronizados del objeto. Cuando un hilo está dentro de un método sincronizado, ningún otro hilo puede llamar a ningún otro método sincronizado del mismo objeto. Esto permitirá escribir un código multihilo muy claro y conciso, debido a que el soporte para la sincronización se encuentra establecido dentro del lenguaje.

A menudo los hilos necesitan comunicarse unos con otros, la forma de comunicarse consiste sualmente en compartir un objeto. En el siguiente ejemplo dos hilos comparten un objeto de la lase Contador. Esta clase define un atributo contador y tres métodos, uno de ellos incrementa una unidad su valor, el otro lo decrementa y el tercero devuelve su valor; el constructor asigna un valor inicial al contador:



Este ejemplo muestra una clase `Contador` en Java, que permite incrementar, decrementar y obtener el valor de un contador.

```java
public class Contador {
    private int c = 0; // atributo contador

    public Contador(int c) {
        this.c = c;
```

```
    }

    public void incrementa() {
        c = c + 1;
    }

    public void decrementa() {
        c = c - 1;
    }

    public int getValor() {
        return c;
    }
} // CONTADOR
```

Para probar el objeto compartido se definen dos clases que extienden Thread. En la clase Hilo se usa el método del objeto contador que incrementa en uno su valor. En la clase Hilo se usa el método que decrementa su valor. Se añade un sleepO intencionadamente para probar que un hilo se duerma y mientras el otro haga otra operación con el contador, así la CPU no realiza de una sola vez todo un hilo y después otro, y podemos observar mejor el efecto:



Este ejemplo muestra un hilo llamado `HiloA` que incrementa el valor de un contador compartido, heredando de la clase `Thread`.

```
public class HiloA extends Thread {
    private Contador contador;

    public HiloA(String n, Contador c) {
        setName(n);
        contador = c;
    }

    public void run() {
        for (int j = 0; j < 300; j++)
            contador.incrementa(); // incrementa el contador
        try {
            sleep(100);
        } catch (InterruptedException e) {}
        System.out.println(getName() + " contador vale " + contador.getValor());
    }
```

```
} // FIN HILOA
```



Este ejemplo muestra un hilo llamado `HiloB` que decrementa el valor de un contador compartido, heredando de la clase `Thread`.

```
public class HiloB extends Thread {
    private Contador contador;

    public HiloB(String n, Contador c) {
        setName(n);
        contador = c;
    }

    public void run() {
        for (int j = 0; j < 300; j++)
            contador.decrementa(); // decrementa el contador
        try {
            sleep(100);
        } catch (InterruptedException e) {}
        System.out.println(getName() + " contador vale " + contador.getValor());
    }
} // FIN HILOB
```

A continuación se crea el método main(), donde primero se define un objeto de la clase Contador y se le asigna el valor inicial de 100. A continuación se crean los dos hilos pasándoles dos parámetros: un nombre y el objeto Contador. Seguidamente se inicia la ejecución de los hilos:



Este ejemplo muestra la clase `CompartirContador`, que crea un contador compartido y lo pasa a dos hilos (`HiloA` y `HiloB`) para incrementar y decrementar su valor simultáneamente.

```
public class CompartirContador {

    public static void main(String[] args) {
```

```
        Contador contador = new Contador(100);
        HiloA a = new HiloA("HiloA", contador);
        HiloB b = new HiloB("HiloB", contador);

        a.start();
        b.start();

    }

}
```

Nos puedo dar la impresión de que al ejecutar los hilos, el valor de contador en el hilo A debería ser 400, ya que empieza en 100 y le suma 300; y en B, 100 ya que se resta 300; pero no es así. Al ejecutar el programa los valores de salida pueden no ser los esperados y variará de una ejecución a otra:

```
HiloA contador vale 100

HiloB contador vale 100
```

Una forma de evitar que esto suceda es hacer que las operaciones de incremento y decremento del objeto contador se hagan de forma atómica, es decir, si estamos realizando la suma nos aseguramos de que nadie realice la resta hasta que no terminemos la suma. Esto se puede lograr añadiendo la palabra synchronized a la parte de código que queramos que se ejecute de forma atómica. Java utiliza los bloques synchronized para implementar las regiones críticas . El formato es el siguiente:



Este ejemplo muestra el uso del bloque `synchronized` en Java, que sincroniza las sentencias críticas para evitar condiciones de carrera cuando varios hilos acceden al mismo recurso.

```
synchronized (object) {
    // sentencias críticas
}
```

Los métodos run() de las clases HiloA e HiloB se pueden sustitur por los siguientes:



Estos métodos `run` de las clases `HiloA` y `HiloB` usan un bloque `synchronized` para sincronizar el acceso al contador compartido, evitando condiciones de carrera.

```
public void run() {
    synchronized (contador) {
        for (int j = 0; j < 300; j++)
            contador.incrementa(); // incrementa el contador
    }
    try {
        sleep(100);
    } catch (InterruptedException e) {}
    System.out.println(getName() + " contador vale " + contador.getValor());
}

public void run() {
    synchronized (contador) {
        for (int j = 0; j < 300; j++)
            contador.decrementa(); // decrementa el contador
    }
    try {
        sleep(100);
    } catch (InterruptedException e) {}
    System.out.println(getName() + " contador vale " + contador.getValor());
}
```

El bloque synchronized lleva entre paréntesis la referencia al objeto contador. Cada vez que un hilo intenta acceder a un bloque sincronizado le pregunta a ese objeto si no hay algún otro hilo que ya le tenga bloqueado. Es decir, le pregunta si no hay otro hilo ejecutando algun bloque sincronizado con ese obieto. Si está tomado por otro hilo, entonces el hilo actual se suspende y se pone en espera hasta que se libere el bloqueo. Si está libre, el hilo actual bloquea el objeto y ejecuta el bloque; el siguiente hilo que intente ejecutar un bloque sincronizado con ese objeto, será puesto en espera. El bloqueo del objeto se libera cuando el hilo que lo tiene tomado sale del bloque porque termina la ejecución, ejecuta un return o lanza una excepción.

La compilación y ejecución muestra la siguiente salida:

```
HiloA contador vale 400

HiloB contador vale 100
```

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Monitores. Segmentos de código synchronized.

Hay casos en los que no se puede, o no interesa sincronizar un método. Por ejemplo, no podremos sincronizar un método que no hemos creado nosotros y que por tanto no podemos acceder a su código fuente para añadir synchronized en su definición. La forma de resolver esta situación es poner las llamadas a los métodos que se quieren sincronizar dentro de segmentos sincronizados de la siguiente forma: synchronized (objeto){ // sentencias segmento; }

En este caso el funcionamiento es el siguiente:

- El objeto que se pasa al segmento, es el objeto donde está el método que se quiere sincronizar.
- Dentro del segmento se hará la llamada al método que se quiere sincronizar.
- El hilo que entra en el segmento declarado synchronized se hará con el monitor del objeto, si está libre, o se bloqueará en espera de que quede libre. El monitor se libera al salir el hilo del segmento de código synchronized.
- Sólo un hilo puede ejecutar el segmento synchronized a la vez.

En el ejemplo del problema de los jardines, aplicando este procedimiento, habría que sincronizar el objeto que denominaremos jardin y que será desde donde se invoca al método incrementaCuenta(), método que manipula la variable cuenta que modifican diferentes hilos:

> El siguiente método `run()` simula 10 accesos incrementando la cuenta de accesos en el objeto `jardin` de forma sincronizada.

```
public void run() {
    // método que incrementa la cuenta de accesos
    for (int i = 1; i <= 10; i++) // se simulan 10 accesos
    {
        // segmento que se sincroniza
        synchronized (jardin) {
            jardin.incrementaCuenta();
        }
    }
}
```

Observa que:

- Ahora el método incrementaCuenta() no será synchronized (se haría igual para decrementaCuenta()),
- Se está consiguiendo un acceso con exclusión mutua sobre el objeto jardin, aún cuando su clase no contiene ningún segmento ni método synchronized.

En el siguiente enlace dispones del ejemplo completo con los cambios que acabamos de indicar.



Debes tener en cuenta que:

Declarar un método o segmento de código como sincronizado ralentizará la ejecución del programa, ya que la adquisición y liberación de monitores genera una sobrecarga.
Siempre que sea posible, por legibilidad del código, es mejor sincronizar métodos completos.
Al declarar bloques synchronized puede aparecer un nuevo problema, denominado interbloqueo (lo veremos más adelante).

> Muchos métodos de las clases predefinidas de Java ya están sincronizados. Por ejemplo, el método de la clase Component de Java AWT que agrega un objeto MouseListener a un Component (para que MouseEvents se registren en el MouseListener) está sincronizado. Si compruebas el código fuente de AWT y Swing, encontrarás que el prototipo de este método es: public synchronized void addMouseListener(MouseListener l)

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

¿Dónde usar synchronized?

Se debe evitar la sincronización de bloques de código y sustituirlas siempre que sea posible por la sincronización de métodos. Imaginemos la situación que dos personas comparten una cuenta y pueden sacar dinero de ella en cualquier momento; antes de retirar dinero se comprueba siempre si existe saldo. La cuenta tiene 50€, una de las personas quiere retirar 40 y la otra 30. La primera llega al cajero, revisa el saldo, comprueba que hay dinero y se prepara para retirar el dinero, pero antes de retirarlo llega la otra persona a otro cajero, comprueba el saldo que todavia muestra los 50€ y también se dispone a retirar el dinero. Las dos personas retiran el dinero, pero entonces el saldo actual será ahora de -20. Veamos mediante clases como sería el ejemplo.

Se define la clase Cuenta, define un atributo saldo y tres métodos, uno devuelve el valor del saldo, otro resta al saldo una cantidad y el tercero realiza las comprobaciones para hacer la retirada de dinero, es decir que el saldo actual sea > que la cantidad que se quiere retirar; el constructor inicia el saldo actual. También se añade un sleep0 intencionadamente para probar que un hilo se duerma mientras el otro haga la operaciones:



Este ejemplo muestra la clase `Cuenta`, que permite realizar operaciones de retiro de saldo. Incluye verificación de fondos y manejo de condiciones de saldo insuficiente.

```
class Cuenta {
    private int saldo;

    public Cuenta(int s) {
        this.saldo = s;
    }

    int getSaldo() {
        return saldo;
    }

    void restar(int cantidad) {
        saldo = saldo - cantidad;
    }

    void retirarDinero(int cant, String nombre) {
        if (getSaldo() >= cant) {
            System.out.println("SE VA A RETIRAR SALDO (ACTUAL ES:" + getSaldo() +
```

```
")");
            try {
                Thread.sleep(500);
            } catch (InterruptedException ex) {
                ex.printStackTrace();
            }
            restar(cant);
            System.out.println(nombre + " retira =>" + cant + " ACTUAL(" +
getSaldo() + ")");
        } else {
            System.out.println(nombre + " No puede retirar dinero, NO HAY SALDO("
+ getSaldo() + ")");
        }

        if (getSaldo() < 0) {
            System.out.println("SALDO NEGATIVO =>" + getSaldo());
        }
    }
}
```

A continuación se crea la clase SacarDinero que extiende Thread y usa la clase Cuenta pul compartir dinero. El constructor recibe una cadena, para dar nombre al hilo. y la cuenta que se contempar iRa por varias personas. En el método runO se realizare a bule Y n de se invoca método RetirarDinero0 de la clase Cuenta varias veces realiza un bucle dorrar, en este di siempre es 10, y el nombre del hilo:



Este ejemplo muestra la clase `SacarDinero`, que extiende `Thread` y permite que varios hilos intenten retirar dinero de la misma cuenta.

```
public class SacarDinero extends Thread {

    private Cuenta c;

    public SacarDinero(String nombre, Cuenta c) {
        super(nombre);
        this.c = c;
    }

    public void run() {
        for (int x = 1; x <= 4; x++) {
```

```
            c.retirarDinero(10, getName());
        }
    }

}
```

Por último se crea el método main0, donde primero se define un objeto de la clase Cuenta y se le asigna un saldo inicial de 40. A continuación se crean dos objetos de la clase SacarDinero, imaginemos que son las dos personas que comparten la cuenta, y se inician los hilos:



Este ejemplo muestra la clase `Cajero`, que crea una cuenta compartida entre dos hilos (`SacarDinero`) que intentan retirar dinero simultáneamente.

```
public class Cajero {

    public static void main(String[] args) {

        Cuenta c = new Cuenta(40);
        SacarDinero h1 = new SacarDinero("Ana", c);
        SacarDinero h2 = new SacarDinero("Juan", c);

        h1.start();
        h2.start();

    }

}
```

Compilamos y ejecutamos. Observamos que se ha permitido retirar saldo cuando este era 0:

```
Ana retira =>10 ACTUAL(30)

Juan retira =>10 ACTUAL(30)

SE VA A RETIRAR SALDO (ACTUAL ES:30)

SE VA A RETIRAR SALDO (ACTUAL ES:30)
```

```
Ana retira =>10 ACTUAL(20)

SE VA A RETIRAR SALDO (ACTUAL ES:20)

Juan retira =>10 ACTUAL(20)

SE VA A RETIRAR SALDO (ACTUAL ES:20)

Ana retira =>10 ACTUAL(10)

Juan retira =>10 ACTUAL(10)

SE VA A RETIRAR SALDO (ACTUAL ES:10)

SE VA A RETIRAR SALDO (ACTUAL ES:10)

Ana retira =>10 ACTUAL(0)

Juan retira =>10 ACTUAL(-10)
```

Para evitar esta situación la operación de retirar dinero, método Retirar Dinero0 de la clase Cuenta, debería ser atómica e indivisible, es decir, si una persona está retirando dinero, la otra debería ser incapaz de retirarlo hasta que la primera haya realizado la operación. Para ello declaramos el método como synchronized. Cuando un hilo invoca un método synchronized, trata de tomar el bloqueo del obieto a que pertenezca. Si está libre, lo toma y se ejecuta. Si el bloqueo está tomado por otro hilo se suspende el que invoca hasta que aquel finalice y libere el bloqueo. La forma de declararlo es la siguiente:



Este ejemplo muestra un método `synchronized` en Java. Al declarar el método como sincronizado, se asegura que solo un hilo a la vez pueda ejecutarlo, garantizando la atomicidad de las instrucciones.

```java
synchronized public void metodo() {
    // instrucciones atómicas
}
```

El método retirarDinero() en nuestro ejemplo quedaría así:



Este ejemplo muestra el método `retirarDinero` declarado como `synchronized`, lo que garantiza que solo un hilo a la vez pueda acceder a esta sección crítica, evitando problemas de concurrencia.

```
synchronized void retirarDinero(int cant, String nombre) {
    // las mismas instrucciones que antes
}
```

La ejecución mostraría la siguiente salida, recuerda que de una ejecución a otra puede variar, pero en este caso el saldo no será negativo:

```
SE VA A RETIRAR SALDO (ACTUAL ES:40)

SE VA A RETIRAR SALDO (ACTUAL ES:40)

Ana retira =>10 ACTUAL(20)

SE VA A RETIRAR SALDO (ACTUAL ES:20)

Juan retira =>10 ACTUAL(20)

SE VA A RETIRAR SALDO (ACTUAL ES:20)

Juan retira =>10 ACTUAL(10)

Ana retira =>10 ACTUAL(10)

SE VA A RETIRAR SALDO (ACTUAL ES:10)

SE VA A RETIRAR SALDO (ACTUAL ES:10)

Juan retira =>10 ACTUAL(0)

Ana retira =>10 ACTUAL(0)

Juan No puede retirar dinero, NO HAY SALDO(0)

Ana No puede retirar dinero, NO HAY SALDO(0)
```

Se debe tener en cuenta que la sincronización disminuye el rendimiento de una aplicación, por tanto, debe emplearse solamente donde sea estrictamente necesario.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Comunicación entre hilos con métodos de java.lang.Object

La comunicación entre hilos la podemos ver como un mecanismo de auto-sincronización, que consiste en lograr que un hilo actúe sólo cuando otro ha concluido cierta actividad (y viceversa).

Java soporta comunicación entre hilos mediante los siguientes métodos de la clase java.lang.Object.

- **wait**(). Detiene el hilo (pasa a "no ejecutable"), el cual no se reanudará hasta que otro hilo notifique que ha ocurrido lo esperado.
- **wait**(long tiempo). Como el caso anterior, solo que ahora el hilo también puede reanudarse (pasar a "ejecutable") si ha concluido el tiempo pasado como parámetro.
- **notify**(). Notifica a uno de los hilos puestos en espera para el mismo objeto, que ya puede continuar.
- **notifyAll**(). Notifica a todos los hilos puestos en espera para el mismo objeto que ya pueden continuar

> La llamada a estos métodos se realiza dentro de bloques synchronized.

¿Cómo funcionan realmente estos métodos?

wait()

Detiene al hilo que lo invoca hasta que le sea notificada la posibilidad de continuar.

- El método wait() se debe invocar sobre un objeto compartido por los hilos a sincronizar.
- Para poder invocar a wait() el hilo debe tener el mutex del objeto compartido.
- La invocación de wait() detiene al hilo, pasa a “no ejecutable”, lo pone en una cola de espera asociada al objeto (cola wait del objeto), y libera el mutex del objeto.
- El método wait() puede provocar una InterruptedException.

notify()

Notifica a un hilo que invocó wait() sobre el mismo objeto y que está en la cola de espera del objeto (cola wait), que ya puede continuar.

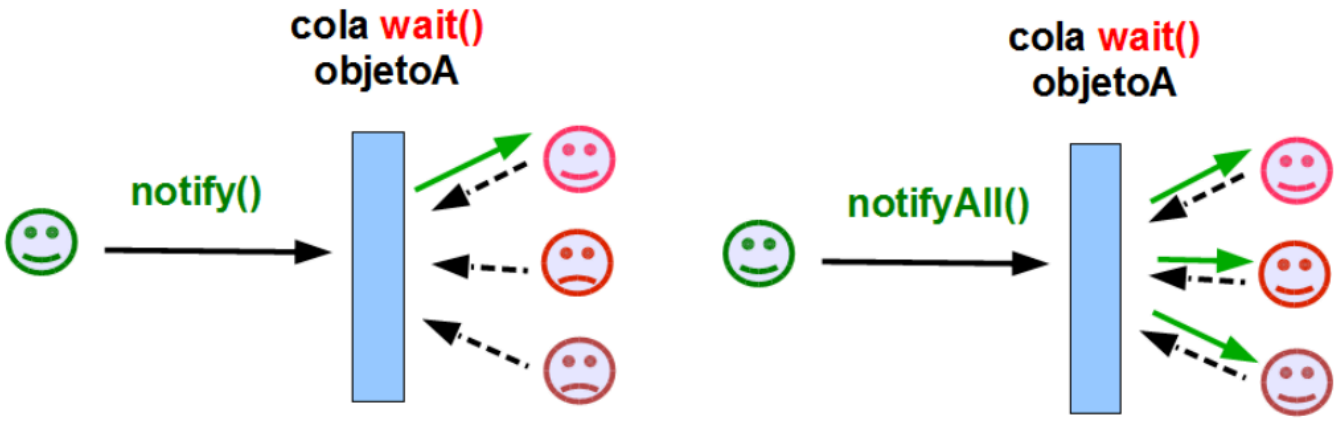
- Un hilo sale de la cola de espera del objeto (cola wait) pasando al estado “ejecutable”, y se bloquea hasta conseguir el mutex del objeto para continuar su ejecución.
- Si hay más de un hilo en la cola de espera (cola wait), notify() reactivará solo a uno de ellos. El criterio de selección del hilo a reactivar o pasar a “ejecutable”

depende de la implementación de Java.

- Una vez re-obtenido el mutex del objeto, el hilo que salió de la lista de espera (cola wait) continuará la ejecución del método en la instrucción siguiente a la llamada a wait().

notifyAll()

Notifica a todos los hilos puestos en espera para el mismo objeto (cola wait) que ya pueden continuar.



El hilo que invoca notify() no tiene ninguna referencia del hilo que está en espera por haber invocado wait() sobre el mismo objeto. Cuando un hilo invoca a
notify(), otro hilo (no se sabe cual) de los que están en espera es reactivado (pasa a "ejecutable").
Con notifyAll() se reactivan, volverán "ejecutables" todos los hilos que estaban bloqueados en la cola de espera del objeto. Sin embargo el mutex, solo podrán tomarlo de uno en uno.

Dos problemas clásicos que permiten ilustrar la necesidad de sincronizar y comunicar hilos son:

- El problema del Productor-Consumidor. Del que has visto un ejemplo anteriormente y que permite modelar situaciones en las que se divide el trabajo entre los hilos. Modela el acceso simultáneo de varios hilos a una estructura de datos u otro recurso, de manera que unos hilos producen y almacenan los datos en el recurso y otros hilos (consumidores) se encargan de eliminar y procesar esos datos.
- El problema de los Lectores-Escritores. Permite modelar el acceso simultáneo de varios hilos a una base de datos, fichero u otro recurso, unos queriendo leer y otros escribir o modificar los datos.

En el siguiente recurso tienes el ejemplo del problema de los lectores-escritores, resuelto mediante los métodos wait() y notify() vistos anteriormente.



En el siguiente enlace puedes ver otro ejemplo de Productor_Consumidor.



Explicación:

- Clase Buffer: Actúa como el recurso compartido, con una capacidad limitada. Utiliza una cola circular para almacenar los elementos producidos. La clase incluye métodos sincronizados producir() y consumir() que usan wait() y notifyAll() para coordinar el acceso entre el productor y el consumidor.
- Clase Productor: En su método run(), genera elementos y los coloca en el buffer. Si el buffer está lleno, se bloquea hasta que haya espacio disponible.
- Clase Consumidor: En su método run(), retira elementos del buffer. Si el buffer está vacío, se bloquea hasta que haya elementos disponibles.
- Métodos wait() y notifyAll(): Se utilizan para que el productor espere cuando el buffer está lleno y el consumidor espere cuando está vacío. notifyAll() despierta a todos los hilos esperando en el monitor del buffer, permitiendo que el otro hilo (productor o consumidor) actúe cuando las condiciones cambian.
- Demo (ProductorConsumidorDemo): Crea un buffer de tamaño 5 y lanza un hilo productor y un hilo consumidor para interactuar con el buffer.

Este código implementa la sincronización adecuada para evitar que el productor agregue elementos al buffer cuando está lleno o que el consumidor intente retirar elementos cuando está vacío.

Diferencias entre wait() y notify()

wait()	notify()
Cuando se llama al método wait() en un hilo, libera el bloqueo de monitor que tiene y entra en un estado de espera.	El método notify() no libera el bloqueo. Simplemente indica a los otros hilos que el bloqueo podría liberarse pronto. La liberación

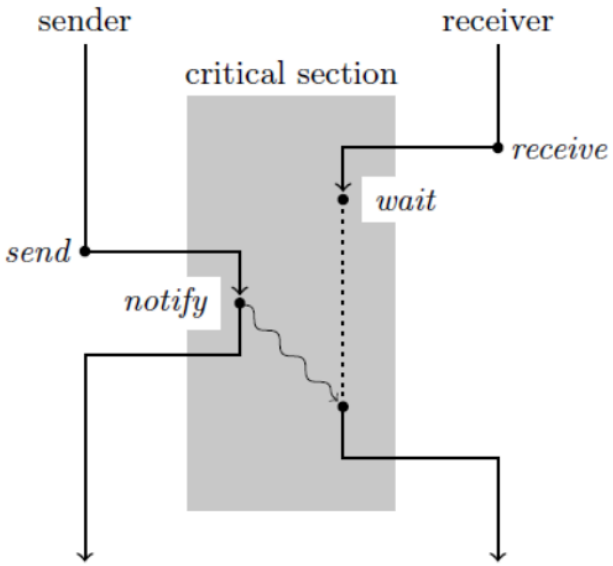
wait()	notify()
	real del bloqueo ocurre cuando el hilo actual termina de ejecutar el bloque sincronizado.
Varios hilos pueden estar en espera al mismo tiempo.	Cuando un hilo llama al método notify() en un objeto, se selecciona aleatoriamente un hilo en espera en el objeto y se le notifica. El hilo notificado sale del estado de espera y entra en el estado bloqueado.
En la clase Object se define el método wait().	El método notify() también se define en la clase Object.
En la comunicación entre hilos, se usa el método wait().	Para despertar un hilo, se utiliza el método notify().
El método wait() está definido en la clase java.lang.Object.	El método notify() no tiene un valor de retorno.
El método wait() requiere un bloqueo de sincronización para ser llamado.	El método notify() despierta a uno de los hilos que están en espera en un objeto particular.

¿Por qué encerrar wait() en un bucle while?

Envolver wait() en un bucle while es una práctica recomendada para asegurar que la condición que un hilo espera realmente se cumple antes de continuar. Esto se debe a dos razones principales:

1. Despertar aleatorio con notify() o notifyAll(): Los métodos notify() y notifyAll() despiertan hilos esperando en el monitor del objeto de forma no determinista. Es posible que un hilo se despierte incluso si la condición que estaba esperando no se cumple todavía. Envolver wait() en un bucle while permite que el hilo vuelva a comprobar la condición después de despertarse y solo continúe si la condición es verdadera.
2. Despertares espurios (spurious wakeups): En algunos entornos de ejecución, un hilo que llama a wait() puede despertarse sin recibir una notificación (notify() o notifyAll()). Este "despertar espurio" ocurre de forma inesperada y sin motivo aparente, lo que podría hacer que el hilo continúe incorrectamente si no vuelve a comprobar la condición.

Esta imagen representa un diagrama de sincronización entre dos hilos o procesos, un emisor (sender) y un receptor (receiver), que se comunican y sincronizan a través de una sección crítica usando los métodos wait() y notify().



1. Sección Crítica:
    1. La sección crítica es el área sombreada en el centro del diagrama. Es una porción de código que necesita protección para asegurar que solo un hilo o proceso acceda a ella a la vez.
2. Emisor (Sender):
    - El emisor ejecuta la acción send, que inicia el proceso de comunicación y trata de ingresar a la sección crítica.
    - Una vez que el emisor está en la sección crítica, realiza alguna operación.
    - Después de realizar su tarea, el emisor llama a notify() para informar al receptor que ha terminado y que la sección crítica está disponible.
3. Receptor (Receiver):
    - El receptor comienza intentando recibir (receive), pero si la sección crítica está ocupada (es decir, el emisor está en ella), el receptor entra en un estado de espera usando wait().
    - El receptor espera a que el emisor complete su tarea y llame a notify() para salir de wait().
    - Una vez que el receptor recibe la notificación, puede ingresar a la sección crítica y realizar su tarea.
4. Sincronización:
    - El método wait() hace que el receptor espere hasta que el emisor complete su tarea y llame a notify().
    - notify() permite que el receptor salga del estado de espera y continúe con su ejecución.
    - Esto asegura que el emisor y el receptor no entren en la sección crítica al mismo tiempo, evitando conflictos y garantizando la consistencia de los datos compartidos.

Este ejemplo sigue el patrón del diagrama: el emisor envía un mensaje y notifica al receptor, quien espera para recibirlo. Luego el receptor notifica al emisor cuando ha recibido el mensaje, lo que permite la sincronización y evita el acceso simultáneo a la sección crítica.



Explicación del Código

- Comunicador:
  - Tiene un método enviarMensaje que simula el envío de un mensaje por parte del emisor.
  - El método recibirMensaje simula la recepción de un mensaje por parte del receptor.
  - Se utiliza wait() y notify() para coordinar que el emisor envíe el mensaje antes de que el receptor intente recibirlo, y viceversa.
- Hilos emisor y receptor:
  - El hilo emisor envía tres mensajes consecutivos.
  - El hilo receptor espera recibir tres mensajes, usando el mecanismo de sincronización para garantizar que la comunicación sea secuencial y sin conflictos.

Saber más

En el siguiente enlace encontrarás información y un ejemplo de lo que se conoce como el búfer circular.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

El problema del interbloqueo (deadlock)

El interbloqueo o bloqueo mutuo (deadlock) consiste en que uno a más hilos se bloquean o esperan indefinidamente.

¿Cómo se llega a una situación de interbloqueo? A dicha situación se llega

- Porque cada hilo espera a que le llegue un aviso de otro hilo que nunca le llega.
- Porque todos los hilos, de forma circular, esperan para acceder a un recurso.

El problema del bloqueo mutuo, en las aplicaciones concurrentes, se podrá dar fundamentalmente cuando un hilo entra en un bloque synchronized, y a su vez llama a otro bloque synchronized, o bien al utilizar clases de java.util.concurrent que llevan implícita la exclusión mutua.

En el siguiente enlace puedes consultar un artículo y ejemplo detallado sobre el interbloqueo.

En el siguiente enlace encontrarás un ejemplo que produce interbloqueo. Observa que no finaliza la ejecución del programa y que en la barra de estado del IDE NetBeans (a la derecha) aparece la indicación de programa bloqueado. Habrá que finalizar manualmente el programa.



En este ejemplo, se utilizan dos hilos (Thread1 y Thread2) y dos recursos (lock1 y lock2). Cada hilo intenta bloquear ambos recursos, pero en un orden diferente, lo que puede llevar a un interbloqueo.



Explicación:

1. Objetos lock1 y lock2: Representan dos recursos que los hilos intentarán bloquear.
2. Hilo 1:

- Bloquea lock1 usando synchronized(lock1).
- Luego, intenta bloquear lock2.
- Este hilo hará una pausa (Thread.sleep(100)) antes de intentar bloquear lock2 para simular una posible condición de interbloqueo.

3. Hilo 2:
   - Bloquea lock2 usando synchronized(lock2).
   - Luego, intenta bloquear lock1.
   - También tiene una pausa antes de intentar bloquear lock1 para simular la condición de interbloqueo.
4. Interbloqueo:
   - Si Thread1 bloquea lock1 y Thread2 bloquea lock2, cada hilo intentará bloquear el recurso que el otro ya tiene bloqueado. Como ambos están esperando al otro para liberar el recurso, ninguno podrá avanzar y el programa quedará en un estado de interbloqueo.

Cómo identificar el interbloqueo:
Cuando ejecutas este código, el programa se "colgará" después de imprimir que cada hilo ha bloqueado su primer recurso. Ningún hilo podrá avanzar porque ambos están esperando indefinidamente. Esta es una manifestación clásica de un interbloqueo.

Otro problema, menos frecuente, es la inanición (starvation), que consiste en que un hilo es desestimado para su ejecución. Se produce cuando un hilo no puede tener acceso regular a los recursos compartidos y no puede avanzar, quedando bloqueado. La inanición puede ocurrir cuando un hilo no tiene la oportunidad de ejecutarse debido a que otros hilos constantemente tienen prioridad o están acaparando el recurso compartido.

En este ejemplo, tenemos varios hilos con diferentes prioridades que intentan acceder a un recurso compartido. Los hilos de alta prioridad ocupan el recurso constantemente, y un hilo de baja prioridad nunca obtiene la oportunidad de ejecutarse, lo que resulta en inanición para el hilo de baja prioridad.



Explicación:

1. Recurso compartido: La variable recurso es un objeto sobre el cual se sincronizan los hilos para simular el acceso a un recurso compartido.

2. Hilos de alta prioridad: Creamos dos hilos de alta prioridad (Thread.MAX_PRIORITY). Estos hilos acaparan el recurso casi todo el tiempo debido a su alta prioridad.
3. Hilo de baja prioridad: Creamos un hilo con la prioridad mínima (Thread.MIN_PRIORITY). Este hilo intentará acceder al recurso, pero los hilos de alta prioridad casi siempre tienen preferencia, lo que resulta en inanición para este hilo de baja prioridad.
4. Clase Tarea: Cada hilo ejecuta esta tarea en un bucle infinito:
    - Intenta obtener el recurso mediante synchronized (recurso).
    - Una vez dentro de la sección crítica, simula el uso del recurso durante 50 milisegundos (Thread.sleep(50)).
    - Luego, libera el recurso y duerme por un breve período para permitir que otros hilos intenten acceder al recurso.

Resultado:
Cuando ejecutas este programa, verás que los hilos de alta prioridad continuamente obtienen el recurso y lo liberan, mientras que el hilo de baja prioridad tiene muy pocas oportunidades para acceder al recurso. Esto causa inanición para el hilo de baja prioridad, ya que se ve impedido de acceder al recurso de forma justa debido a la constante preferencia dada a los hilos de alta prioridad.

Está claro que los programas que desarrollemos deben estar exentos de estos problemas, por lo que habrá que ser cuidadosos en su diseño. La inanición es un problema común en sistemas donde se utilizan prioridades y puede ser mitigado en algunos sistemas de planificación de hilos mediante técnicas como la **promoción de prioridad dinámica**, que aumenta la prioridad de los hilos que han estado esperando demasiado tiempo**.**

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Lock

La interfaz Lock de Java, java.util.concurrent.locks.Lock, representa un bloqueo concurrente que se puede utilizar para proteger contra condiciones de carrera dentro de secciones críticas. Así, la interfaz Lock de Java ofrece una alternativa más flexible a un bloque synchronized en Java. En este tutorial de Lock en Java, explicaré cómo funciona la interfaz Lock y cómo usarla.

Principales diferencias entre un Lock y un bloque synchronized:

- Un bloque synchronized no garantiza el orden en que los hilos que esperan entrar en él obtendrán acceso.
- No se pueden pasar parámetros al intentar entrar en un bloque synchronized. Por lo tanto, no es posible establecer un tiempo de espera para obtener acceso a un bloque synchronized.
- Un bloque synchronized debe estar completamente contenido dentro de un único método. En cambio, un Lock puede tener sus llamadas a lock() y unlock() en métodos separados.

Implementaciones de Lock en Java

Dado que Lock es una interfaz, no se puede crear una instancia de Lock directamente. Debes crear una instancia de una clase que implemente la interfaz Lock. El paquete java.util.concurrent.locks tiene las siguientes implementaciones de la interfaz Lock:

- java.util.concurrent.locks.ReentrantLock

En las siguientes secciones, explicaré cómo usar la clase ReentrantLock como un Lock.

Crear un ReentrantLock

Para crear una instancia de la clase ReentrantLock, simplemente usa el operador new, de la siguiente manera:

```
Lock lock = new ReentrantLock();
```

Ahora tienes una instancia de Lock en Java, en realidad una instancia de ReentrantLock.

Bloqueo y desbloqueo de un Lock en Java

Dado que Lock es una interfaz, debes utilizar una de sus implementaciones para usar un Lock en tus aplicaciones. En el siguiente ejemplo, creo una instancia de ReentrantLock. Para bloquear la instancia de Lock, debes llamar a su método lock(). Para desbloquearla, debes

llamar a su método unlock(). Aquí tienes un ejemplo de bloqueo y desbloqueo de una instancia de Lock en Java:

```
Lock lock = new ReentrantLock();

 lock.lock();

// sección crítica

lock.unlock();
```

Primero, se crea un Lock. Luego, se llama a su método lock(). Ahora la instancia de Lock está bloqueada. Cualquier otro hilo que llame a lock() será bloqueado hasta que el hilo que bloqueó el Lock llame a unlock(). Finalmente, se llama a unlock(), y el Lock se desbloquea, permitiendo que otros hilos puedan bloquearlo.

Es importante que todos los hilos compartan la misma instancia de Lock. Si cada hilo crea su propia instancia de Lock, estarán bloqueando en diferentes objetos, y por lo tanto, no se bloquearán entre sí. Más adelante en este tutorial de Lock en Java, te mostraré un ejemplo de cómo se ve una instancia de Lock compartida.

Bloqueo y desbloqueo seguro

Si observas el ejemplo de la sección anterior, imagina lo que sucedería si se lanza una excepción entre la llamada a lock.lock() y lock.unlock(). La excepción interrumpiría el flujo del programa y lock.unlock() nunca se ejecutaría. Como resultado, el Lock permanecería bloqueado indefinidamente.

Para evitar que una excepción deje bloqueado un Lock para siempre, deberías bloquear y desbloquear dentro de un bloque try-finally, de esta forma:

```
Lock lock = new ReentrantLock();

try { lock.lock();

  // sección crítica

} finally {
```

```
lock.unlock();
}
```

De esta manera, el Lock se desbloqueará incluso si se lanza una excepción dentro del bloque try.

Ejemplo de contador protegido con Lock

Para entender mejor cómo el uso de un Lock se diferencia de un bloque synchronized, he creado dos clases simples de contadores concurrentes que protegen su conteo interno de diferentes maneras. La primera clase usa un bloque synchronized, y la segunda clase usa un Lock en Java:



En la primera clase, SynchronizedCounter, el método increment() y el método getCount() están protegidos mediante el uso de synchronized, lo que asegura que solo un hilo puede acceder a estos métodos a la vez.

En la segunda clase, LockCounter, los métodos increment() y getCount() están protegidos usando un Lock. Aquí, el método lock() asegura que un solo hilo acceda a la sección crítica a la vez, mientras que unlock() libera el bloqueo, permitiendo que otros hilos entren en la sección crítica. Además, al usar un bloque try-finally, se garantiza que el Lock se libere incluso si ocurre una excepción.

Observa que la clase CounterLock es más extensa que la clase CounterSynchronized. Sin embargo, usar un Lock en Java para proteger la variable count internamente puede ofrecer un mayor grado de flexibilidad, en caso de que la necesites. Estos ejemplos simples no requieren realmente esa flexibilidad, pero contadores más avanzados podrían beneficiarse de ella.

Reentrada en Lock

Un Lock se denomina reentrante si el hilo que posee el bloqueo puede bloquearlo nuevamente. Un Lock no reentrante es aquel que no se puede volver a bloquear si ya está bloqueado, ni siquiera por el hilo que tiene el bloqueo. Los bloqueos no reentrantes pueden llevar a una situación de bloqueo por reentrada, similar a un deadlock.

La clase ReentrantLock es un bloqueo reentrante. Esto significa que, incluso si un hilo tiene el bloqueo, puede volver a bloquearlo. En consecuencia, el hilo debe desbloquearlo tantas veces como lo haya bloqueado para liberar completamente el bloqueo y permitir que otros hilos lo adquieran.

Un Lock reentrante es útil en ciertos diseños concurrentes. A continuación, se muestra una implementación concurrente de una calculadora. La calculadora puede mantener el resultado actual internamente y ofrece un conjunto de métodos para realizar cálculos sobre ese resultado.



En esta implementación, la calculadora protege su variable result utilizando un ReentrantLock, lo que le permite realizar operaciones de manera segura en un entorno concurrente.

Observa cómo el método calculate() bloquea el Lock de la instancia de Calculator antes de realizar cualquier cálculo y también llama a los métodos add() y subtract(), los cuales también bloquean el mismo Lock. Gracias a que ReentrantLock es reentrante, esto no causa problemas. El hilo que llama a calculate() puede volver a bloquear el Lock sin bloquearse a sí mismo.

Equidad en los Lock

Un Lock no equitativo no garantiza el orden en que los hilos en espera para bloquear el Lock obtendrán acceso a él. Esto significa que un hilo en espera podría quedarse esperando indefinidamente si otros hilos siguen intentando bloquear el Lock y reciben prioridad sobre el hilo en espera. Esta situación puede llevar a una inanición (starvation). Este tema se trata en más detalle en mi tutorial sobre inanición y equidad.

El comportamiento de ReentrantLock es no equitativo por defecto. Sin embargo, puedes configurarlo para que opere en modo equitativo a través de su constructor. La clase ReentrantLock tiene un constructor que toma un parámetro booleano que especifica si el ReentrantLock debe proporcionar equidad o no a los hilos en espera. Aquí tienes un ejemplo de cómo crear una instancia de ReentrantLock en modo equitativo:

```
Lock fairLock = new ReentrantLock(true);
```

En este ejemplo, el parámetro true le indica a ReentrantLock que debe operar en modo equitativo, lo que significa que el hilo que ha esperado más tiempo obtendrá el Lock cuando

esté disponible.

en en cuenta que el método tryLock() (que se verá más adelante en este tutorial de Lock en Java) sin parámetros no respeta el modo de equidad de ReentrantLock. Para obtener equidad, debes usar el método tryLock(long timeout, TimeUnit unit), de la siguiente manera:

```
lock.tryLock(0, TimeUnit.SECONDS);
```

En este caso, el uso de tryLock con un tiempo de espera específico asegura que el Lock respete la política de equidad establecida, ya que este método permite que los hilos esperen su turno de acuerdo con el modo de equidad configurado en el ReentrantLock.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

La clase Semaphore

El uso de la palabra clave synchronized permite establecer secciones críticas a las que únicamente tiene acceso un hilo en un momento determinado, pero existen otros escenarios en los que los recursos limitados permiten el acceso de más de un hilo.

Cuando una sección crítica permite ser ejecutada por más de un hilo, pero el número de estos está limitado se utiliza un elemento de programación concurrente conocido como semáforo e implementado en Java por la clase Semaphore.

Los semáforos se suelen utilizar cuando un recurso tiene una capacidad limitada y se desea controlar el número de consumidores de dicho recurso. Es, por lo tanto, un elemento de sincronización.

Al construir el semáforo se le proporciona a través del constructor una capacidad (contador) que hace referencia al número de hilos que puede estar ejecutando concurrentemente, es decir, el número de recursos disponibles y que se compartirán entre los hilos. En otras palabras, este capacidad o contador se convierte en el número de permisos de acceso que se conceden a los bloques de código que quieran hacer uso del recurso limitado. Si todos los permisos están concedidos, los hilos quedan en espera a que los propietarios de los permisos los liberen.

La clase Semaphore del paquete java.util.concurrent, permite definir un semáforo para controlar el acceso a un recurso compartido. La clase java.util.concurrent.Semaphore es un semáforo contador. Esto significa que tiene dos métodos principales:

- acquire()
- release()

El semáforo contador se inicializa con un número determinado de "permisos". Por cada llamada a acquire(), un permiso es tomado por el hilo que llama. Por cada llamada a release(), un permiso es devuelto al semáforo. Así, como máximo N hilos pueden pasar el método acquire() sin que se realicen llamadas a release(), donde N es el número de permisos con los que se inicializó el semáforo. Los permisos son simplemente un contador. No hay nada complejo aquí.

Uso del Semaphore

Un semáforo típicamente tiene dos usos:

- Proteger una sección crítica para evitar la entrada de más de N hilos a la vez.

- Enviar señales entre dos hilos.

Protección de Secciones Críticas

Si usas un semáforo para proteger una sección crítica, el hilo que intenta entrar en la sección crítica típicamente intentará primero adquirir un permiso, entrar en la sección crítica y luego liberar el permiso después. Así:



El siguiente código muestra el uso de un `Semaphore` en Java para controlar el acceso a una sección crítica, asegurando que solo un hilo a la vez pueda ejecutarlo.

```
Semaphore semaphore = new Semaphore(1);

try {
    semaphore.acquire();
    // sección crítica
} finally {
    semaphore.release();
}
```

En este ejemplo, el semáforo se inicializa con 1 permiso, permitiendo que solo un hilo entre en la sección crítica a la vez. Otros hilos que intenten llamar a acquire() mientras el permiso esté ocupado quedarán en espera hasta que se libere.

En el siguiente ejemplo, se construye con semáforo con capacidad para tres permisos. A su vez, cinco hilos construidos a partir de la misma instancia de Runnable acceden a la sección crítica, solicitan el permiso, realizan los pasos de la actividad sincronizada y liberan el bloqueo.



Esta clase `SemaforoBasico` utiliza un semáforo para controlar el acceso concurrente. El semáforo permite que un máximo de tres hilos ejecuten la sección crítica al mismo tiempo, mientras que los demás deben esperar.

```
import java.util.concurrent.Semaphore;

public class SemaforoBasico implements Runnable {

    Semaphore semaforo = new Semaphore(3);
```

```
    @Override
    public void run() {
        try {
            semaforo.acquire();
            System.out.println("Paso 1");
            Thread.sleep(1000);
            System.out.println("Paso 2");
            Thread.sleep(1000);
            System.out.println("Paso 3");
            Thread.sleep(1000);
            semaforo.release();

        } catch (InterruptedException ie) {
            ie.printStackTrace();
        }
    }

    public static void main(String[] args) {
        SemaforoBasico sb = new SemaforoBasico();
        for (int i = 0; i < 5; i++) {
            new Thread(sb).start();
        }
    }
}
```

La salida muestra que los tres primeros hilos que han intentado acceder a la sección crítica a través del bloqueo del semáforo se han ejecutado concurrentemente, y el resto de los hilos se han ejecutado cuando los primeros han terminado y liberado los bloqueos.

```
Paso 1
Paso 1
Paso 1
Paso 2
Paso 2
Paso 2
Paso 3
Paso 3
Paso 3
Paso 1
Paso 1
```

```
Paso 2
Paso 2
Paso 3
Paso 3
```

No solo se pueden utilizar los semáforos cuando los recursos son limitados. En ocasiones es conveniente limitar el número de hilos que realizan determinadas tareas para no saturar el sistema o alguno de sus componentes. Por ejemplo, limitar el número de hilos que acceden a servicios web o a bases de datos puede ser una estrategia adecuada en muchos casos.

> **Nota**
>
> **El concepto de semáforo fue introducido por Edsger Dijkstra en 1965 y se utilizó por primera vez en el sistema operativo THEOS.**

Creación y uso de semáforos

Para crear y usar un objeto Semaphore haremos lo siguiente:

- Indicar al constructor Semaphore (int permisos) el total de permisos que se pueden dar para acceder al mismo tiempo al recurso compartido. Este valor coincide con el número de hilos que pueden acceder a la vez al recurso.
- Indicar al semáforo mediante el método acquire(), que queremos acceder al recurso, o bien mediante acquire(int permisosAdquirir) cuántos permisos se quieren consumir al mismo tiempo.
- Indicar al semáforo mediante el método release(), que libere el permiso, o bien mediante release(int permisosLiberar), cuantos permisos se quieren liberar al mismo tiempo.
- Hay otro constructor Semaphore (int permisos, boolean justo) que mediante el parámetro justo permite garantizar que el primer hilo en invocar adquire() será el primero en adquirir un permiso cuando sea liberado. Esto es, garantiza el orden de adquisición de permisos, según el orden en que se solicitan.

¿Desde dónde se deben invocar estos métodos? Esto dependerá del uso de Semaphore.

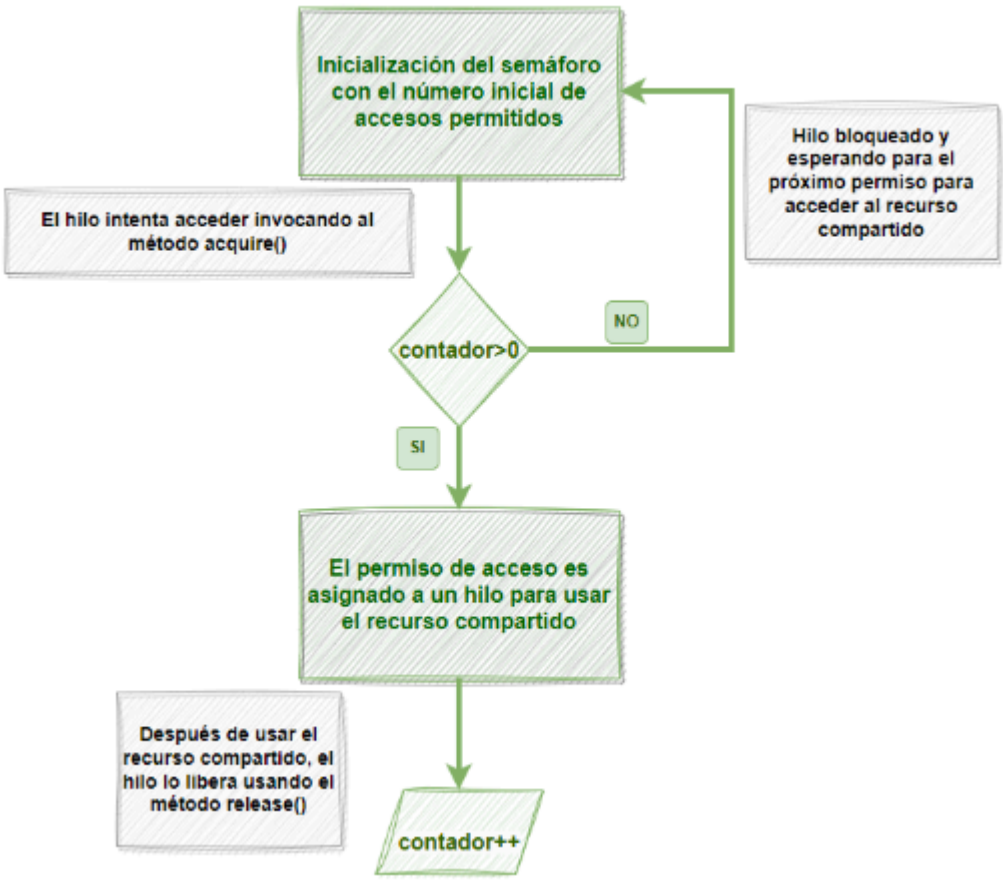
- Si se usa para proteger secciones críticas, la llamada a los métodos acquire() y release() se hará desde el recurso compartido o sección crítica, y el número de permisos pasado al constructor será 1.
- Si se usa para comunicar hilos, en este caso un hilo invocará al método acquire() y otro hilo invocará al método release() para así trabajar de manera coordinada. El número de permisos pasado al constructor coincidirá con el número máximo de hilos bloqueados en la cola o lista de espera para adquirir un permiso.

Semáforos contadores: Controlar el acceso simultáneo a múltiples copias de un recurso

Los semáforos son un mecanismo de sincronización genérico que puede utilizar para proteger cualquier crítica sección de información en cualquier problema.

Cuando un hilo quiere acceder a uno de estos recursos compartidos, primero debe adquirir el semáforo. Si el contador interno del semáforo es mayor que 0, el semáforo disminuye el contador y permite el acceso al recurso compartido. Un contador mayor que 0 significa que hay recursos disponibles que se pueden usar, por lo que el hilo puede acceder y usar uno de ellos. De lo contrario, si el contador del semáforo es 0, el semáforo pone el hilo en suspensión hasta que el contador es mayor que 0. Un valor de 0 en el contador significa que todos los recursos compartidos son utilizado por otros hilos, por lo que el hilo que quiera utilizar uno de ellos debe esperar hasta que uno de los recursos a compartir esté libre.

Cuando el hilo ha terminado el uso del recurso compartido, debe liberar el semáforo para que el otro hilo pueda acceder al recurso compartido. Esa operación aumenta la contador interno del semáforo.



En Java, la clase Semaphore pertenece al paquete java.util.concurrent y se utiliza para controlar el acceso concurrente a recursos compartidos. Dos de los métodos clave de la clase Semaphore son acquire() y release().

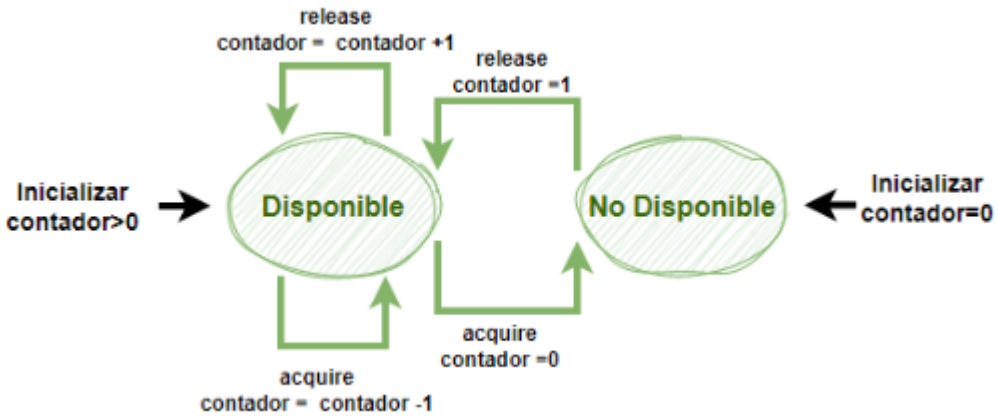
acquire():

- El método acquire() se utiliza para adquirir un permiso del semáforo.
- Si hay permisos disponibles, el hilo que llama a acquire() los adquiere y continúa ejecutándose.
- Este método devuelve true si un permiso está disponible inmediatamente y lo adquiere; de lo contrario, devuelve false.
- Si no hay permisos disponibles, el hilo se bloquea hasta que se libere al menos un permiso.
- Si el hilo es interrumpido mientras espera, puede lanzar una excepción InterruptedException.



Este ejemplo muestra el uso de la clase `Semaphore` para controlar el acceso a una sección crítica, donde se inicializa el semáforo con 5 permisos, permitiendo que hasta cinco hilos accedan simultáneamente.

```java
public class Semaforo {
    public static void main(String[] args) {
        Semaphore semaphore = new Semaphore(5); // Inicialización con 5 permisos
        try {
            semaphore.acquire(); // Adquirir un permiso
            // Operaciones críticas
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            semaphore.release(); // Liberar el permiso
        }
    }
}
```



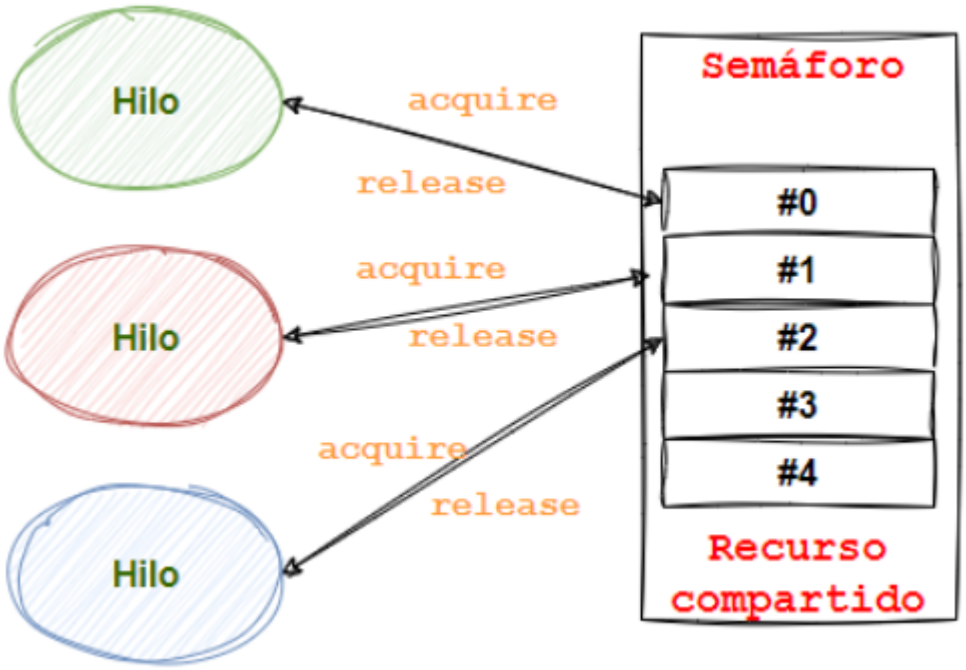
En este ejemplo se ha creado un semáforo contador para controlar el acceso a 5 recursos compartidos, de tal manera que cada vez que se haga una operación acquire se disminuirá el

contador una unidad siempre y cuando sea mayor a 0. Si el contador interno es 0, acquire provocará el bloqueo (la espera) del hilo que quiere acceder a uno de los recursos. Hasta que no esté disponible un recurso de los 5, el hilo quedará en bloqueo. Un recurso estará disponible cuando el hilo deje de hacer uso de él y lo indique con la función release, la cual provocará el aumento del contador en una unidad.

release():

- El método release() se utiliza para liberar un permiso en el semáforo.
- Incrementa el número de permisos disponibles en el semáforo.
- Si hay hilos bloqueados esperando adquirir un permiso, uno de ellos se desbloqueará y adquirirá el permiso liberado.

> **Advertencia**
>
> Es importante destacar que la liberación de permisos no está vinculada al hilo que los adquirió originalmente. Cualquier hilo puede llamar a release(), lo que permite que otros hilos adquieran los permisos liberados.



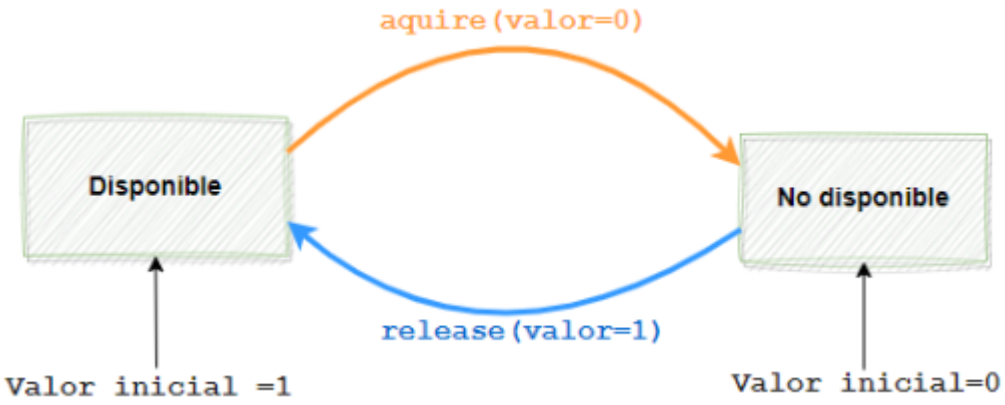
Además de estos métodos la clase Semaphore proporciona métodos de gestión y consulta del estado del semáforo.

Semáforos binarios

Este tipo de semáforos protegen el acceso a un recurso compartido único, por lo que el

contador interno del semáforo sólo puede tomar los valores 1 o 0. Para mostrar cómo usarlo, implementará una cola de impresión que puede ser utilizada por tareas simultáneas para imprimir sus trabajos. Esta cola de impresión estará protegida por un semáforo binario, por lo que sólo se puede imprimir un hilo a la vez.

Los semáforos binarios son bastante similares a semáforos contadores, pero su valor está restringido a 0 y 1. En este tipo de semáforo, la operación de espera funciona sólo si el semáforo = 1, y la operación de señal tiene éxito cuando el semáforo = 0. Es más fácil de implementar que contar semáforos.





Este ejemplo muestra el uso de un semáforo binario en Java. Un semáforo binario permite un máximo de un hilo en la sección crítica, funcionando como un mecanismo de exclusión mutua.

```java
public class SemaforoBinario {
    public static void main(String[] args) {
        Semaphore binarySemaphore = new Semaphore(1); // Semáforo binario
        try {
            binarySemaphore.acquire(); // Adquirir el permiso (cerrar)
            // Operaciones críticas
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            binarySemaphore.release(); // Liberar el permiso (abrir)
        }
    }
}
```

En este ejemplo se ha creado un semáforo binario para controlar el acceso a un sólo recurso compartido, de tal manera que cada vez que se haga una operación acquire se disminuirá el contador una unidad siempre y cuando sea mayor a 0. Si el contador interno es 0, acquire provocará el bloqueo (la espera) del hilo que quiere acceder al recurso. Hasta que no esté disponible el recurso el hilo quedará en bloqueo. El recurso estará disponible cuando el hilo que lo tiene deje de hacer uso de él y lo indique con la función release, la cual provocará que el contador tome el valor 1.

En el siguiente enlace tienes un ejemplo del uso de Semaphore para proteger secciones críticas o recursos compartidos. Es el ejemplo que vimos del acceso simultáneo de 4 terminales a un Servidor, pero resuelto ahora con la clase Semaphore en vez de con synchronized.



En el siguiente enlace tienes un ejemplo del uso de Semaphore para comunicar hilos. Es el ejemplo de los Lectores-Escritores. Se inician 5 hilos lectores y 2 escritores, como en el ejemplo que resolvimos con wait() y notify().



Envío de señales entre Hilos

Si usas un semáforo para enviar señales entre hilos, normalmente un hilo llamará al método acquire() y otro hilo llamará al método release().

Si no hay permisos disponibles, la llamada a acquire() se bloqueará hasta que otro hilo libere un permiso. De manera similar, una llamada a release() se bloquea si no se pueden liberar más permisos en ese semáforo.

De esta manera, es posible coordinar hilos. Por ejemplo, si acquire() se llamara después de que el Hilo 1 hubiera insertado un objeto en una lista compartida, y el Hilo 2 llamara a release() justo antes de tomar un objeto de esa lista, habrías creado esencialmente una cola bloqueante.

El número de permisos disponibles en el semáforo correspondería a la cantidad máxima de elementos que la cola bloqueante podría contener.

Equidad

No se garantiza la equidad en la obtención de permisos en el Semaphore. Es decir, no hay garantía de que el primer hilo que llame a acquire() sea también el primer hilo en obtener un permiso. Si el primer hilo está bloqueado esperando un permiso, otro hilo que verifique la disponibilidad de un permiso justo cuando se libera uno podría obtener el permiso antes que el primer hilo.

Si deseas imponer equidad, la clase Semaphore tiene un constructor que acepta un valor booleano que indica si el semáforo debe forzar la equidad. Imponer equidad tiene un costo en el rendimiento y en la concurrencia, por lo que no se debe habilitar a menos que sea necesario.

Aquí tienes cómo crear un Semaphore en modo equitativo:



Este código utiliza un `Semaphore` en Java con prioridad justa para controlar el acceso a una sección crítica, permitiendo un solo hilo a la vez.

```
Semaphore semaphore = new Semaphore(1, true);

try {
    semaphore.acquire();
    // sección crítica
} finally {
    semaphore.release();
}
```

En este ejemplo, el segundo parámetro true le indica al semáforo que aplique una política de equidad, asegurando que los hilos adquieran permisos en el orden en que los solicitaron.

Ventajas de los semáforos

A continuación se detallan las ventajas de utilizar Semaphore:

- Permite que más de un hilo acceda a la sección crítica.
- Los semáforos son independientes de la máquina.
- No permiten que múltiples procesos ingresen a la sección crítica.

- Como hay mucha espera en el semáforo, nunca se pierden recursos ni tiempo de proceso.
- Permiten una gestión flexible de los recursos.

Desventaja de los semáforos

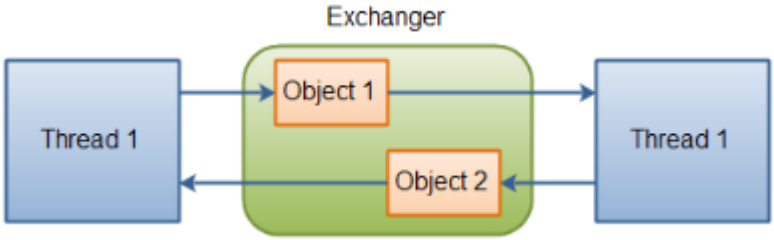
Aquí están las desventajas/desventajas del semáforo.

- Una de las mayores limitaciones de un semáforo es la inversión de prioridad.
- El sistema operativo debe realizar un seguimiento de todas las llamadas para esperar y señalar el semáforo.
- Para evitar interbloqueos en el semáforo, las operaciones de espera y señal deben ejecutarse en el orden correcto.
- La programación de semáforos es complicada, por lo que existen posibilidades de no lograr la exclusión mutua.
- Puede provocar un punto muerto o una violación de la exclusión mutua debido a un error del programador.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

La clase Exchanger

La clase Exchanger, del paquete java.util.concurrent, establece un punto de sincronización donde se intercambian objetos entre dos hilos. La clase Exchanger es genérica, lo que significa que tendrás que especificar en el tipo de objeto a compartir entre los hilos.



Existen dos métodos definidos en esta clase:

- `public V exchange(V x) throws InterruptedException`: El hilo actual espera a que otro hilo llame a su método exchange antes de continuar con su ejecución cuando se invoca esta función. La operación de los hilos continúa después de que intercambian datos y otro hilo invoque su método exchange.
- `public V exchange(V x, long timeout, TimeUnit unit) throws InterruptedException, Timeo` : El hilo actual espera a que otro hilo llame a su método exchange antes de continuar con su ejecución cuando se invoca esta función. La operación de los hilos continúa después de que intercambian datos, y otro hilo invoca su procedimiento de intercambio. Solo se permite que el hilo espere durante el tiempo indicado por el argumento de tiempo de espera (timeout); si se supera ese tiempo, se lanza una TimeoutException.

Ambos métodos exchange() permiten intercambiar objetos entre dos hilos. El hilo que desea obtener la información, esperará realizando una llamada al método exchange() hasta que el otro hilo sitúe la información utilizando el mismo método, o hasta que pase un periodo de tiempo establecido mediante el parámetro timeout .

Cuando se invoca exchange, espera a que el otro hilo del par lo llame también. En ese momento, el segundo hilo encuentra que el primer hilo está esperando con su objeto. Los hilos intercambian los objetos que están sosteniendo y señalan el intercambio, y ahora pueden regresar.

Este proceso se utiliza para sincronizar dos hilos que necesitan intercambiar datos entre sí. La clase Exchanger de Java (java.util.concurrent.Exchanger) facilita este tipo de interacción. Cada hilo llama al método exchange() con el objeto que quiere intercambiar, y ambos hilos esperan

hasta que el otro esté listo para intercambiar, momento en el cual se produce el intercambio y ambos hilos pueden continuar.

El funcionamiento, tal y como puedes apreciar en la imagen anterior, sería el siguiente:

- Dos hilos (hiloA e hiloB) intercambiarán objetos del mismo tipo, objetoA y objetoB.
- El hiloA invocará a exchange(objetoA) y el hiloB invocará a exchange(objetoB).
- El hilo que procese su llamada a exchange(objeto) en primer lugar, se bloqueará y quedará a la espera de que lo haga el segundo. Cuando eso ocurra y se libere el bloqueo sobre ambos hilos, la salida del método exchange(objetoA) proporciona el objeto objetoB al hiloA, y la del método exchange(objetoB) el objeto objetoA al hiloB.

Te estarás preguntando ¿y cuándo puede ser útil Exchanger? Los intercambiadores se emplean típicamente cuando un hilo productor está rellenando una lista o búfer de datos, y otro hilo consumidor los está consumiendo.

De esta forma cuando el consumidor empieza a tratar la lista de datos entregados por el productor, el productor ya está produciendo una nueva lista. Precisamente, esta es la principal utilidad de los intercambiadores: que la producción y el consumo de datos, puedan tener lugar concurrentemente.

En el siguiente enlace encontrarás un ejemplo donde un hilo productor se encarga de rellenar una cadena de diez caracteres (o sea, una lista de 10 caracteres) mientras que un hilo consumidor la imprime.



En el siguiente ejemplo el lector genera datos continuamente y, cuando su búfer está lleno, los intercambia con el procesador. El procesador recibe estos datos, los transfiere a su propio búfer y los procesa. Cuando el búfer está vacío, intercambia nuevos datos con el lector. Cuando el búfer para el escritor está lleno, intercambia su contenido con el escritor.
El escritor recibe los datos y los imprime. Cuando su búfer se vacía, intercambia su contenido con el procesador para recibir nuevos datos. Este enfoque permite la sincronización de datos entre los hilos de una manera eficiente y controlada, utilizando Exchanger para evitar bloqueos y permitir una transferencia ordenada de datos entre los hilos.



En el código del ejemplo:

- Método main: Todo el código se encuentra en el método main, permitiendo que se ejecute directamente.
- Exchangers: lectorExchanger y escritorExchanger son los mismos Exchanger para sincronizar la transferencia de datos entre hilos.
- Hilos: Cada Runnable (lector, procesador, escritor) funciona igual que antes:Lector: Genera datos y los intercambia con el procesador.
- Procesador: Recibe datos del lector, los procesa y los intercambia con el escritor.
- Escritor: Recibe datos y los imprime.
- Ejecución: CompletableFuture.allOf(...).join() ejecuta los tres hilos (lector, procesador, escritor) de manera asíncrona y simultánea.

En el siguiente ejemplo una clase genera una cadena de 8 caracteres y otra clase recibe dicha cadena a través del uso de Exchanger



Explicación del código:

Clase Principal EjemploExchanger:

- Se crea un Exchanger para que los hilos CrearCadena y UsarCadena intercambien una cadena.
- Se inician dos hilos (UseString y CrearCadena) que comparten este Exchanger.

Clase CrearCadena :

- Este hilo genera una cadena de 8 caracteres en cada iteración y la intercambia con UsarCadena a través del Exchanger.
- Si está en la tercera iteración (cuando i == 2), hace un intercambio con un límite de tiempo de 250 ms usando intercambiador.exchange(s, 250, TimeUnit.MILLISECONDS).

- En las demás iteraciones, intercambia el buffer completo con un buffer vacío.

Clase UsarCadena :

- Este hilo recibe la cadena intercambiada por CrearCadena .
- Si está en la tercera iteración, duerme 500 ms antes de continuar, lo que puede provocar un TimeoutException en CrearCadena debido al límite de tiempo de 250 ms.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Las clase CountDownLatch

La clase CountDownLatch del paquete java.util.concurrent es una utilidad de sincronización que permite que uno o más threads esperen hasta que otros threads finalicen su trabajo.

El funcionamiento esquemático de CountDownLatch o "cuenta atrás de cierre" es el siguiente:

- Implementa un punto de espera que denominaremos "puerta de cierre", donde uno o más hilos esperan a que otros finalicen su trabajo.
- Los hilos que deben finalizar su trabajo se controlan mediante un contador que llamaremos "cuenta atrás".
- Cuando la "cuenta atrás" llega a cero se reanudará el trabajo del hilo o hilos interrumpidos y puestos en espera.
- No será posible volver a utilizar la "cuenta atrás", es decir, no se puede reiniciar. Si fuera necesario reiniciar la "cuenta atrás" habrá que pensar en utilizar la clase CyclicBarrier.

Los aspectos más importantes al usar la clase CountDownLatch son los siguientes:

- Al constructor countDownLatch(int cuenta) se le indica, mediante el parámetro "cuenta", el total de hilos que deben completar su trabajo, que será el valor de la "cuenta atrás".
- El hilo en curso desde el que se invoca al método await() esperará en la "puerta de cierre" hasta que la "cuenta atrás" tome el valor cero. También se puede utilizar el método await(long tiempoespera, TimeUnit unit), para indicar que la espera será hasta que la cuenta atrás llegue a cero o bien se sobrepase el tiempo de espera especificado mediante el parámetro tiempoespera.
- La "cuenta atrás" se irá decrementando mediante la invocación del método countDown(), y cuando ésta llega al valor cero se libera el hilo o hilos que estaban en espera, continuando su ejecución.
- No se puede reinicar o volver a utilizar la "cuenta atrás" una vez que ésta toma el valor cero. Si esto fuera necesario, entonces debemos pensar en utilizar la clase CyclicBarrier.
- El método getCount() obtiene el valor actual de la "cuenta atrás" y generalmente se utiliza durante las pruebas y depuración del programa.

En el siguiente enlace puedes ver un ejemplo de cómo utilizar CountDownLatch para sumar todos los elementos de una matriz. Cada fila de la matriz es sumada por un hilo. Cuando todos los hilos han finalizado su trabajo, se ejecuta el procedimiento que realiza la suma global.



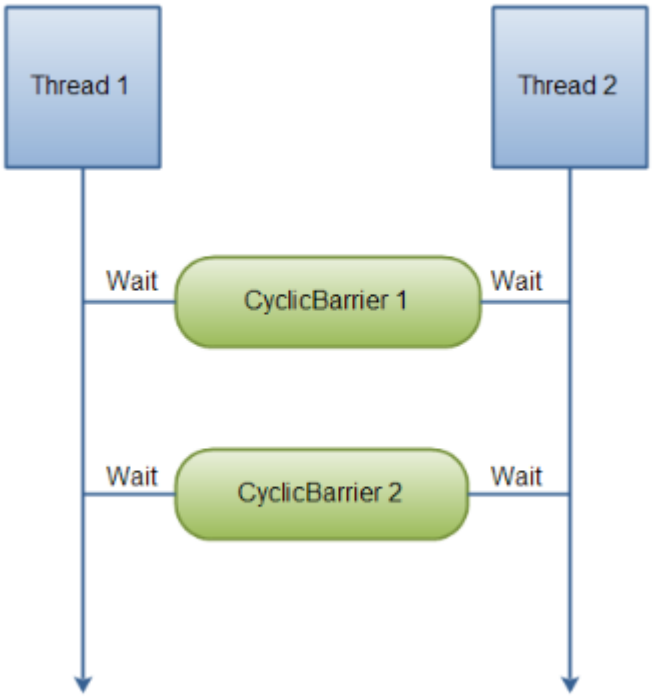
Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

La clase CyclicBarrier

La clase CyclicBarrier del paquete java.util.concurrent es una utilidad de sincronización que permite que uno o más threads se esperen hasta que todos ellos finalicen su trabajo.

El funcionamiento esquemático de CyclicBarrier o "barrera cíclica" es el siguiente:

- Implementa un punto de espera que llamaremos "barrera", donde cierto número de hilos esperan a que todos ellos finalicen su trabajo. Esta barrera representa el punto de sincronización.
- Cada hilo que alcanza la barrera queda en espera, bloqueado, hasta que el último hilo en el grupo la alcanza.
- Finalizado el trabajo de los hilos, se dispara la ejecución de una determinada acción o bien el hilo interrumpido continúa su trabajo.
- La barrera se llama cíclica, porque se puede volver a utilizar después de que los hilos en espera han sido liberados tras finalizar todos su trabajo, y también se puede reiniciar.



Los aspectos más importantes al usar la clase CyclicBarrier son los siguientes:

- Indicar al constructor CyclicBarrier(int hilosAcceden) el total de hilos que van a usar la barrera mediante el parámetro hilosAcceden. Este número sirve para disparar la barrera.
- La barrera se dispara cuando llega el último hilo.
- Cuando se dispara la barrera, dependiendo del constructor, CyclicBarrier(int hilosAcceden) o CyclicBarrier(int hilosAcceden, Runnable acciónBarrera) se lanzará o no una acción, y entonces se

liberan los hilos de la barrera. Esa acción puede ser realizada mediante cualquier objeto que implemente Runnable.

- El método principal de esta clase es await() que se utiliza para indicar que el hilo en curso ha concluido su trabajo y queda a la espera de que lo hagan los demás.

Otros métodos de esta clase que puedes utilizar son:

- El método await(long tiempoespera, TimeUnit unit) funciona como el anterior, pero en este caso el hilo espera en la barrera hasta que los demás finalicen su trabajo o se supere el tiempoespera.
- El método reset() permite reiniciar la barrera a su estado inicial.
- El método getNumber Waiting() devuelve el número de hilos que están en espera en la barrera.
- El método getParties() devuelve el número de hilos requeridos para esa barrera.

En el siguiente enlace puedes ver un ejemplo parecido al anterior, pero ahora resuelto con CyclicBarrier. Cada fila de la matriz ahora representa los valores recaudados por un cobrador. Cada fila es sumada por un hilo. Cuando 5 de estos hilos finalizan su trabajo, se dispara un objeto que implementa Runnable para obtener la suma recaudada hasta el momento. Como la matriz del ejemplo tiene 10 filas, la suma de sus elementos se hará mediante una barrera de 5 hilos y que se utilizará por tanto de forma cíclica dos veces.



El CyclicBarrier es útil en algoritmos donde es necesario que los hilos sincronicen su progreso en ciertos puntos antes de seguir adelante, asegurando que todos están listos para avanzar al mismo tiempo en la siguiente fase.

En el siguiente código se utiliza CyclicBarrier para sincronizar dos hilos en dos puntos de espera (barreras). Después de que ambos hilos lleguen a una barrera, se ejecuta una acción específica antes de que puedan continuar.



Explicación del Código

1. Runnable accionBarrera1 y accionBarrera2: Estas acciones se ejecutan cuando ambos hilos han alcanzado barrera1 y barrera2, respectivamente. Las acciones imprimen un mensaje en la consola.
2. Barreras barrera1 y barrera2: Cada CyclicBarrier está configurado para esperar a que dos hilos lleguen antes de continuar. Cuando ambos hilos alcanzan una barrera, se ejecuta la acción asignada.
3. Clase TareaConBarreras: Implementa Runnable y define el comportamiento de cada hilo:
4. Cada hilo espera en la barrera1, luego en la barrera2, y finalmente imprime un mensaje indicando que ha terminado.
   Ejecución en el main: Se crean dos instancias de TareaConBarreras (tarea1 y tarea2) y se ejecutan en hilos separados. Ambas tareas se sincronizan en las barreras y avanzan solo cuando ambas han alcanzado los puntos de espera.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Aplicaciones multihilo

Una aplicación multihilo debe reunir las siguientes propiedades:

1. Seguridad

Definición: La seguridad en una aplicación multihilo significa que los recursos compartidos no llegarán a un estado inconsistente debido al acceso concurrente. Esto requiere sincronizar los hilos y asegurar la exclusión mutua cuando se accede a datos o recursos compartidos.

Ejemplo: Supongamos que varios hilos están accediendo y modificando una variable de saldo en una cuenta bancaria. Para evitar que se produzcan inconsistencias, podemos utilizar el bloque synchronized para garantizar que solo un hilo a la vez acceda al saldo.



Este código muestra una clase `CuentaBancaria` con métodos sincronizados para realizar depósitos y retiros, y una clase `Transaccion` que ejecuta una transacción en un hilo.

```
public class CuentaBancaria {
    private int saldo = 100;

    public synchronized void depositar(int monto) {
        saldo += monto;
    }

    public synchronized void retirar(int monto) {
        if (saldo >= monto) {
            saldo -= monto;
        } else {
            System.out.println("Fondos insuficientes");
        }
    }

    public synchronized int obtenerSaldo() {
        return saldo;
    }
}

class Transaccion implements Runnable {
    private CuentaBancaria cuenta;
```

```
    public Transaccion(CuentaBancaria cuenta) {
        this.cuenta = cuenta;
    }

    @Override
    public void run() {
        cuenta.depositar(50);
        cuenta.retirar(30);
        System.out.println("Saldo actual: " + cuenta.obtenerSaldo());
    }
}
```

En este ejemplo, los métodos depositar, retirar y obtenerSaldo están sincronizados, lo que garantiza que solo un hilo a la vez pueda ejecutar estos métodos, manteniendo la seguridad y evitando inconsistencias en el saldo.

2. Viveza

Definición: La viveza asegura que la aplicación no se quedará bloqueada indefinidamente y que todos los hilos tendrán una oportunidad de ejecutarse. Esto implica evitar interbloqueos (deadlocks) e inanición (starvation).

Ejemplo de interbloqueo: Aquí un ejemplo de cómo un mal uso de la sincronización puede causar un interbloqueo.



Este código demuestra un ejemplo de interbloqueo entre dos hilos que intentan adquirir dos bloqueos en orden inverso, lo que puede causar un bloqueo mutuo.

```
public class InterbloqueoDemo {
    private static final Object lock1 = new Object();
    private static final Object lock2 = new Object();

    public static void main(String[] args) {
        Thread hilo1 = new Thread(() -> {
            synchronized (lock1) {
                System.out.println("Hilo 1 bloqueó lock1");
                try { Thread.sleep(50); } catch (InterruptedException e) {}
                synchronized (lock2) {
                    System.out.println("Hilo 1 bloqueó lock2");
```

```
                }
            }
        });

        Thread hilo2 = new Thread(() -> {
            synchronized (lock2) {
                System.out.println("Hilo 2 bloqueó lock2");
                try { Thread.sleep(50); } catch (InterruptedException e) {}
                synchronized (lock1) {
                    System.out.println("Hilo 2 bloqueó lock1");
                }
            }
        });

        hilo1.start();
        hilo2.start();
    }
}
```

En este ejemplo, el hilo1 bloquea lock1 y luego intenta obtener lock2, mientras que hilo2 bloquea lock2 y luego intenta obtener lock1. Ambos hilos quedan esperando indefinidamente a que el otro libere el recurso, provocando un interbloqueo.

3. Corrección de la aplicación

- Corrección parcial: La aplicación cumple la propiedad de seguridad, es decir, los datos son consistentes, pero puede tener problemas de viveza.
- Corrección total: La aplicación cumple tanto con la propiedad de seguridad como con la propiedad de viveza. Esto implica que los recursos están protegidos adecuadamente y todos los hilos tienen oportunidad de ejecutarse.

Ejemplo de corrección total:

Para lograr corrección total, podemos usar las clases de concurrencia de Java, como ReentrantLock, que permite verificar si un recurso está bloqueado y evitar posibles interbloqueos.



Este código muestra cómo evitar un interbloqueo utilizando `ReentrantLock` con el método `tryLock()` para intentar adquirir los bloqueos sin quedar bloqueado indefinidamente.

```java
import java.util.concurrent.locks.ReentrantLock;

public class CorreccionTotalDemo {
    private static ReentrantLock lock1 = new ReentrantLock();
    private static ReentrantLock lock2 = new ReentrantLock();

    public static void main(String[] args) {
        Thread hilo1 = new Thread(() -> {
            try {
                if (lock1.tryLock()) {
                    System.out.println("Hilo 1 bloqueó lock1");
                    Thread.sleep(50);
                    if (lock2.tryLock()) {
                        System.out.println("Hilo 1 bloqueó lock2");
                        lock2.unlock();
                    }
                    lock1.unlock();
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        Thread hilo2 = new Thread(() -> {
            try {
                if (lock2.tryLock()) {
                    System.out.println("Hilo 2 bloqueó lock2");
                    Thread.sleep(50);
                    if (lock1.tryLock()) {
                        System.out.println("Hilo 2 bloqueó lock1");
                        lock1.unlock();
                    }
                    lock2.unlock();
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        hilo1.start();
        hilo2.start();
    }
```

```
}
```

En este ejemplo, los hilos utilizan tryLock() para intentar bloquear lock1 y lock2. Si no pueden obtener ambos bloqueos, liberan el recurso que ya tenían y vuelven a intentarlo, evitando el interbloqueo.

4. Situación de los hilos en la aplicación

- **Hilos independientes**: Si los hilos no comparten recursos, no necesitan sincronización. Pueden ejecutarse de manera independiente.
- **Hilos colaborando o compitiendo**: Si los hilos comparten recursos, es necesario sincronizarlos para evitar conflictos y mantener la seguridad de los datos. Por ejemplo, los hilos en un sistema de productor-consumidor deben coordinarse para evitar que el consumidor lea datos antes de que el productor los haya escrito.

Ejemplo de productor-consumidor:



Este ejemplo demuestra un modelo productor-consumidor en Java utilizando un `Queue` como buffer limitado y métodos de sincronización para gestionar el acceso concurrente.

```
import java.util.LinkedList;
import java.util.Queue;

public class ProductorConsumidorDemo {
    private static Queue buffer = new LinkedList<>();
    private static final int CAPACIDAD = 5;

    public static void main(String[] args) {
        Thread productor = new Thread(() -> {
            int valor = 0;
            while (true) {
                synchronized (buffer) {
                    while (buffer.size() == CAPACIDAD) {
                        try { buffer.wait(); } catch (InterruptedException e) {}
                    }
                    buffer.add(valor);
                    System.out.println("Produjo: " + valor);
                    valor++;
                    buffer.notifyAll();
```

```
                }
            }
        });

        Thread consumidor = new Thread(() -> {
            while (true) {
                synchronized (buffer) {
                    while (buffer.isEmpty()) {
                        try { buffer.wait(); } catch (InterruptedException e) {}
                    }
                    int valor = buffer.poll();
                    System.out.println("Consumió: " + valor);
                    buffer.notifyAll();
                }
            }
        });

        productor.start();
        consumidor.start();
    }
}
```

El productor añade elementos al buffer, y el consumidor los retira. La sincronización evita que el consumidor intente consumir cuando el buffer está vacío y que el productor intente producir cuando el buffer está lleno.

5. Gestionar las prioridades

Java permite asignar prioridades a los hilos, pero no garantiza que el sistema operativo respete siempre estas prioridades, ya que algunos sistemas no implementan time-slicing y pueden provocar inanición en hilos de baja prioridad.



Este ejemplo demuestra cómo crear dos hilos en Java con diferentes prioridades usando `Thread.MAX_PRIORITY` y `Thread.MIN_PRIORITY`.

```
Thread hiloAltaPrioridad = new Thread(() -> {
    System.out.println("Hilo de alta prioridad ejecutándose");
});
hiloAltaPrioridad.setPriority(Thread.MAX_PRIORITY);
```

```
Thread hiloBajaPrioridad = new Thread(() -> {
    System.out.println("Hilo de baja prioridad ejecutándose");
});
hiloBajaPrioridad.setPriority(Thread.MIN_PRIORITY);

hiloAltaPrioridad.start();
hiloBajaPrioridad.start();
```

En este ejemplo, el hiloAltaPrioridad tiene una prioridad más alta y es probable que se ejecute antes que el hiloBajaPrioridad. Sin embargo, el comportamiento exacto depende del sistema operativo.

6. Uso de librerías estándar para concurrencia
Java proporciona una biblioteca estándar para concurrencia (java.util.concurrent), que incluye herramientas para facilitar el trabajo con hilos y mejorar la fiabilidad de las aplicaciones multihilo.



Este ejemplo muestra el uso de `ExecutorService` con un pool de dos hilos para ejecutar tareas concurrentes en Java.

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class EjemploExecutorService {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(2);

        executor.submit(() -> {
            System.out.println("Tarea 1 ejecutada por " +
Thread.currentThread().getName());
        });

        executor.submit(() -> {
            System.out.println("Tarea 2 ejecutada por " +
Thread.currentThread().getName());
        });

        executor.shutdown();
    }
```

```
}
```

El uso de ExecutorService permite manejar un grupo de hilos de forma eficiente y evita la creación y destrucción constante de hilos. Además, reduce el riesgo de errores en la gestión manual de los hilos.

Siguiendo estos principios y aprovechando las herramientas de concurrencia de Java, puedes desarrollar aplicaciones multihilo seguras, vivas y correctas, lo cual mejora el rendimiento y la confiabilidad de tus aplicaciones.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

La interfaz Executor y los pools de hilos

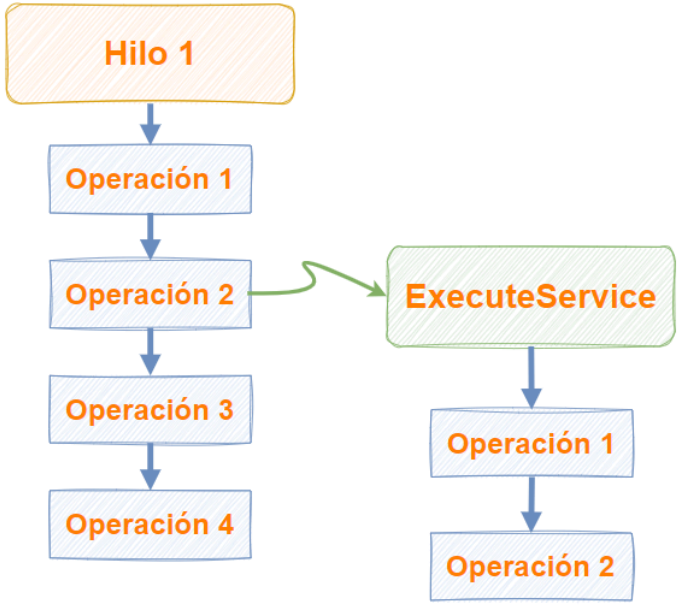
Cuando trabajamos con aplicaciones tipo servidor, éstas tienen que atender un número masivo y concurrente de peticiones de usuario, en forma de tareas que deben ser procesadas lo antes posible. Al principio de la unidad ya te indicamos mediante un ejemplo la conveniencia de utilizar hilos en estas aplicaciones, pero si ejecutamos cada tarea en un hilo distinto, se pueden llegar a crear tantos hilos que el incremento de recursos utilizados puede comprometer la estabilidad del sistema. Los pools de hilos ofrecen una solución a este problema.

Y ¿qué es un pool de hilos? Un pool de hilos (thread pools) es básicamente un contenedor dentro del cual se crean y se inician un número limitado de hilos, para ejecutar todas las tareas de una lista.

Para declarar un pool, lo más habitual es hacerlo como un objeto del tipo **ExecutorService** utilizando alguno de los siguientes métodos de la clase estática **Executors**:

- **newFixedThreadPool**(int numeroHilos): crea un pool con el número de hilos indicado. Dichos hilos son reutilizados cíclicamente hasta terminar con las tareas de la cola o lista.
- **newCachedThreadPool**(): crea un pool que va creando hilos conforme se van necesitando, pero que puede reutilizar los ya concluidos para no tener que crear demasiados. Los hilos que llevan mucho tiempo inactivos son terminados automáticamente por el pool.
- **newSingleThreadExecutor**(): crea un pool de un solo hilo. La ventaja que ofrece este esquema es que si ocurre una excepción durante la ejecución de una tarea, no se detiene la ejecución de las siguientes.
- **newScheduledExecutor**() : crea un pool que va a ejecutar tareas programadas cada cierto tiempo, ya sea una sola vez o de manera repetitiva. Es parecido a un objeto Timer, pero con la diferencia de que puede tener varios threads que irán realizando las tareas programadas conforme se desocupen.

La interfaz ExecutorService de Java, java.util.concurrent.ExecutorService, representa un mecanismo de ejecución asíncrona que es capaz de ejecutar tareas de manera concurrente en segundo plano. A continuación veremos cómo crear un ExecutorService, cómo enviar tareas para que se ejecuten en él, cómo ver los resultados de esas tareas y cómo cerrar el ExecutorService cuando sea necesario.



Un hilo delega una tarea a un ExecutorService para su ejecución asíncrona.
Una vez que el hilo ha delegado la tarea al ExecutorService, el hilo continúa su propia ejecución, independientemente de la ejecución de esa tarea. El ExecutorService luego ejecuta la tarea de manera concurrente, de forma independiente del hilo que envió la tarea.

Los objetos de tipo ExecutorService implementan la interfaz Executor. Esta interfaz define el método execute(Runnable), al que hay que llamar una vez por cada tarea que deba ser ejecutada por el pool (la tarea se pasa como argumento del método).

La interface ExecutorService proporciona una serie de métodos para el control de la ejecución de las tareas, entre ellos el método shutdown(), para indicarle al pool que los hilos no se van a reutilizar para nuevas tareas y deben morir cuando finalicen su trabajo.

En el siguiente enlace dispones de un ejemplo en el que se define una clase que implementa Runnable cuya tarea es generar e imprimir 10 números aleatorios. Se creará un pool de 2 hilos capaz de realizar 30 de esas tareas.



Algunas de sus interfaces derivadas, así como clases directamente relacionadas se muestran en la siguiente tabla:

Nombre	Tipo	Descripción
ExecuteService	Interface	Subinterfaz de Executor, permite gestionar tareas asíncronas
ScheduleExecutorService	Interface	Permite la planificación de la ejecución de tareas asíncronas
Executors	Clase	Fábrica de objetos Executor, ExecutorServices, ScheduledExecutorService, ThreadFactory y Callable
TimeUnit	Enumeración	Proporciona representaciones de unidades de tiempo con distinta granularidad, desde días hasta nanosegundos

El siguiente ejemplo ilustra el uso de ScheduledExecutorService como herramienta para la programación de ejecuciones de tareas. Como se puede observar, mediante la clase Executor se obtiene una instancia de ScheduledExecutorService, que es la interface que permite programar tareas recurrentes en hilos independientes. Una vez obtenido el programador de tareas, se le indica qué tarea se quire ejecutar (objeto sr), cuántas unidades de tiempo se desea esperar hasta que se inicie la primera tarea (1), cuántas unidades de tiempo se desea esperar entre cada repetición de la tarea (2) y en qué unidad están representadas las unidades de tiempo (TimeUnit.SECONDS).





Este ejemplo muestra el uso de un pool de hilos de tamaño fijo para ejecutar una tarea asíncrona. Una vez ejecutada la tarea, el pool se apaga con `shutdown`.

```
ExecutorService executorService = Executors.newFixedThreadPool(10);

executorService.execute(new Runnable() {
    public void run() {
        System.out.println("Tarea asíncrona");
    }
});
```

```
executorService.shutdown();
```

Primero, se crea un ExecutorService usando el método newFixedThreadPool() de la clase Executors. Esto crea un pool de hilos con 10 hilos para ejecutar tareas.

Luego, se pasa una implementación anónima de la interfaz Runnable al método execute(). Esto hace que el Runnable sea ejecutado por uno de los hilos en el ExecutorService.

En los siguientes ejemplos vamos a comprobar cómo podemos usar un Executor en lugar de crear explícitamente objetos Thread. Vamos a usar el objeto Despeque para ejecutar una tarea específica:



Esta clase `Despegue` implementa un contador regresivo que simula una cuenta atrás antes del despegue. Implementa la interfaz `Runnable` para que pueda ejecutarse en un hilo.

```
public class Despegue implements Runnable {
    protected int cuentaAtras = 10; // Valor por defecto
    private static int contadorTareas = 0;
    private final int id = contadorTareas++;

    public Despegue() {}

    public Despegue(int cuentaRegresiva) {
        this.cuentaAtras = cuentaRegresiva;
    }

    public String estado() {
        return "#" + id + "(" + (cuentaAtras > 0 ? cuentaAtras : "¡Despegue!") +
"), ";
    }

    public void run() {
        while (cuentaAtras-- > 0) {
            System.out.print(estado());
            Thread.yield();
        }
    }
```

```
}
```

Muy a menudo, se puede usar un único `Executor` para crear y gestionar todas las tareas en tu sistema. La llamada a `shutdown()` evita que se envíen nuevas tareas a ese `Executor`. El hilo actual (en este caso, el que ejecuta `main()`) continuará ejecutando todas las tareas enviadas antes de que se llame a `shutdown()`. El programa se cerrará tan pronto como todas las tareas en el `Executor` finalicen.

Un `FixedThreadPool` utiliza un conjunto limitado de hilos para ejecutar las tareas enviadas.



Este ejemplo muestra el uso de `ExecutorService` con un `FixedThreadPool`, que crea un pool de tamaño fijo para ejecutar tareas concurrentemente en un número limitado de hilos.

```
public class FixedThreadPool {
    public static void main(String[] args) {
        // Constructor tiene como argumento el número de hilos:
        ExecutorService exec = Executors.newFixedThreadPool(5);
        for (int i = 0; i < 5; i++)
            exec.execute(new Despegue());
        exec.shutdown();
    }
}
```

Con el `FixedThreadPool`, realizas la costosa asignación de hilos una sola vez, al inicio, y así limitas el número de hilos. Esto ahorra tiempo, ya que no incurres constantemente en el costo de crear un hilo para cada tarea individual. Además, en un sistema impulsado por eventos, los manejadores de eventos que requieren hilos pueden ser atendidos tan rápidamente como se desee, simplemente obteniendo hilos del pool. No sobrepasas los recursos disponibles, ya que el `FixedThreadPool` utiliza un número limitado de objetos `Thread`.
Es importante notar que, en cualquiera de los pools de hilos, los hilos existentes se reutilizan automáticamente cuando es posible.

Un `SingleThreadExecutor` es como un `FixedThreadPool` con un tamaño de un solo hilo. Esto es útil para cualquier tarea que desees ejecutar continuamente en otro hilo (una tarea de larga duración), como una tarea que escucha conexiones de socket entrantes. También es práctico

para tareas cortas que quieres ejecutar en un hilo, como pequeñas tareas que actualizan un registro local o remoto, o para un hilo de despacho de eventos.

Si se envían más de una tarea a un `SingleThreadExecutor`, las tareas se pondrán en cola y cada tarea se ejecutará hasta completarse antes de comenzar la siguiente, todo usando el mismo hilo. En el siguiente ejemplo, verás que cada tarea se completa en el orden en que fue enviada antes de comenzar la siguiente. De este modo, un `SingleThreadExecutor` serializa las tareas que se le envían y mantiene su propia cola (oculta) de tareas pendientes.



Este ejemplo muestra el uso de `ExecutorService` con un `SingleThreadExecutor`, que ejecuta tareas de manera secuencial en un único hilo.

```
public class SingleThreadExecutor {
    public static void main(String[] args) {
        ExecutorService exec = Executors.newSingleThreadExecutor();
        for (int i = 0; i < 5; i++)
            exec.execute(new Despegue());
        exec.shutdown();
    }
}
```

Como otro ejemplo, supongamos que tienes varios hilos ejecutando tareas que utilizan el sistema de archivos. Puedes ejecutar estas tareas con un `SingleThreadExecutor` para asegurar que solo una tarea se ejecute a la vez desde cualquier hilo. De esta manera, no necesitas preocuparte por sincronizar el recurso compartido (y evitarás dañar el sistema de archivos mientras tanto). A veces, una mejor solución es sincronizar el recurso (algo que aprenderás más adelante en este capítulo), pero un `SingleThreadExecutor` te permite evitar la complicación de coordinar adecuadamente solo para hacer un prototipo. Al serializar las tareas, puedes eliminar la necesidad de serializar los objetos.

El **newScheduledThreadPool** es una variante del ThreadPoolExecutor que permite programar tareas para que se ejecuten en un momento específico en el futuro o para que se repitan a intervalos fijos. Al igual que el FixedThreadPool, el newScheduledThreadPool limita el número de hilos, realizando la asignación de hilos una sola vez al inicio, pero está diseñado específicamente para manejar tareas que deben ser ejecutadas en un momento programado o de manera periódica.

¿Cómo funciona newScheduledThreadPool?
Con newScheduledThreadPool, puedes programar tareas que:

- Se ejecuten una sola vez después de un retraso especificado.
- Se repitan a intervalos fijos de tiempo.
- Se repitan con un retraso fijo entre el final de una ejecución y el inicio de la siguiente.

Al crear un ScheduledThreadPoolExecutor, proporcionas un número fijo de hilos que podrán ser utilizados para ejecutar estas tareas de manera eficiente. Esto significa que, al igual que el FixedThreadPool, se evita el costo constante de crear y destruir hilos para cada tarea individual. En lugar de eso, las tareas programadas se ejecutan usando los hilos ya asignados en el pool.



Este ejemplo muestra el uso de `ScheduledExecutorService` con un `ScheduledThreadPool`, que permite programar tareas para ejecutarse después de un retraso específico.

```
public class ScheduledThreadPoolExample {
    public static void main(String[] args) {
        // Crea un ScheduledExecutorService con 5 hilos:
        ScheduledExecutorService exec = Executors.newScheduledThreadPool(5);

        // Programa cada tarea para que se ejecute después de un retraso inicial.
        for (int i = 0; i < 5; i++) {
            exec.schedule(new Despegue(), i * 2, TimeUnit.SECONDS);
        }

        // Finaliza el servicio después de un tiempo para permitir que las tareas
se completen.
        exec.shutdown();
    }
}
```

Creación de un ExecutorService

La forma en que creas un ExecutorService depende de la implementación que uses. Sin embargo, también puedes utilizar la clase de fábrica Executors para crear instancias de ExecutorService. Aquí tienes algunos ejemplos de cómo crear un ExecutorService:

Este ejemplo muestra la creación de diferentes tipos de pools de hilos con `ExecutorService` en Java. Cada tipo de pool gestiona hilos de forma distinta según su propósito.

```
ExecutorService executorService1 = Executors.newSingleThreadExecutor();
ExecutorService executorService2 = Executors.newFixedThreadPool(10);
ExecutorService executorService3 = Executors.newScheduledThreadPool(10);
```

En el siguiente enlace dispones de un ejemplo en el que se define una clase que implementa Runnable cuya tarea es generar e imprimir 10 números aleatorios. Se creará un pool de 2 hilos capaz de realizar 30 de esas tareas.



Las interfaces ExecutorService, Callable y Future

Usadas de manera conjunta, estas tres interfaces proporcionan mecanismos para ejecutar el código de manera asíncrona.

ExecutorService proporciona el marco de ejecución de código asíncrono contenido en objetos de las interfaces Callable y Runnable. Sus principales métodos se muestran en la Tabla:

Método	Descripción
awaitTermination	Bloquea el servicio cuando se recibe una petición de apagado hasta que todas las tareas han terminado o se ha alcanzado el tiempo límite de espera (timeout) o ha habido una interrupción.
invokeAll	Permite lanzar una colección de tareas y recoger una lista de objetos Future.
invokeAny	Permite lanzar una colección de tareas y recoger el resultado de la que termine satisfactoriamente (si alguna lo hace).
shutdown()	Hace que el servicio deje de aceptar nuevas tareas, espera a que terminen las que están en ejecución y finaliza.

Método	Descripción
shutdownNow()	Finaliza el servicio y suspende todas las tareas que tiene en ejecución la hagan.
submit()	Envía a ejecución una tarea Runnable o Callable.

Construcción de instancias de ExecutorService

La construcción de instancias de ExecutorService se realiza a través de una serie de métodos estáticos de la clase Executors. Estos métodos permiten indicar la estrategia de hilos que desea que siga el ExecutorService. Algunos de los métodos estáticos de Executors para crear objetos de la interfaz ExecutorService son:

Executors.newCachedThreadPool(): Crea un ExecutorService con un pool de hilos con todos los que sean necesarios, reutilizando aquellos que están libres.
Executors.newFixedThreadPool(int nThreads): Crea un ExecutorService con un pool con un número determinado de hilos.
Executors.newSingleThreadExecutor(): Crea un ExecutorService con un único hilo.
Por su parte, la interfaz Callable es una interfaz que funciona similar a Runnable, pero con la diferencia de que puede devolver un retorno y lanzar una excepción. Es una interfaz funcional, ya que solo tiene el método call, por lo que se puede utilizar en expresiones lambda. El código asíncrono de un objeto Callable se ejecuta a través del método submit de ExecutorService.

La interfaz Future representa un resultado futuro generado por un proceso asíncrono. De alguna manera, es un sistema que permite dejar en suspenso la obtención de un resultado hasta que este está disponible. El resultado de invocar al método submit de un ExecutorService es un objeto Future.

Los métodos de la interfaz Future se recogen en la siguiente tabla:

Método	Descripción
cancel()	Intenta cancelar la ejecución de la tarea.
get()	Espera a que termine la tarea y obtiene el resultado. En una de sus formas admite un timeout para limitar el tiempo de espera.
isCancelled()	Indica si la tarea se ha cancelado antes de terminar.
isDone()	Indica si la tarea ha terminado.

En resumen, la relación entre estas tres interfaces es la siguiente: ExecutorService ejecuta un objeto Callable (o Runnable) con una estrategia multihilo determinada y deposita en un objeto Future el retorno futuro de la tarea.

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class Lector implements Callable {
@Override
public String call() throws Exception {
String textoLeido = "Me gustan las peliculas de acción";
Thread.sleep(1000);
return textoLeido;
}

public static void main(String[] args) {
try {
Lector lector = new Lector();
ExecutorService servicioEjecucion = Executors.newSingleThreadExecutor();
Future resultado = servicioEjecucion.submit(lector);
String texto = resultado.get();
if (resultado.isDone()) {
System.out.println(texto);
System.out.println("Proceso finalizado");
} else if (resultado.isCancelled()) {
System.out.println("Proceso cancelado");
}
servicioEjecucion.shutdown();
} catch (Exception e) {
e.printStackTrace();
}
}
}
```

La salida producida es la siguiente:

```
Me gustan las peliculas de acción

Proceso finalizado
```

Como se ha indicado anteriormente, la principal diferencia entre implementar Callable y Runnable es que la primera opción puede proporcionar un retorno. No obstante, con Runnable existen técnicas para transmitir valores mediante el uso de referencias a objetos.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Timers

Un Timer es una encapsulación de un hilo, y un programa puede iniciar un temporizador que llama a un método a partir de un tiempo determinado y, posiblemente, dejar que el temporizador vuelva a llamar al método después de cierto intervalo. Los temporizadores están representados por dos clases. La primera es la clase TimerTask, que define el método que ejecutará el temporizador, y la otra clase es Timer, que es el temporizador y tiene métodos para iniciar y detener el temporizador. El siguiente programa inicia un temporizador, y luego el programa imprime un mensaje en la pantalla cada medio segundo. El temporizador estará en marcha durante 2 segundos y luego detiene el programa.



Aquí te explico los elementos clave de este código:

- Clase TimerTask: Esta clase permite ejecutar tareas programadas después de un tiempo o a intervalos regulares. En este caso, la tarea simplemente imprime un mensaje y termina la ejecución del programa con System.exit(0);.
- Timer: La instancia de Timer programará la tarea (task) para que se ejecute una sola vez después de 2,000 ms (o 2 segundos).
- Bucle infinito: Después de configurar el temporizador, el programa entra en un bucle infinito (for(;;)) que imprime "Hello world" cada 500 ms. Esto se logra llamando al método delay, que pone el hilo en espera por el tiempo especificado (500 ms en este caso).
- Método pausa: Este método recibe el tiempo en milisegundos y usa Thread.sleep para crear la pausa. También maneja cualquier excepción que pueda ocurrir debido a la interrupción del hilo.

A continuación se muestra otra versión del programa donde la diferencia es que el temporizador inicia por primera cada medio segundo y luego cada segundo:



Observa cómo iniciar el temporizador y cómo especificar el tiempo de inicio y el intervalo del temporizador para definir con qué frecuencia debe activarse. Un temporizador inicia un hilo que, por defecto, no es un hilo daemon. Por lo tanto, el programa termina solo cuando el temporizador se detiene, lo que ocurre en la última instrucción de main().

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Temporizadores

En el siguiente ejemplo se muestra un uso conjunto de las clases Timer y TimerTask. El programa simula controlar un sistema de regadío automático. Este sistema riega por primera vez transcurridos 1000 milisegundos desde el inicio de la ejecución y repite el riego cada 2000 milisegundos.

Código Java



```java
import java.util.Timer;
import java.util.TimerTask;

public class SistemaRiego extends TimerTask {

    @Override
    public void run() {
        System.out.println("Regando...");
    }

    public static void main(String[] args) {
        Timer temporizador = new Timer();
        temporizador.schedule(new SistemaRiego(), 1000, 2000);
    }
}
```

La salida generada transcurridos unos segundos es la que se muestra a continuación. Entre la escritura de cada línea transcurren 2000 milisegundos.

Regando...
Regando...
Regando...
Regando...
Regando...
Regando...
Regando...
Regando...

Actividad propuesta: Planificación de ejercicios de estiramiento

Crea una aplicación que, utilizando las clases Timer y TimerTask recuerde al usuario que hay que levantarse para estirar las piernas cada 30 minutos.
Aunque está fuera del ámbito de este módulo, puedes hacer que se muestre el aviso mediante una notificación del sistema operativo utilizando las clases SystemTray y TrayIcon del paquete java.awt. Puedes encontrar múltiples ejemplos en la web.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Gestión de excepciones

¿Cómo podemos gestionar la excepciones de una aplicación multihilo?

Para gestionar las excepciones de una aplicación multihilo puedes utilizar el método uncaughtExceptionHandler() de la clase thread, que permite definir un manejador de excepciones.

Para crear un manejador de excepciones haremos lo siguiente:

- Crear una clase que implemente la interfaz thread.UncaughtExceptionHandler.
- Implementar el método uncaughtException().

Por ejemplo, podemos crear un manejador de excepciones que utilizarán todos los hilos de una misma aplicación de la siguiente forma:

- El manejador sólo mostrará qué hilo ha producido la excepción y la pila de llamadas de la excepción.



Este ejemplo muestra cómo implementar un manejador de excepciones global para capturar excepciones no controladas en hilos, usando la clase `ManejadorExcepciones` que implementa `Thread.UncaughtExceptionHandler`.

```
public class ManejadorExcepciones implements Thread.UncaughtExceptionHandler {
    // Manejador de excepciones para toda la aplicación

    @Override
    public void uncaughtException(Thread t, Throwable e) {
        System.out.printf("Thread que lanzó la excepción: %s \n", t.getName());
        // Muestra en consola el hilo que produce la excepción
        e.printStackTrace();
        // Muestra en consola la pila de llamadas
    }
}
```

En el siguiente enlace te facilitamos el proyecto completo. Se creará el anterior manejador de excepciones y se implementará un hilo que divide el número 100 por un número aleatorio

comprendido entre 0 y 4, dando así la posibilidad de dividir por 0. Se crean e inician 5 hilos que harán uso del manejador.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Depuración y documentación

Dos tareas muy importantes cuando desarrollamos software de calidad son la depuración y documentación de nuestras aplicaciones.

- Mediante la depuración trataremos de corregir fallos y errores de funcionamiento del programa.
- Mediante la documentación interna aportaremos legibilidad a nuestros programas.

La depuración de aplicaciones multihilo es una tarea difícil debido a que:

- La ejecución de los hilos tiene un comportamiento no determinístico.
- Hay que controlar varios flujos de ejecución.
- Aparecen nuevos errores potenciales debidos a la compartición de recursos entre varios hilos:
- Errores porque no se cumple la exclusión mutua.
- Errores porque se produce interbloqueo.

Podemos realizar seguimientos de la pila de Java tanto estáticos como dinámicos, utilizando los siguientes métodos de la clase thread:

- dumpStack(). Muestra una traza de la pila del hilo (thread) en curso.
- getAllStackTraces(). Devuelve un Map de todos los hilos vivos en la aplicación. (Map es la interfaz hacia objetos StackTraceElement, que contiene el nombre del fichero, el número de línea, y el nombre de la clase y el método de la línea de código que se está ejecutando).
- getStackTrace(). Devuelve el seguimiento de la pila de un hilo en una aplicación.

Tanto getAllStackTraces() como getStackTrace() permiten grabar los datos del seguimiento de pila en un log.

A la hora de documentar una aplicación multihilo no debemos escatimar en comentarios. Si son importantes en cualquier aplicación, con más motivo en una aplicación multihilo, debido a su mayor complejidad.

Para documentar nuestra aplicación Java, utilizaremos el generador JavaDoc, que de forma automática genera la documentación de la aplicación a partir del código fuente. Como ya debes de saber si has estudiado el módulo de "Programación", este sistema consiste en incluir comentarios en el código, utilizando las etiquetas /** y */, que después pueden procesarse y generar un conjunto de páginas navegables HTML.

Te recordamos mediante el vídeo (debes tener instalado JavaDoc en tu equipo, cómo obtener la documentación de tu programa con JavaDoc y Netbeans a partir de tu código

fuente.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Estructuras concurrentes

Entre las colecciones hay que destacar:

- La interfaz Queue. Es una colección diseñada para almacenar elementos antes de procesarlos, ofreciendo diferentes operaciones como inserción, extracción e inspección.
- La interfaz BlockingQueue, diseñada para colas de tipo productor/consumidor, y que son thread-safe (aseguran un funcionamiento correcto de los accesos simultáneos multihilo a recursos compartidos). Son capaces de esperar mientras no haya elementos almacenados en la cola.
- Implementaciones concurrentes de Map y List.

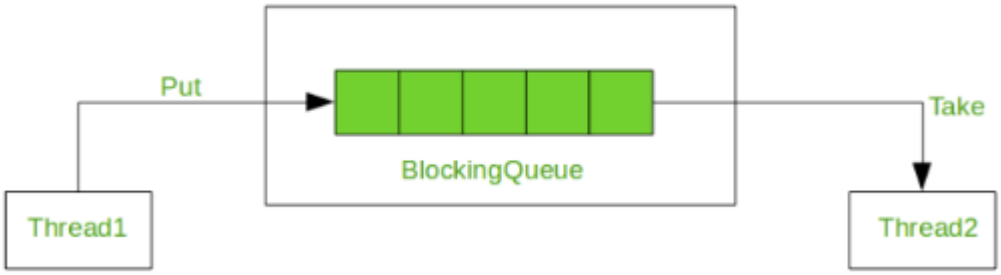
Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

BlockingQueue

La interfaz BlockingQueue de Java, java.util.concurrent.BlockingQueue, representa una cola que es segura para los hilos tanto para insertar elementos como para extraerlos. En otras palabras, varios hilos pueden estar insertando y extrayendo elementos de manera concurrente de un BlockingQueue en Java, sin que surjan problemas de concurrencia.

El término "cola bloqueante" se debe a que el BlockingQueue de Java es capaz de bloquear los hilos que intentan insertar o extraer elementos de la cola. Por ejemplo, si un hilo intenta tomar un elemento y no quedan elementos en la cola, el hilo puede bloquearse hasta que haya un elemento disponible para extraer. Si el hilo que llama se bloquea o no depende de los métodos específicos que se utilicen en BlockingQueue. Estos métodos se explicarán con más detalle posteriormente.

BlockingQueue no acepta valores nulos y lanza NullPointerException si intenta almacenar un valor nulo en la cola. Las implementaciones de Java BlockingQueue son seguras para subprocesos. Todos los métodos de cola son de naturaleza atómica y usan bloqueos internos u otras formas de control de concurrencia. La interfaz de Java BlockingQueue es parte del marco de las colecciones de Java y se usa principalmente para implementar el problema del consumidor del productor. No necesitamos preocuparnos por esperar a que el espacio esté disponible para el productor o el objeto para el consumidor en BlockingQueue porque lo manejan las clases de implementación de BlockingQueue.



Clases que implementan BlockingQueue

No es posible proporcionar directamente una instancia de BlockingQueue ya que es una interfaz, por lo que para implementar BlockingQueue necesitamos crear clases que la implementen. Aquí tienes algunas clases de implementación:

- ArrayBlockingQueue
- DelayQueue
- LinkedBlockingDeque

- LinkedBlockingQueue
- LinkedTransferQueue
- PriorityBlockingQueue
- SynchronousQueue

Estas clases ofrecen diferentes características y comportamientos para el manejo de colas bloqueantes en aplicaciones concurrentes, dependiendo de las necesidades específicas de la aplicación.

LinkedBlockingQueue y ArrayBlockingQueue son las clases utilizadas para implementar la clase BlockingDeque. Estas dos clases son una composición de BlockingDeque con una estructura de datos de lista enlazada y BlockingDeque con un arreglo, respectivamente.

- LinkedBlockingQueue: Se basa en una lista enlazada para almacenar los elementos, permitiendo una cola bloqueante con capacidad potencialmente ilimitada (o limitada si se especifica).
- ArrayBlockingQueue: Utiliza un arreglo de tamaño fijo para almacenar los elementos, proporcionando una cola bloqueante con capacidad fija.

Estas estructuras ofrecen opciones flexibles para el almacenamiento y manejo de datos en entornos concurrentes.

Métodos de la interfaz BlockingQueue

Existen tres categorías de métodos utilizados en la implementación de BlockingQueue:

1. Métodos que lanzan una excepción
    - add(): Inserta un elemento en el BlockingQueue al final de la cola. Lanza una excepción si la cola está llena.
    - element(): Devuelve el elemento superior o la cabeza de la cola. Lanza una excepción si la cola está vacía.
    - remove(): Elimina un elemento del BlockingQueue. Lanza una excepción si la cola está vacía.
2. Métodos que devuelven un valor
    - offer(): Inserta un elemento específico en el BlockingQueue en el extremo posterior de la cola. Devuelve false si la cola está llena. Este método también se puede utilizar con tiempos de espera, pasando las unidades de tiempo como un parámetro.
    - peek():Devuelve el elemento superior o cabeza del BlockingQueue. Retorna null si la cola está vacía.
    - poll():Elimina un elemento del BlockingQueue. Devuelve null si la cola está vacía. Este método también puede utilizarse con tiempos de espera, pasando las unidades de

tiempo como un parámetro.

3. Métodos que bloquean la operación
    - put(): Inserta un elemento en el BlockingQueue. Si la cola está llena, el método put() espera hasta que haya espacio disponible para insertar un elemento.
    - take(): Elimina y devuelve un elemento del BlockingQueue. Si la cola está vacía, el método take() espera hasta que haya elementos disponibles para ser eliminados.

Estos métodos son útiles para manejar operaciones de cola bloqueante en entornos concurrentes, especialmente cuando se necesitan tiempos de espera o bloqueos hasta que las condiciones de la cola cambien.



Este ejemplo demuestra cómo usar una `BlockingQueue` en Java para almacenar letras en una cola acotada y luego recuperar elementos de ella.

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ArrayBlockingQueue;

public class Alfabetos {
    public static void main(String[] args) {
        // Declaración de una BlockingQueue de naturaleza 'acotada' usando
ArrayBlockingQueue
        BlockingQueue colaAlfabetos = new ArrayBlockingQueue<>(7);

        try {
            // Insertar elementos en la cola bloqueante
            colaAlfabetos.put("A");
            colaAlfabetos.put("B");
            colaAlfabetos.put("C");
            colaAlfabetos.put("D");
            colaAlfabetos.put("E");
            colaAlfabetos.put("F");
            colaAlfabetos.put("G");

            System.out.println("Contenido de la BlockingQueue: " + colaAlfabetos);

            // Eliminar un elemento de la cola
            String temp = colaAlfabetos.take();
            System.out.println("El elemento eliminado es: " + temp);

            // Contenido de la BlockingQueue después de eliminar un elemento
```

```
            System.out.println("Contenido de la BlockingQueue después de eliminar
un elemento: " + colaAlfabetos);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

ArrayBlockingQueue

Java incluye la clase ArrayBlockingQueue (del paquete java.util.concurrent); una clase de búfer completamente implementada, segura para los subprocesos, que implementa a la interfaz BlockingQueue. Esta interfaz extiende a la interfaz Queue y declara los métodos put y take, los equivalentes con bloqueo de los métodos offer y poll de Queue, respectivamente. El método put coloca un elemento al final del objeto BlockingQueue, y espera si la cola está llena. El método take elimina un elemento de la parte inicial del objeto BlockingQueue, y espera si la cola está vacía. Estos métodos hacen que la clase ArrayBlockingQueue sea una buena opción para implementar un búfer compartido. Debido a que el método put bloquea hasta que haya espacio en el búfer para escribir datos, y el método take bloquea hasta que haya nuevos datos para leer. En el problema del productor-consumidor, el productor debe producir primero un valor, el consumidor sólo consume correctamente hasta después de que el productor escribe un valor, y el productor produce correctamente el siguiente valor (después del primero) sólo hasta que el consumidor lea el valor anterior (o primero). ArrayBlockingQueue almacena los datos compartidos en un arreglo. El tamaño de este arreglo se especifica como argumento para el constructor de ArrayBlockingQueue. Una vez creado, un objeto ArrayBlockingQueue tiene su tamaño fijo y no se expandirá para dar cabida a más elementos.



1. Clase Productor: Representa un productor que produce continuamente objetos de tipo DatoProducido y los coloca en una cola compartida (BlockingQueue). Utiliza el método cola.put(new DatoProducido()); para añadir datos a la cola. Si el hilo es interrumpido, se maneja la excepción InterruptedException, se decrementa el contador de producción y se establece la bandera de interrupción.
2. Clase Consumidor: Representa un consumidor que intenta continuamente recuperar datos de la cola con cola.poll(10, TimeUnit.MILLISECONDS);. Si obtiene un objeto, aumenta el contador de consumo. La condición if (dato != null) asegura que solo se cuente si realmente se obtuvo un dato de la cola. Si el hilo es interrumpido, se maneja la excepción InterruptedException, se decrementa el contador de consumo y se establece la bandera de interrupción.
3. Clase EjemploProductorConsumidor: Esta es la clase principal que crea la cola (ArrayBlockingQueue), y lanza los hilos Productor y Consumidor. Después de un segundo, interrumpe ambos hilos para detener la ejecución.

4. Clase DatoProducido: Es una clase vacía que actúa como un objeto de datos producido y consumido por los hilos.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Ejercicios

EJERCICIO 1: Realiza un programa que crea y ejecuta 10 hilos. Cada uno de esos hilos realiza una operación matemática de multiplicación de tal forma que multiplica un número recibido cuando se crea dicho hilo por 7, 8 y 9, imprimiendo el resultado de cada multiplicación.

EJERCICIO 2:Realice un ejercicio que crea 3 hilos y que cada uno de ellos vaya mostrando el contenido de un contador.Una posible salida en consola del ejercicio podría ser la siguiente:

```
CREANDO HILO:1
Hilo:1 contador=0
Hilo:1 contador=1
Hilo:1 contador=2
Hilo:1 contador=3
Hilo:1 contador=4
Hilo:1 contador=5
CREANDO HILO:2
CREANDO HILO:3
SE HAN CREADO 3 HILOS
Hilo:3 contador=0
Hilo:2 contador=0
Hilo:3 contador=1
Hilo:3 contador=2
Hilo:3 contador=3
Hilo:3 contador=4
Hilo:3 contador=5
Hilo:2 contador=1
Hilo:2 contador=2
Hilo:2 contador=3
Hilo:2 contador=4
Hilo:2 contador=5
```

EJERCICIO 3:Realice un ejercicio en el que se pueda crear hilos de dos formas:

- Una clase que herede de Thread.
- Una clase que implemente la interfaz Runnable.
- Los dos hilos imprimirán varias veces un texto. Este texto en el caso del hilo que hereda de Thread, mostrará el nombre de dicho hilo que se podrá implementar como un atributo. Es recomendable tener dos constructores para este hilo, uno para inicializar este atributo con un valor pasado como parámetro y otro que quede por defecto un valor asignado en la definición.
- Una posible salida en consola del ejercicio podría ser la siguiente:

```
Pepito
Hilo_SinNombre
Hilo_SinNombre
Pepito
Hilo_SinNombre
Hilo_SinNombre
Hilo_SinNombre
Pepito
Pepito
Pepito
  Hilo_Runnable
  Hilo_Runnable
  Hilo_Runnable
  Hilo_Runnable
  Hilo_Runnable
```

EJERCICIO 4:En este ejercicio se va a comprobar el estado de los hilos. El estado de los hilos es similar al estado de los procesos visto en teoría. Desde Java se nos permite consultar en el estado de un hilo usando la función getState(). El método isAlive() comprueba si el hilo está vivo devolviendo un boolean para indicarlo. Un hilo está vivo si está preparado para ejecutarse o se está ejecutando.

Crea un hilo que realice una cuenta atrás y consulta el estado de dicho hilo en distintas partes del código para comprobar por los estados en los pasa el hilo. Una posible salida en consola del ejercicio podría ser la siguiente:

```
Hilo Contador Nuevo: Estado=NEW¿Está vivo? isAlive()=false
Hilo Contador Iniciado: Estado=RUNNABLE¿Está vivo? isAlive()=true

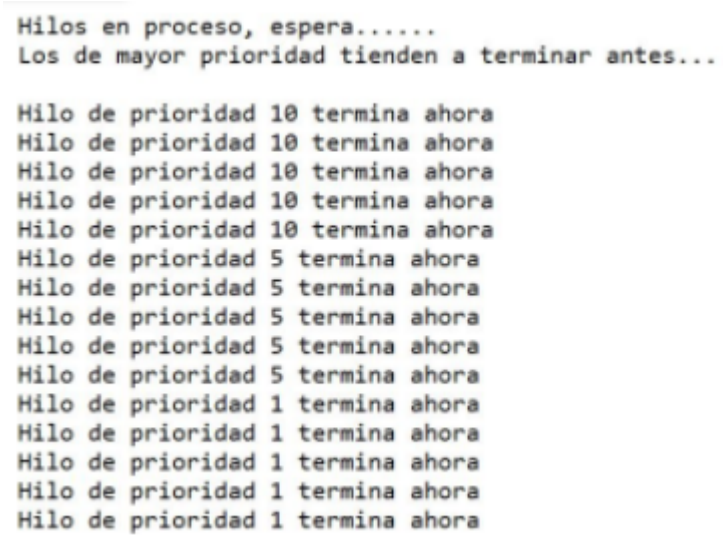
10,9,8,7,6,5,4,3,2,1,
 Hilo Contador Muerto: Estado=TERMINATED¿Está vivo? isAlive()=false
```

EJERCICIO 5: Realice un programa cuyo objetivo es crear 3 hilos con distinta prioridad y comprobar cómo afecta dicha prioridad al orden de ejecución. Para ello será necesario asignar distinta prioridad a cada uno de ellos usando las prioridades que Java nos ofrece por defecto. Estas prioridades son:

- Thread.MIN_PRIORITY
- Thread.NORM_PRIORITY
- Thread.MAX_PRIORITY

En caso de Thread.NORM_PRIORITY es el valor de prioridad que adquiere cualquier hilo al cual no se le indica ningún valor, es decir, es el valor por defecto.

Una posible solución sería crear 5 hilos de cada prioridad y lanzarlo en orden de prioridad ascendente para comprobar su funcionamiento como muestra la siguiente imagen.



EJERCICIO 6: Crear un programa que lance dos hilos. El primero de ellos imprimirá por la pantalla 10 veces la palabra “PSP” el segundo 10 veces la palabra “Aprobado”. Incluir un tiempo de retardo aleatorio entre sentencias de hasta 3 segundos.

EJERCICIO 7: Crear un programa que pida al usuario un número mayor de 10. Lanzar dos hilos:

- El primero realiza la cuenta creciente, es decir, imprime por pantalla los número del 0 al número introducido.
- El segundo realiza la cuenta decreciente, es decir, imprime por pantalla los números del número introducido al 0.
- Tiempo de retardo máximo 1 segundo.

Intenta parametrizar el límite de la cuenta de tal manera que se pueda pasar como parámetro en tiempo de ejecución.

EJERCICIO 8: Modifica las clases Productor y Consumidor vistas en clase para que el Productor envíe las cadenas PING y PONG (de forma alternativa, una vez PING y otra vez PONG) a la cola y la clase Consumidor tome la cadena de la cola y la visualice. La salida tiene que mostrar lo siguiente: PING PONG PING PONG PING PONG PING PONG PING PONG PING PONG PING PONG PING PONG PING PONG PING PONG PING PONG PING PONG PING PONG PING PONG...

EJERCICIO 9: Nos han encargado crear una aplicación para ser instalada en un parking que dispone de 4 plazas. Obviamente, el parking sólo podrá albergar 4 vehículos al mismo tiempo, haciendo esperar al resto de coches en la cola hasta que una de las plazas quede libre. Se

proporciona la clase principal, que crea 5 coches y un semáforo. Implementa la clase Coche para que imprima cuándo entra (indicando el nombre del coche) y sale del parking.

```
public class Parking {
public static void main(String[] args) {

// El parking tiene sólo 4 plazas libres
Semaphore semaforo = new Semaphore(4);

Coche coche1 = new Coche(semaforo, 100000, "SEAT Ibiza");
Coche coche2 = new Coche(semaforo, 5000, "FORD Fiesta");
Coche coche3 = new Coche(semaforo, 6000, "RENAULT Clio");
Coche coche4 = new Coche(semaforo, 9000, "BMW S1");
Coche coche5 = new Coche(semaforo, 8000, "MERCEDES E240");

//Orden de ejecución de hilos
coche1.start();
coche2.start();
coche3.start();
coche4.start();
//El quinto coche no podrá entrar en el parking
//hasta que uno de los 4 primeros no deje plaza libre.
coche5.start();
}

}
```

EJERCICIO 10: A partir del siguiente enunciado, implementa una solución. Puedes utilizar el código realizado hasta ahora en las actividades del productor consumidor y de la sincronización con monitores o elegir los semáforos. Se tiene una cuenta única en un banco. Se desean realizar depósitos y extracciones de la misma, bajo las siguientes condiciones:

● Garantizando que éstas (depósitos y extracciones) no se realicen de manera simultánea.

● Se tiene una cantidad inicial en la cuenta de 1000 euros.

● Se irán realizando extracciones e ingresos de cantidades aleatorias entre 0 y 1000, actualizando el saldo existente en la cuenta.

● El depósito no tiene restricciones pero el retiro sí, no se puede retirar una cantidad mayor a la existente en la cuenta. Es decir, el saldo nunca debe ser negativo.

Si quieres puedes introducir cualquier ampliación que se te ocurra (alguna mejora, etc.)

EJERCICIO 11:Escribe un programa que haga rebotar una pelota azul dentro de un JPanel. La pelota debe comenzar a moverse con un evento mousePressed. Cuando la pelota toque el borde del JPanel, debe rebotar en el borde y continuar en la dirección opuesta. La pelota debe actualizarse usando un Runnable.

EJERCICIO 12:Modifica el programa anterior para agregar una nueva pelota cada vez que el usuario haga clic con el ratón. Proporciona un mínimo de 20 pelotas. Elige aleatoriamente el color de cada nueva pelota.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0